Quantitative Risk Management
Fall 2025
Professor
Dr. Urban Ulrych

Project 1

Group

VELAY Mahé	345882
ROBERT Adélaïde	346471
POTTIÉ Timothée	327549

Introduction

PETITE INTRO STYLEE Rapide

Contents

1 Empirical Stylized Facts **3**
1.1 Log-Returns . . . 3
1.2 Cross-Correlation And Autocorrelation . . . 3
1.3 Normality Assessment of Log-Returns . . . 6

2 Single-window modeling: VaR, ES, and distributions **8**
2.1 Historical Simulation . . . 8
2.2 Gaussian . . . 8
2.3 Student-t . . . 9
2.4 Conditional parametric . . . 9
2.5 Filtered Historical Simulation (FHS) . . . 12
2.6 Comparison of models . . . 13

3 Backtesting VaR and ES **17**
3.1 Rolling–window modelling . . . 17
3.2 Backtesting VaR: Kupiec POF and Christoffersen Conditional Coverage Tests . . 19
3.3 Expected Shortfall Backtest : Acerbi and Székely Z_1 test . . . 24
3.4 Overall of backtest . . . 27

4 Copula Fitting **30**

4.1 Pseudo-observations and empirical dependence 30
4.2 Gaussian and Student-t copula estimation 31
4.3 Simulation from copulas and comparison of marginal distributions 32
4.4 Visualization 33
4.5 Summary 36

5 Backtesting equally weighted portfolio VaR and ES 38
5.1 Modelling the portfolio VaR and ES with rolling window 38
5.2 Backtest on the Portfolio 39
5.3 Copula Implementation for Portfolio VaR/ES 41
5.4 Comparison of Models and Impact of Copulas 43

A Commented Code 45
A.1 Main implementation 45

1 Empirical Stylized Facts

1.1 Log-Returns

We begin by examining daily log-returns, a standard transformation of price data that offers convenient statistical properties and highlights the fundamental dynamics of financial time series, including volatility fluctuations and extreme events.

$$R_t = log(\frac{P_t}{P_t - 1})$$

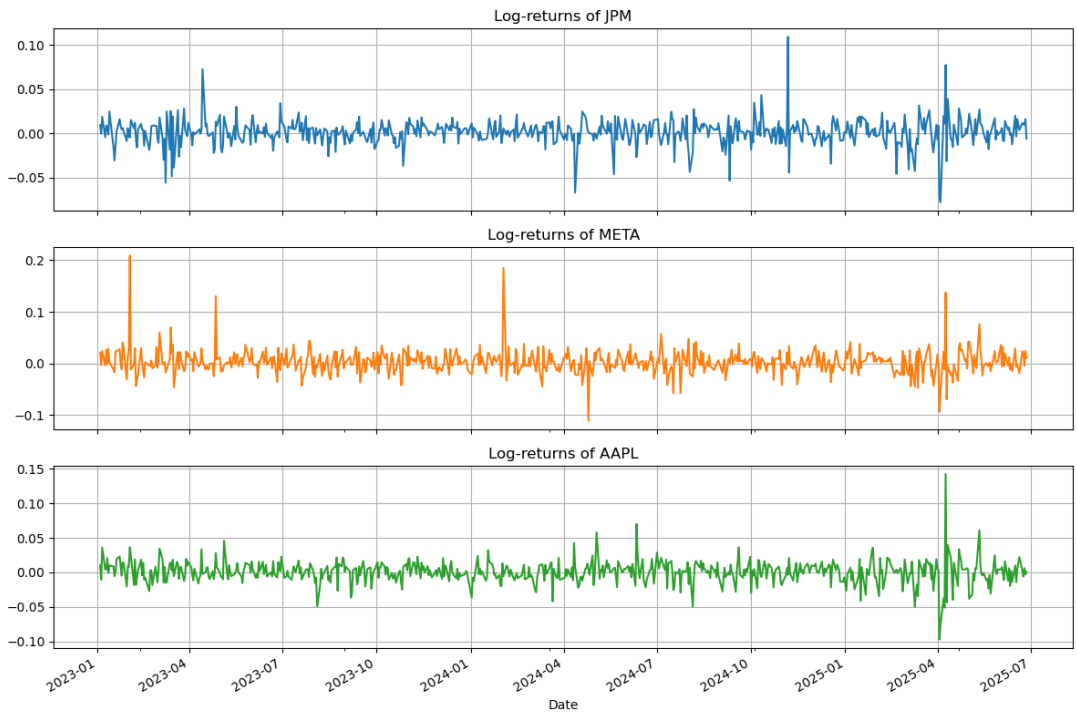


Figure 1: Daily log-returns of JPM, META, and AAPL from 2023 to mid-2025.

Comments

The log-return series of JPM, META, and AAPL fluctuate around zero with no actual trend, consistent with the usual assumption that returns are approximately mean-zero and stationary.

However, the amplitude of the fluctuations is far from constant: all three assets exhibit clear **volatility clustering**, with periods of low volatility are followed by period of low volatility and vice-versa.

META shows the highest overall volatility, while AAPL and JPM display smaller but still noticeable spikes, including a few extreme return observations. These features suggest **time-varying volatility** and **fat-tailed** behavior, motivating the use of models such as Student-t distributions and GARCH-type dynamics in subsequent sections.

1.2 Cross-Correlation And Autocorrelation

Cross-Correlation

We first compute the cross-correlation of daily log-returns. For two return series $r_t^{(i)}$ and $r_t^{(j)}$,

the empirical cross-correlation at lag k is defined as

$$\rho_{ij}(k) = \mathrm{Corr}\left(r_t^{(i)},\, r_{t-k}^{(j)}\right),$$

which measures the linear dependence between the returns of stock j shifted by k days and the contemporaneous returns of stock i, (i, j = AAPL, META, JPM.)

We compute this quantity for lags from 0 to 25 days, both for the raw log-returns and for their absolute values, in order to capture dependence in volatility as well as dependence in direction.

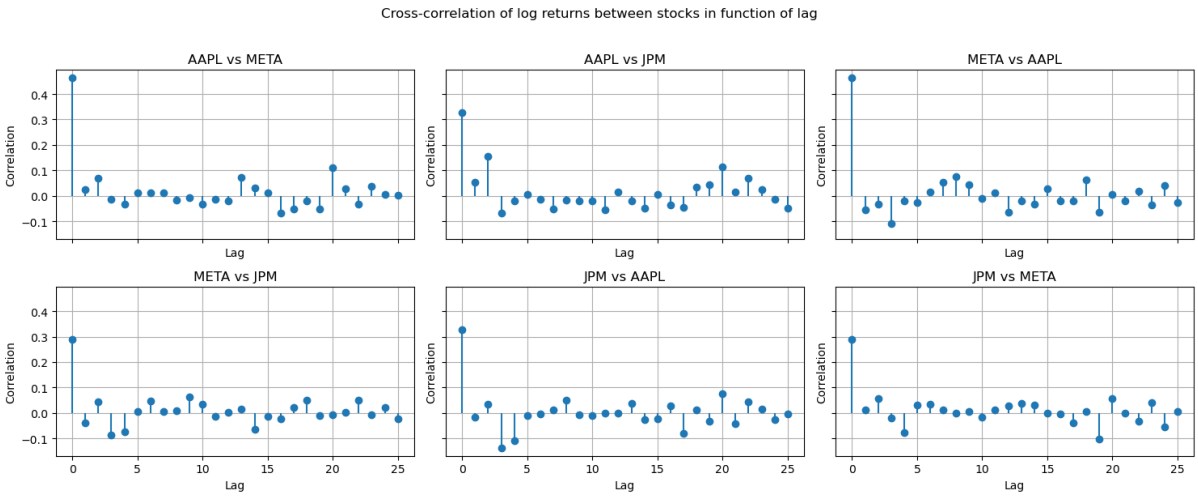


Figure 2: Cross-correlation of daily log-returns between AAPL, META, and JPM for lags from 0 to 25 days.

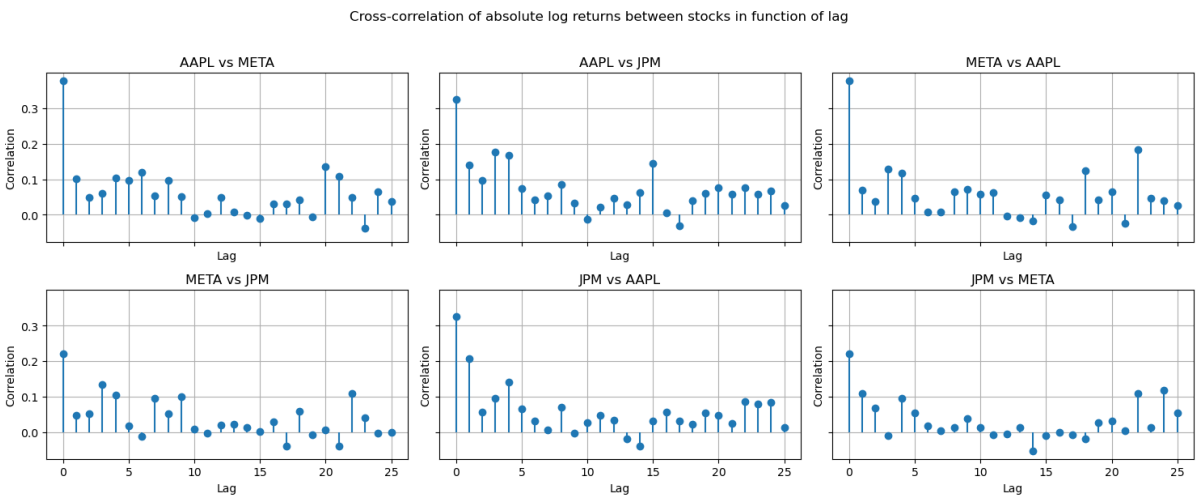


Figure 3: Cross-correlation of absolute daily log-returns between AAPL, META, and JPM across lags from 0 to 25 days

Comments on Cross-Correlations

- **Returns and diversification.** The cross-correlations of raw log-returns are small at all lags. This weak dependence implies that, at the return level, the assets do not move perfectly together, so diversification benefits are present.
- **Co-movement of volatility.** The cross-correlations of absolute log-returns are clearly

positive at lag 0 and remain positive for several lags. This indicates that volatility shocks tend to be shared across assets: when one stock is volatile, the others are more likely to be volatile as well.

- **Lead–lag effects.** Beyond lag 0, cross-correlations for both returns and absolute returns remain low and do not show a systematic pattern. This suggests that, in our sample, past movements in one asset do not provide strong predictive power for future movements in another asset at short horizons.

Autocorrelation

We also compute the autocorrelation of each stock's daily log-returns in order to examine the persistence of their own past movements. The empirical autocorrelation at lag k is simply

$$\rho_i(k) = \mathrm{Corr}\left(r_t^{(i)},\, r_{t-k}^{(i)}\right),$$

which quantifies how strongly the returns of stock i are related to their own values k days earlier.

It is also computed for lags from 0 to 25 days, using both the raw log-returns and their absolute values.

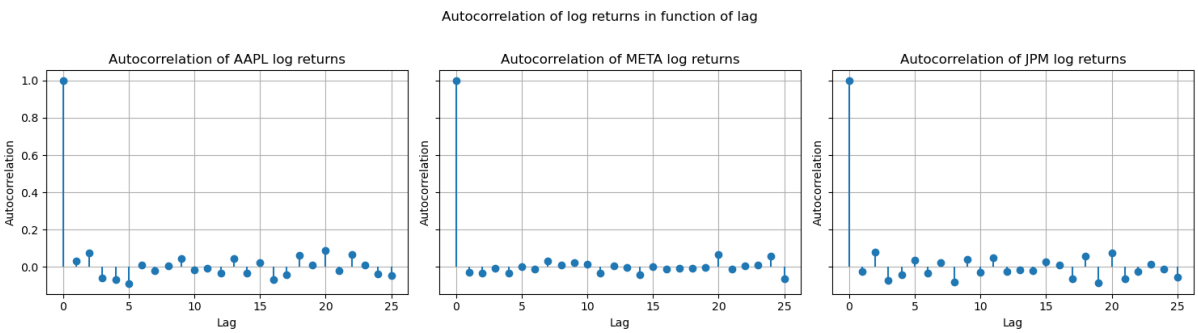


Figure 4: Autocorrelation of daily log-returns for AAPL, META, and JPM across lags from 0 to 25 days.

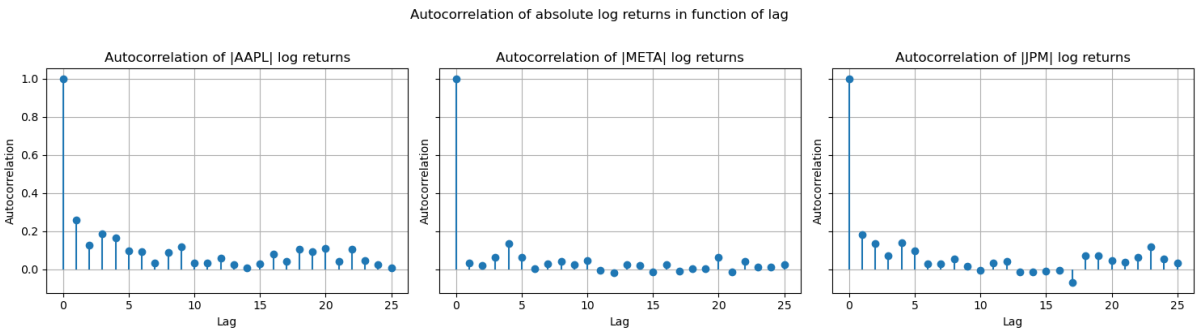


Figure 5: Autocorrelation of absolute daily log-returns for AAPL, META, and JPM across lags from 0 to 25 days.

Comments on Autocorrelations

- **Lack of return predictability.** The autocorrelation of raw log-returns is close to zero for all lags and all assets. This indicates that past returns provide little information

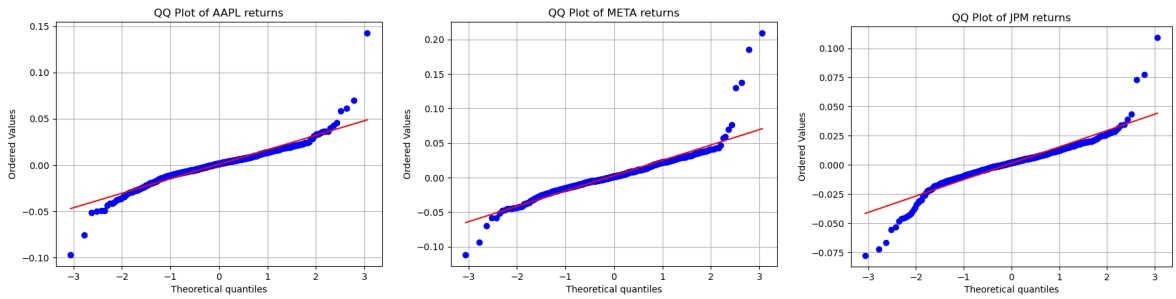
about future returns, which is consistent with weak-form market efficiency and the idea that price changes are largely unpredictable.

- **Volatility clustering.** In contrast, the autocorrelation of absolute log-returns is strongly positive at short lags and decays slowly. This pattern reflects the phenomenon of volatility clustering that we talked before: periods of high volatility tend to be followed by high volatility, and vice-versa. The persistence observed across multiple lags provides support for models with time-varying volatility such as GARCH.
- **Common behaviour across assets.** Although the magnitudes differ slightly, all three assets exhibit a similar contrast: no significant autocorrelation in returns, but clear persistence in absolute returns. This reinforces the idea that unpredictability of direction coexists with predictability of volatility.

1.3 Normality Assessment of Log-Returns

To evaluate whether daily log-returns are consistent with a normal distribution, we first look at QQ-plots to compare the empirical quantiles of the return series with the theoretical quantiles of a normal distribution.

We complement this with the Jarque–Bera test. The test evaluates the null hypothesis that the data follow a normal distribution. A large test statistic (and small p-value) indicates a rejection of normality.



(a) AAPL (b) META (c) JPM

Figure 6: QQ-plots of daily log-returns against the standard normal distribution.

The QQ-plots of the three assets returns all deviate from the straight line in the tails.

- Both lower and upper tails lie outside the theoretical normal quantiles, indicating much heavier tails than the Gaussian distribution.
- The central part (middle quantiles) aligns reasonably well with the normal line, but the divergence becomes large for extreme quantiles.

These plots provides strong visual evidence against normality. They confirm that returns distributions are fat tailed and that Gaussian based distribution will underestimate tail risk.

Stock	JB statistic	p-value
JPM	2203.50	0
META	7608.78	0
AAPL	3619.29	0

Table 1: Jarque–Bera normality test results for daily log-returns.

The Jarque–Bera statistics for all three assets are extremely large (JPM = 2203, META = 7609, AAPL = 3619), and all p-values are effectively zero. This means we strongly reject the null hypothesis of normality for every return series.

Overall, the JB test statistically validates the visual evidence: daily returns are not Gaussian, especially in the tails, and normal-based risk models will underestimate extreme losses.

2 Single-window modeling: VaR, ES, and distributions

We now estimate 1-day ahead risk measures using a single estimation window of $W = 252$ observations. Working with losses $L_t = -R_t$, we fit several distributional models (both parametric and non-parametric) to compute and compare VaR and ES at the 95% and 99% levels.

2.1 Historical Simulation

This is a non-parametric method that relies directly on the empirical distribution of past losses. Given the series of realized losses $\{L_t\}_{t=1}^{W}$, the HS Value-at-Risk at confidence level α is defined as the empirical α-quantile of the sample,

$$VaR_\alpha^{\text{HS}} = \hat{q}_\alpha(L),$$

The corresponding Expected Shortfall is obtained as the average of all losses exceeding this VaR level,

$$ES_\alpha^{\text{HS}} = \frac{1}{\#\{t : L_t \geq \ VaR_\alpha^{\text{HS}}\}} \sum_{t:L_t \geq \ VaR_\alpha^{\text{HS}}} L_t.$$

We apply this procedure to each asset (AAPL, META, JPM) and for confidence levels $\alpha \in \{0.95, 0.99\}$. Therefore we get :

	$\alpha = 0.95$		$\alpha = 0.99$	
Ticker	$\text{VaR}_{0.95}$	$\text{ES}_{0.95}$	$\text{VaR}_{0.99}$	$\text{ES}_{0.99}$
AAPL	0.017909	0.027557	0.036446	0.042831
META	0.027839	0.036607	0.043652	0.045113
JPM	0.018842	0.031276	0.043159	0.052027

Table 2: Historical Simulation 1-day VaR and ES at confidence levels 95% and 99%.

2.2 Gaussian

We now consider the parametric Gaussian approach, which models the loss series L_t as normally distributed with mean μ and standard deviation σ. Under the normality assumption, the α–level Value-at-Risk is given by

$$VaR_\alpha^{\mathcal{N}} = \mu + \sigma\, z_\alpha, \qquad z_\alpha = \Phi^{-1}(\alpha),$$

where Φ^{-1} denotes the inverse standard normal cdf. The corresponding Expected Shortfall has the closed-form expression

$$ES_\alpha^{\mathcal{N}} = \mu + \sigma\, \frac{\varphi(z_\alpha)}{1-\alpha},$$

with φ the standard normal density.
The results of the fitted parameters and of the VaR and ES are :

Asset	$\hat{\mu}$	$\hat{\sigma}$
AAPL	-0.001509	0.012724
META	-0.004061	0.023889
JPM	-0.001085	0.012996

Table 3: Gaussian fit: estimated mean and standard deviation of daily losses L_t.

	$\alpha = 0.95$		$\alpha = 0.99$	
Asset	$\mathrm{VaR}_{0.95}$	$\mathrm{ES}_{0.95}$	$\mathrm{VaR}_{0.99}$	$\mathrm{ES}_{0.99}$
AAPL	0.019421	0.024738	0.028092	0.032404
META	0.035232	0.045215	0.051512	0.059608
JPM	0.020292	0.025723	0.029149	0.033553

Table 4: Parametric Gaussian 1-day VaR and ES at confidence levels 95% and 99%.

2.3 Student-t

We now assume that daily losses follow a scaled and shifted Student-t distribution,

$$L_t = \mu + \sigma X_t, \qquad X_t \sim t_\nu,$$

with location μ, scale $\sigma > 0$ and degrees of freedom $\nu > 2$. For a confidence level α, let $q_{\nu,\alpha}$ be the α-quantile of the standard t_ν distribution and f_ν its density. The one–day VaR and ES of L_t are then

$$VaR_\alpha = \mu + \sigma\, q_{\nu,\alpha}, \qquad ES_\alpha = \mu + \sigma\, \frac{f_\nu(q_{\nu,\alpha})}{1-\alpha}\, \frac{\nu + q_{\nu,\alpha}^2}{\nu - 1}.$$

We estimate (ν, μ, σ) by maximum likelihood for each asset and use the formulas above to obtain VaR_α and ES_α at the 95% and 99% levels. The results are given by :

Asset	$\hat{\nu}$	$\hat{\mu}$	$\hat{\sigma}$
AAPL	6.927440	-0.001537	0.010747
META	4.391324	-0.002611	0.015704
JPM	3.514470	-0.001640	0.008801

Table 5: Fitted Student-t parameters (ν, μ, σ) for each asset.

	$\alpha = 0.95$		$\alpha = 0.99$	
Asset	$\mathrm{VaR}_{0.95}$	$\mathrm{ES}_{0.95}$	$\mathrm{VaR}_{0.99}$	$\mathrm{ES}_{0.99}$
AAPL	0.018856	0.026424	0.030775	0.039146
META	0.030027	0.045386	0.053435	0.073638
JPM	0.017892	0.028770	0.034002	0.050030

Table 6: Student-t parametric estimates of VaR and ES for each asset.

2.4 Conditional parametric

Selection of the AR order p.

To choose the order of the autoregressive model AR(p) we rely on the partial autocorrelation function (PACF) of the log–return series. For each asset and confidence level $1 - \alpha \in \{95\%, 99\%\}$, the PACF is estimated up to lag 25 , together with $(1 - \alpha)$ confidence intervals $\left[\mathrm{CI}_{\mathrm{low}}(k), \mathrm{CI}_{\mathrm{up}}(k)\right]$.

A lag $k \geq 1$ is deemed *significant* if its confidence interval does not contain zero,

$$0 \notin \left[\mathrm{CI}_{\mathrm{low}}(k), \mathrm{CI}_{\mathrm{up}}(k)\right],$$

while lag 0 is ignored (its PACF is always 1). Starting from lag $k = 1$ we count how many consecutive lags are significant; as soon as we encounter the first non-significant lag, we stop. The number of such consecutive significant lags defines the selected AR order p for the corresponding asset and confidence level.

Note that if lag k=1 is not significant, then the order p of the AR model will be 0.

Here are the plots showing the PACF across lags and the confidence intervall.

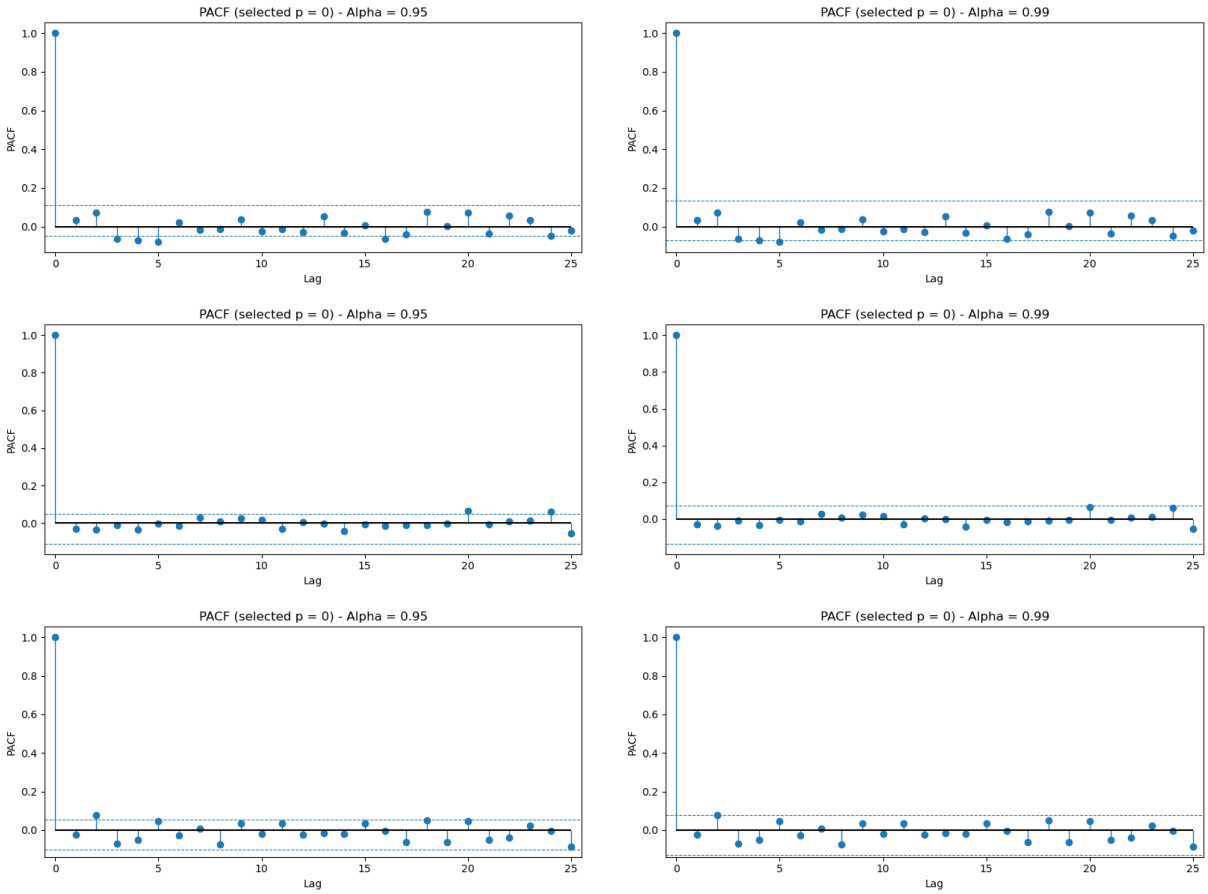


Figure 7: PACF of daily log-returns with CI bands im function of lag

Using the PACF "first run of significant lags" rule with 95% and 99% confidence bands, no lag beyond 0 is significant for any asset; hence the selected mean model is AR(p) with $p = 0$ across assets and both confidence levels (see Table).

	$\alpha = 0.95$	$\alpha = 0.99$
Asset	$p_{0.95}$	$p_{0.99}$
JPM	0	0
META	0	0
AAPL	0	0

Table 7: Selected AR order p by asset and confidence level.

These selections (all $p = 0$) are consistent with the PACF panels: aside from the trivial spike at lag 0, all subsequent partial autocorrelations lie within the confidence bands, indicating no

meaningful serial correlation in returns and justifying an AR(0) mean specification across assets and levels.

Fitting the AR(p)–GARCH(1,1) Model

The aim is to estimate the conditional one day ahead distribution for each asset and confidence level by combining a mean model AR(p) and a volatility model GARCH(1,1). From this fitted model, we get the one-step forecasts of the conditional mean and variance,

$$\mu_{t+1|t} = \mathbb{E}[R_{t+1} \mid \mathcal{F}_t], \qquad \sigma^2_{t+1|t} = \mathbb{V}[R_{t+1} \mid \mathcal{F}_t],$$

and compute parametric VaR and ES for the *loss* $L_{t+1} = -R_{t+1}$ under both Gaussian and Student-t innovations.

Model Description

We use

$$R_t = \mu_t + \sigma_t Z_t, \qquad \mu_t = c + \sum_{j=1}^{p} \phi_j R_{t-j}, \qquad \sigma_t^2 = \omega + \alpha R_{t-1}^2 + \beta\, \sigma_{t-1}^2,$$

estimated by MLE.

We scale returns (e.g., $\times 100$) for numerical stability: tiny raw magnitudes make the optimizer ill-conditioned and slow to converge. This improves estimation without changing the shape parameters (ϕ, α, β); forecasts are then de-scaled back to original units.

Diagnostics.

On standardized residuals $\hat{Z}_t = \hat{\varepsilon}_t / \hat{\sigma}_t$ we apply Ljung–Box tests:

- On $\hat{Z}_t$: checking remaining linear autocorrelation in the mean
- On $\hat{Z}_t^2$: checking remaining volatility clustering/ARCH.

Non-rejection at conventional levels supports an adequate AR–GARCH specification.

Risk measures from the conditional forecast.

With $L_{t+1} = -R_{t+1}$ and innovation Z:

$$\mathrm{VaR}_\alpha(L_{t+1} \mid \mathcal{F}_t) = -\mu_{t+1|t} + \sigma_{t+1|t}\, q_\alpha(Z), \qquad \mathrm{ES}_\alpha(L_{t+1} \mid \mathcal{F}_t) = -\mu_{t+1|t} + \sigma_{t+1|t}\, \mathrm{ES}_\alpha(Z),$$

where $q_\alpha(Z)$ is the α-quantile of Z.

- Gaussian innovations: $Z \sim \mathcal{N}(0,1)$, $q_\alpha(Z) = \Phi^{-1}(\alpha)$, $\quad \mathrm{ES}_\alpha(Z) = \dfrac{\varphi(\Phi^{-1}(\alpha))}{1-\alpha}$.
- Student-t innovations: $Z \sim t_\nu$. We estimate ν from the standardized residuals. Using the variance-normalized t (so that $\mathrm{Var}\, Z = 1$), the ES factor is

$$\mathrm{ES}_\alpha(Z) = \frac{f_\nu(t_{\nu,\alpha})\,(\nu + t_{\nu,\alpha}^2)}{(\nu-1)\,(1-\alpha)}, \quad t_{\nu,\alpha} = F_\nu^{-1}(\alpha),$$

  where f_ν and F_ν are the pdf and cdf of t_ν.

Results

Asset	p	$\mu_{t+1\|t}$	$\sigma_{t+1\|t}$
AAPL	0	0.001273	0.011860
META	0	0.002911	0.014640
JPM	0	0.001525	0.008995

Table 8: AR(p)–GARCH(1,1) one-step forecasts.

	LB $Q(10)$ p-value		LB $Q(25)$ p-value	
Asset	mean	squared	mean	squared
AAPL	0.551	0.977	0.381	0.685
META	0.942	0.999	0.841	1.000
JPM	0.152	0.741	0.158	0.138

Table 9: Ljung–Box p-values on standardized residuals up to lag m (mean) and on their squares (ARCH check). $Q(m)$ tests the joint null $\rho_1 = \cdots = \rho_m = 0$.

		$\alpha = 0.95$		$\alpha = 0.99$	
Asset	Method	$\mathrm{VaR}_{0.95}$	$\mathrm{ES}_{0.95}$	$\mathrm{VaR}_{0.99}$	$\mathrm{ES}_{0.99}$
AAPL	Gaussian	0.01823	0.02318	0.02631	0.03033
META	Gaussian	0.02118	0.02729	0.03115	0.03612
JPM	Gaussian	0.01327	0.01703	0.01940	0.02245

Table 10: One–day conditional risk measures from AR(p)–GARCH(1,1) with Gaussian innovations.

Discussion

PACF selection yields $p = 0$ for all assets. The Ljung–Box tests on standardized residuals return large p-values at both horizons. These indicate no statistically significant leftover linear autocorrelation or ARCH effects up to lag 25. Overall, the AR(0)–GARCH(1,1) specification appears adequate.

Table 9 reports the conditional mean and volatility. As expected for daily data, $\mu_{t+1|t}$ is small across assets, while the conditional standard deviations differ markedly. This ordering largely explains the cross-section of conditional risk.

Table 10 shows one–day VaR and ES under Gaussian and Student-t innovations. Monotonicity holds: for each asset, $\mathrm{VaR}_{0.99} > \mathrm{VaR}_{0.95}$ and ES > VaR. At a given confidence level, Student-t innovations systematically produce larger tail risk than Gaussian, in line with heavier conditional tails of the standardized residuals.

2.5 Filtered Historical Simulation (FHS)

FHS is attractive because it keeps today's volatility state (via a parametric filter) while letting the data determine tail shape—so it adapts to regimes and heavy tails without assuming Gaussian shocks.

We first fit an AR(p)–GARCH(1,1) on the recent returns to obtain the one–day conditional mean and volatility $(\hat{\mu}_{t+1|t}, \hat{\sigma}_{t+1|t})$ and the standardized residuals (innovations).

Instead of imposing a distribution, we bootstrap these standardized innovations, then scale by $\hat{\sigma}_{t+1|t}$ and shift by $\hat{\mu}_{t+1|t}$ to generate a conditional loss distribution $L^* = -R^*$.

VaR and ES at the desired level are then read directly from this simulated conditional distribution.

	$\alpha = 0.95$		$\alpha = 0.99$	
Asset	$\text{VaR}_{0.95}$	$\text{ES}_{0.95}$	$\text{VaR}_{0.99}$	$\text{ES}_{0.99}$
AAPL	0.015900	0.024066	0.029762	0.037432
META	0.019442	0.025276	0.030197	0.032733
JPM	0.014206	0.019739	0.022889	0.031435

Table 11: Filtered Historical Simulation (FHS) one–day conditional risk measures.

Results (summary). We report one–day FHS VaR and ES by asset and confidence level in Table 11 Magnitudes align with the forecasted conditional volatility (higher $\hat{\sigma}_{t+1|t} \Rightarrow$ larger risk), and, as expected, ES > VaR and the 99% measures exceed the 95% measures.

2.6 Comparison of models

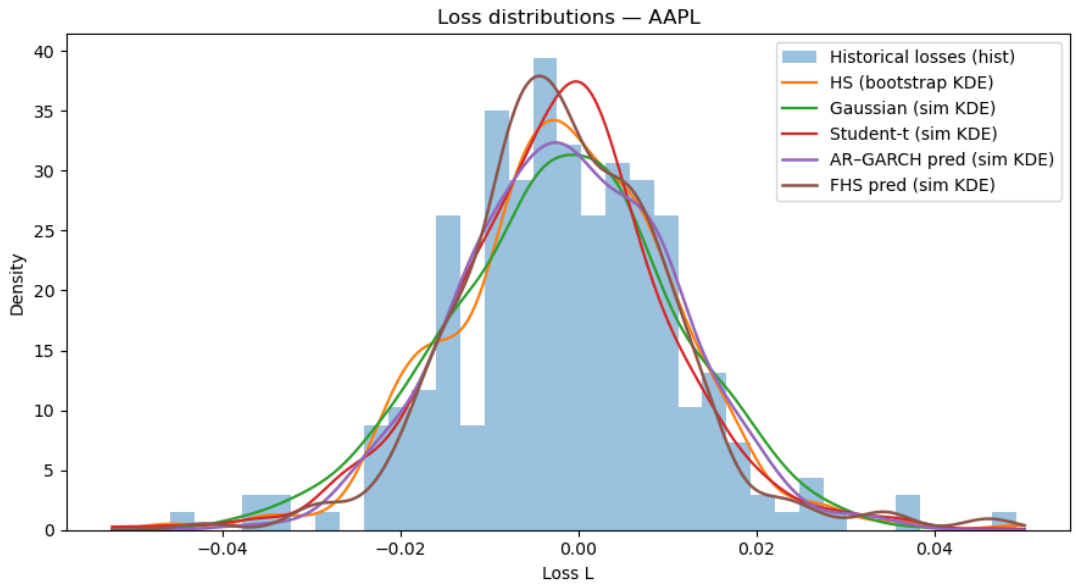


Figure 8: AAPL — Loss distributions with historical histogram and conditional overlays.

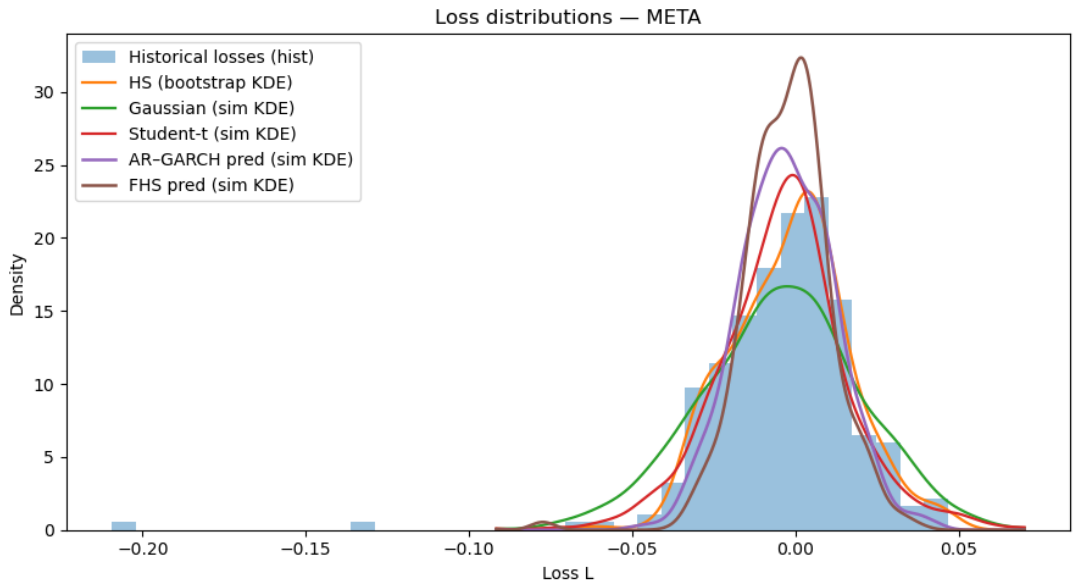


Figure 9: META — Loss distributions with historical histogram and conditional overlays.

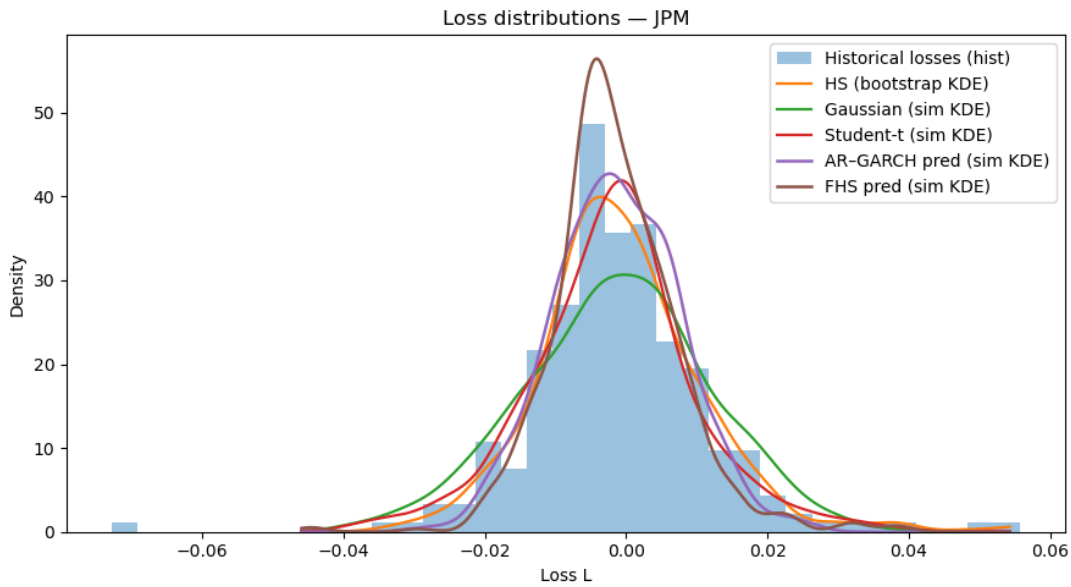


Figure 10: JPM — Loss distributions with historical histogram and conditional overlays.

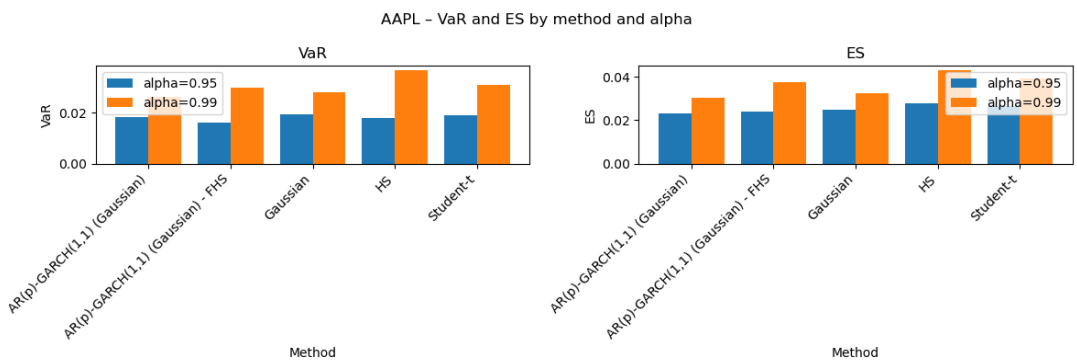


Figure 11: AAPL — One–day VaR and ES by method at confidence levels 95% and 99%.

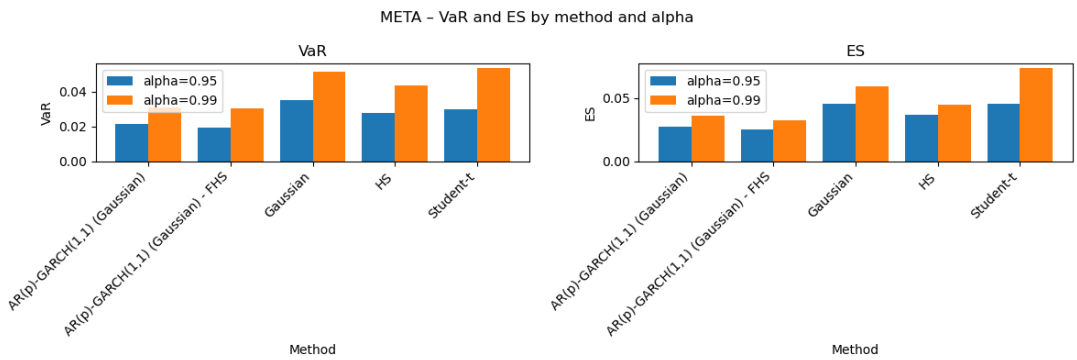


Figure 12: META — One–day VaR and ES by method at confidence levels 95% and 99%.

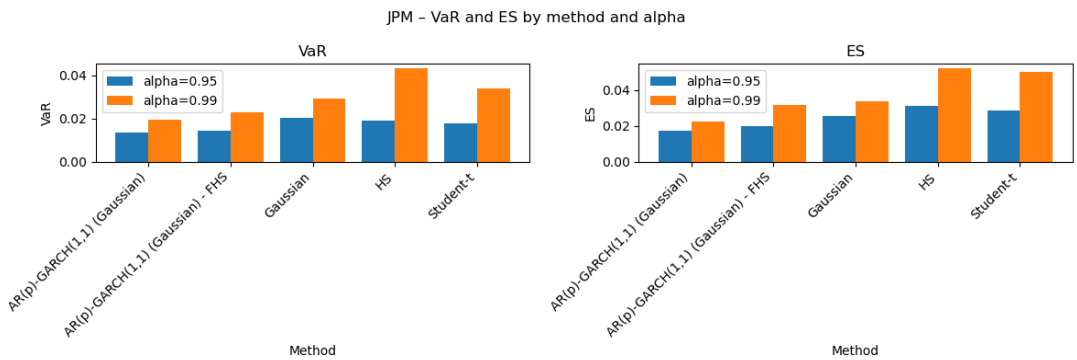


Figure 13: JPM — One–day VaR and ES by method at confidence levels 95% and 99%.

Interpretation

- Across assets
  - VaR/ES at 99% > 95%.
  - Risk ordering is stable: Gaussian lowest, Student-t and FHS highest; HS and AR-GARCH in between.
  - Density overlays: Gaussian is too light-tailed; Student-t/FHS fit the empirical left tail best.
- AAPL
  - Student-t/FHS > Gaussian/AR-GARCH for both VaR and ES; consistent with ES backtest outcomes.
- META
  - Ordering: Student-t/FHS $\gtrsim$ HS $\geq$ AR-GARCH > Gaussian; tails moderate but heavier than Gaussian.
- JPM
  - Heaviest tails; largest gap between Gaussian/AR-GARCH and Student-t/FHS.

Takeaways Heavy-tailed specifications (Student-t, FHS) better capture loss tails and yield more prudent VaR/ES; Gaussian (i.i.d. or in AR-GARCH) is systematically light-tailed. Asset

heterogeneity matters: JPM is the most tail-heavy, AAPL intermediate, META the least.

Numerical implementation considerations

Building block 1: `forecast_model`
For each model we implement a function of the form

$$\texttt{forecast_model}(\text{window}, \alpha) \longrightarrow (\widehat{\text{VaR}}_\alpha, \widehat{\text{ES}}_\alpha, \texttt{params}),$$

where:

- `window` is the estimation window (typically the losses $L_{t-W+1:t}$; for GARCH-type models we fit on returns $R_{t-W+1:t}$),
- $\alpha \in \{0.95, 0.99\}$ is the confidence level,
- $\widehat{\text{VaR}}_\alpha$ and $\widehat{\text{ES}}_\alpha$ use the convention $\text{VaR}_\alpha = q_\alpha(L)$ and $\text{ES}_\alpha = \mathbb{E}[L \mid L \geq \text{VaR}_\alpha]$,
- `params` is a dictionary storing everything needed to reconstruct (and later simulate from) the predicted distribution of L_{t+1}.

This block is used directly in Question 2 (single-window VaR/ES estimation) and is the core ingredient for building rolling forecasts in Question 3.

Examples of `params`.

- Historical Simulation (HS): `params` stores the list of window losses (used for bootstrap resampling).
- Gaussian: `params` stores (μ, σ) fitted on window losses L.
- Student-t: `params` stores (ν, μ, σ) fitted on window losses L.
- AR–GARCH: the model is fitted on returns R, we extract the conditional forecast $R_{t+1} \mid \mathcal{F}_t \sim \mathcal{N}(\mu_{t+1}, \sigma^2_{t+1})$, and store $(\mu_{t+1}, \sigma_{t+1})$ to simulate $L_{t+1} = -R_{t+1}$.
- Filtered Historical Simulation (FHS): we store $(\mu_{t+1}, \sigma_{t+1})$ and a pool of standardized residuals $\{z\}$ so that $R_{t+1} = \mu_{t+1} + \sigma_{t+1} z^\star$ with $z^\star$ bootstrapped, hence $L_{t+1} = -R_{t+1}$.

Building block 2: `simulate_one_step_model`
In the next question, we'll backtest the ES risk value. ES backtesting (Acerbi-Székely Z_1 test) requires Monte Carlo sampling from the *forecast* distribution at each time step. Hence, for each model we implement

$$\texttt{simulate_one_step_model}(\texttt{params}, M, \texttt{rng}) \longrightarrow \{L^{(m)}_{t+1}\}_{m=1}^{M},$$

where M is the number of simulations and `rng` is a pseudo-random generator.
This is useful beyond the Z_1 test: it enables diagnostic plots of the predictive distributions in this question and ensures internal consistency: the same `params` used to compute VaR/ES are also used when generating Monte Carlo scenarios.

3 Backtesting VaR and ES

In this part, the objective is to evaluate the out of sample performance of the VaR and ES models we developed in the part above. The goal is to asses wheras each model assess accurate risk forecast when confronted with new market data.

3.1 Rolling–window modelling

We adopt a rolling–window procedure with window size $W = 252$. At each date t we estimate the model using only the most recent W returns, produce a 1-day-ahead forecast for $t+1$, then advance the window by one day and repeat:

$$\underbrace{\{R_{t-W+1}, \ldots, R_t\}}_{\text{estimation sample}} \xrightarrow{\text{fit model}} (\widehat{q}_{t,\alpha},\, \widehat{e}_{t,\alpha}) \xrightarrow{\text{forecast}} \text{VaR/ES for } t{+}1.$$

After observing R_{t+1}, the window shifts to $\{R_{t-W+2}, \ldots, R_{t+1}\}$ and the process continues until the end of the out-of-sample period, yielding a sequence of out-of-sample forecasts $\{(\widehat{q}_{t,\alpha}, \widehat{e}_{t,\alpha})\}$ aligned with realized returns.

Using a rolling-window allow us to control for:

- *Non-stationarity.* Financial volatility and tail behavior evolve over time; re-estimating on the last W points lets forecasts adapt to regime changes.
- *Real-time information set.* Using only data available up to t avoids look-ahead bias and mimics how risk is produced in practice.

Results

Each panel plots realized one–day losses (black) and 1-step-ahead VaR at level α from HS, Gaussian, Student-t, AR-GARCH, and FHS. Exceptions occur when the black line drops below a model's VaR curve. The bottom panel uses $\alpha = 0.99$ (higher thresholds).

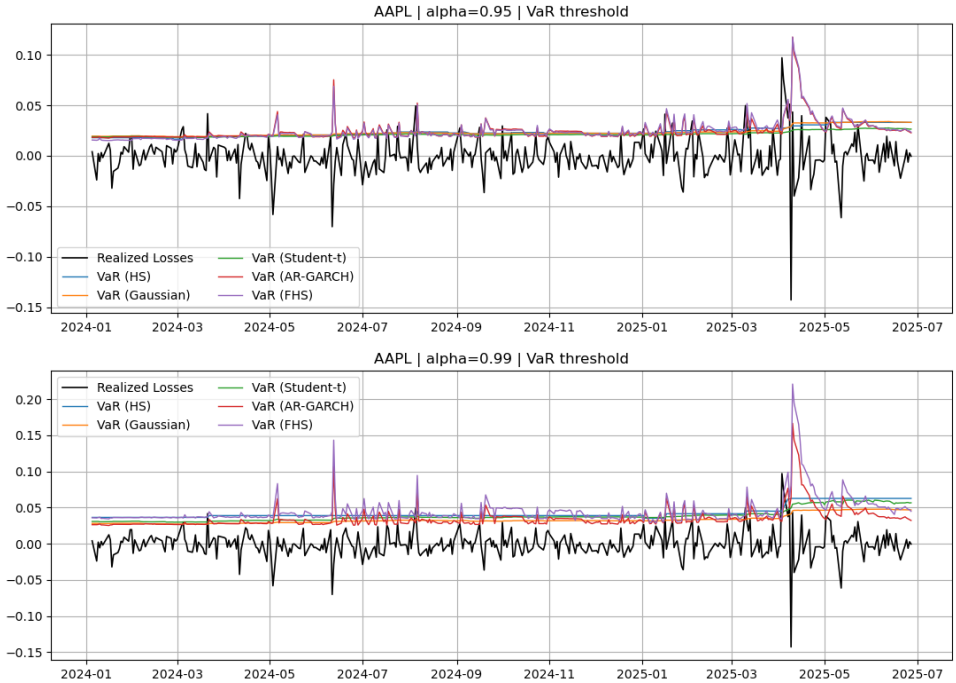


Figure 14: AAPL: At 95% (top) and 99% (bottom), a spring-2025 stress episode produces the largest losses. Student-t and FHS raise VaR most in stress, while Gaussian and AR-GARCH sit lower and show more crossings, especially at 99%.

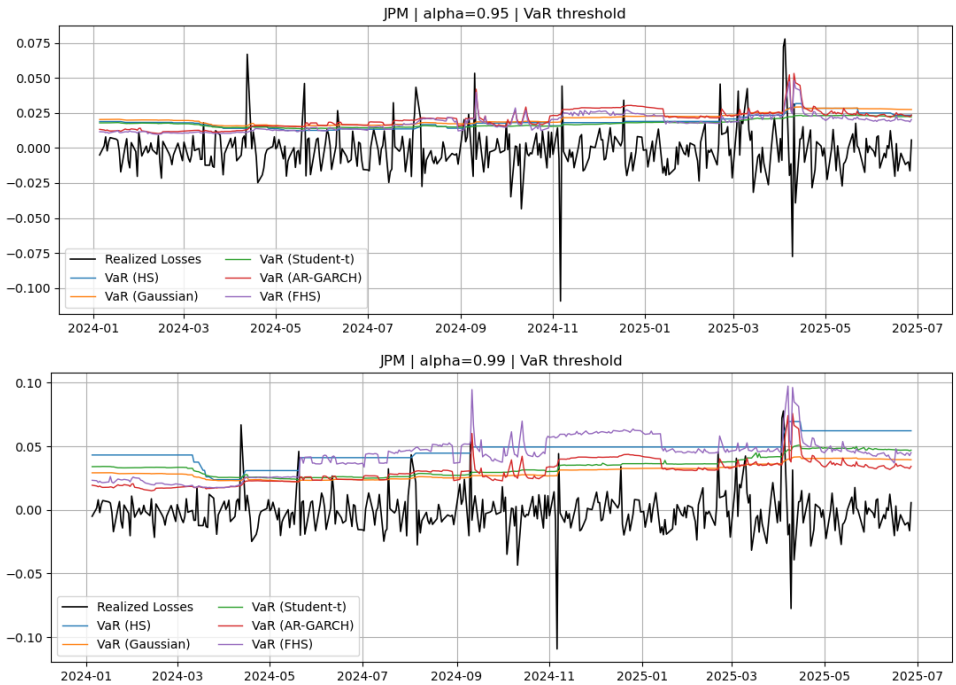


Figure 15: JPM: Pronounced drawdowns with clustered volatility. HS is relatively flat and shows more crossings at 95%; Student-t/FHS lift VaR more aggressively around stress, reducing exceptions at 99%.

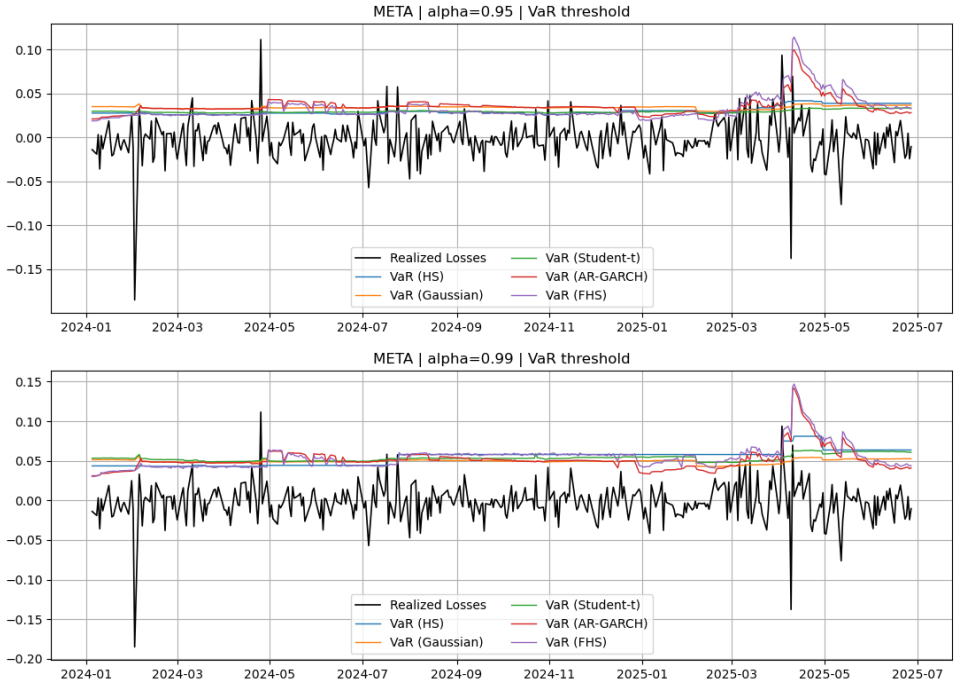


Figure 16: META: Long calm period followed by a regime shift in early-2025. Student-t and FHS adapt faster and sit above Gaussian/AR-GARCH during the spike, yielding fewer crossings; thresholds increase further at 99%.

3.2 Backtesting VaR: Kupiec POF and Christoffersen Conditional Coverage Tests

3.2.1 Kupiec Proportion Of Failure Test

Theory

The goal of the Kupiec test it to check whereas the obtained Value at Risk obtained with our different models is consistent or not with the nominal level. This Test only checks the amount of VaR violations occured on the time frame but it doesn't take into account when these violations occured.

Let I_t be the number of violation which means, the number of time the loss actually exceeded the VaR forcasted by our model. Let T be the period of our dataset and therefore the length of forcasted we have computed.

$$I_t = 1_{\{L_t > \hat{q}_{t,\alpha}\}}, \forall t = 1, ..., T$$

$$N = \sum_{t=1}^{T} I_t, \qquad \hat{p} = \frac{N}{T}.$$

Therefore we have that $\hat{p}$ is the density of violation.
At a nominal level α, we should have probability of violation of $p = 1 - \alpha$. The hypothesis are :

- H_0 : $\{I_t\}$ are $i.i.d$ Bernoulli with probability $\hat{p} = p = 1 - \alpha$
- H_1 : $\hat{p} \neq p$

Which corresponds to the following likelihood :

- $L_0 = (1-p)^{T-N} p^N$

- $L_1 = (1-\hat{p})^{T-N}\hat{p}^N$

And the test statistic is :

$$LR_{\text{POF}} = -2\log\left(\frac{L_0}{L_1}\right) = -2\log\left(\frac{(1-p)^{T-N}\,p^N}{(1-\hat{p})^{T-N}\,\hat{p}^N}\right).$$

where $LR_{\text{POF}} \sim \chi^2_1$ as $T \to \infty$.

Large values indicate that the model produces too many or too few exceptions relative to the nominal level.

Results

Level	Asset	Model	N	LR_POF	p-value	Decision
$\alpha = 0.95$	AAPL	HS	33	9.803	0.001742	Fail
		Gaussian	31	7.455	0.006324	Fail
		Student-t	40	20.314	$7.67e^{-6}$	Fail
		AR-GARCH	31	7.455	0.006324	Fail
		FHS	31	7.455	0.006324	Fail
	META	HS	21	0.341	0.55902	Pass
		Gaussian	17	0.137	0.71694	Pass
		Student-t	24	1.580	0.20876	Pass
		AR-GARCH	16	0.379	0.54196	Pass
		FHS	21	0.341	0.55902	Pass
	JPM	HS	22	0.6589	0.41649	Pass
		Gaussian	19	0.010	0.90546	Pass
		Student-t	23	1.073	0.30023	Pass
		AR-GARCH	20	0.124	0.72380	Pass
		FHS	22	0.6589	0.41649	Pass
$\alpha = 0.99$	AAPL	HS	6	1.215	0.27027	Pass
		Gaussian	9	5.477	0.01926	Fail
		Student-t	7	2.3559	0.12480	Pass
		AR-GARCH	13	14.310	0.000150	Fail
		FHS	8	3.788	0.05160	Pass
	META	HS	5	0.4156	0.51910	Pass
		Gaussian	9	5.477	0.01926	Fail
		Student-t	5	0.4156	0.51910	Pass
		AR-GARCH	4	0.0239	0.87870	Pass
		FHS	6	2.3559	0.12480	Pass
	JPM	HS	6	1.215	0.27027	Pass
		Gaussian	13	14.310	0.000150	Fail
		Student-t	12	11.827	0.000583	Fail
		AR-GARCH	13	14.310	0.000150	Fail
		FHS	6	2.3559	0.12480	Pass

Table 12: **Kupiec Proportion-of-Failures Test Results.** The table reports the number of VaR violations (N), the likelihood ratio statistic (LR_POF), its associated p-value, and whether the null hypothesis of correct unconditional coverage is rejected at the 5% significance level for each model, asset, and confidence level. Red cells indicate rejection of H_0 (model fails the POF test), while green cells indicate failure to reject H_0 (model passes).

Interpretation

The Kupiec test evaluates whether each VaR model produces a number of violations consistent with the nominal confidence level. A rejection indicates that the observed frequency of breaches deviates significantly from what the model predicts, meaning the VaR estimates are mis-calibrated. Given $T = 370$ observations, the expected (nominal) number of violations is $0.05 \times 370 = 18.5$ at the 5% level and $0.01 \times 370 = 3.7$ at the 1% level.

Across the results, AAPL stands out: all models fail at the 95% level, with far too many violations, showing that AAPL's risk is systematically underestimated. At the 99% level, only HS, FHS, and Student-t pass, confirming that heavier-tailed or non-parametric approaches capture AAPL's extreme risk better than Gaussian or AR-GARCH.

For META and JPM, most models pass at 95%, indicating good calibration in the central part of the distribution. However, at the 99% level, Gaussian and AR-GARCH VaR frequently fail, especially for JPM, suggesting that standard parametric models struggle with tail risk. In contrast, HS, FHS, and Student-t generally provide acceptable coverage in the extremes.

Overall, these results show that simple Gaussian VaR and AR-GARCH often underestimate tail losses, while models incorporating historical data or heavier tails perform more robustly at high confidence levels.

3.2.2 Christoffersen Independence and Conditional Coverage Test

Theory

While the Kupiec test focuses only on whether the number of VaR violations matches the expected nominal level, it does not account for when these violations occur. Christoffersen's tests extend this analysis by examining the temporal structure of violations. The idea is that even if a model produces the correct amount of exceptions, it may still be inadequate if those exceptions tend to appear in clusters, which would indicate that the model is not capturing shifts in market volatility properly.

As before, let $\{I_t\}$ be the violation sequence. Model it as a first-order Markov chain with transition probabilities :

$$\pi_{ij} = \Pr(I_t = j \mid I_{t-1} = i), \qquad i, j \in \{0, 1\}.$$

Which means that π_{01} is the probability of violation after no violation, π_{11} is the probability of another violation after violation ...

The transition counts :

$$N_{ij} = \#\{\, t : I_{t-1} = i,\ I_t = j \,\}.$$

Counts how many times does the phenomenon happens. (for example N_{01} is the number of violation after no violation transition).

The transition probabilities are then given by :

$$\hat{\pi}_0 = \frac{N_{01}}{N_{00} + N_{01}}, \qquad \hat{\pi}_1 = \frac{N_{11}}{N_{10} + N_{11}}, \qquad \hat{p} = \frac{N_{01} + N_{11}}{N_{00} + N_{01} + N_{10} + N_{11}}.$$

Where $\hat{\pi}_0$ is the probability of a violation given previous day was no violation; $\hat{\pi}_1$ is Probability of a violation given previous day was a violation $\rightarrow$ clustering indicator and $\hat{p}$ is the probability of violation.

Likelyhood

- L_1 : likelihood under the Markov model (violations may depend on previous day)

- L_0 : likelihood under the i.i.d. model (no dependence)

$$L_1 = (1-\hat{\pi}_0)^{N_{00}}\hat{\pi}_0^{N_{01}}(1-\hat{\pi}_1)^{N_{10}}\hat{\pi}_1^{N_{11}}, \qquad L_0 = (1-\hat{p})^{N_{00}+N_{10}}\hat{p}^{N_{01}+N_{11}}.$$

The **independence test** is then given by

$$LR_{\text{ind}} = -2\log\left(\frac{L_0}{L_1}\right) \sim \chi_1^2.$$

If we reject it, then violations are not independent but clustered.

And the **Conditional coverage test** is given by

$$LR_{\text{cc}} = LR_{\text{POF}} + LR_{\text{ind}} \sim \chi_2^2.$$

If it's too large ,the model fails either due to wrong number of violations or clustering, or both.

Results

Level	Asset	Model	$\mathbf{LR}_{\text{ind}}$	p-value	Decision
$\alpha = 0.95$	AAPL	HS	3.088196	0.078862	Pass
		Gaussian	2.166115	0.141082	Pass
		Student-t	3.297690	0.069377	Pass
		AR-GARCH	0.069110	0.792637	Pass
		FHS	0.069110	0.792637	Pass
	META	HS	0.037859	0.845727	Pass
		Gaussian	0.061229	0.804564	Pass
		Student-t	0.260311	0.609907	Pass
		AR-GARCH	1.451044	0.228360	Pass
		FHS	0.037859	0.845727	Pass
	JPM	HS	4.296685	0.038187	Fail
		Gaussian	3.163826	0.075287	Pass
		Student-t	3.733039	0.053346	Pass
		AR-GARCH	2.688893	0.101050	Pass
		FHS	4.296685	0.038187	Fail
$\alpha = 0.99$	AAPL	HS	3.143202	0.076244	Pass
		Gaussian	1.616578	0.203570	Pass
		Student-t	2.534580	0.111377	Pass
		AR-GARCH	0.526274	0.468178	Pass
		FHS	2.033781	0.153838	Pass
	META	HS	0.137404	0.710876	Pass
		Gaussian	1.616578	0.203570	Pass
		Student-t	0.137404	0.710876	Pass
		AR-GARCH	0.087703	0.767118	Pass
		FHS	0.198401	0.656015	Pass
	JPM	HS	3.143202	0.076244	Pass
		Gaussian	0.526274	0.468178	Pass
		Student-t	0.728823	0.393265	Pass
		AR-GARCH	0.526274	0.468178	Pass
		FHS	2.534580	0.111377	Pass

Table 13: **Christoffersen Independence Test Results.** The table reports the likelihood ratio statistic (LR_{ind}), its p-value, and the decision at the 5% level.

Interpretation

Overall: the null of independence is not rejected for almost all asset–model pairs.

- $\alpha = 0.95$:
  - JPM: HS and FHS *fail* (small p-values) $\Rightarrow$ clustered exceptions.
  - AAPL, META: all models *pass* (some borderline but $p > 0.05$).
- $\alpha = 0.99$: all asset–model pairs *pass* comfortably.

Takeaway: aside from JPM with HS/FHS at 95%, there is no statistical evidence of dependence; exception timing is consistent with independence. The JPM–HS/FHS failures suggest naive (non-conditional) historical methods can miss volatility regime shifts on JPM, leading to clustered violations.

Level	Asset	Model	LR_{cc}	p-value	Decision
$\alpha = 0.95$	AAPL	HS	12.891376	0.001587	Fail
		Gaussian	9.621425	0.008142	Fail
		Student-t	23.329130	0.000009	Fail
		AR-GARCH	7.524420	0.023232	Fail
		FHS	7.524420	0.023232	Fail
	META	HS	0.379253	0.827268	Pass
		Gaussian	0.192670	0.908160	Pass
		Student-t	1.840412	0.398437	Pass
		AR-GARCH	1.822959	0.401929	Pass
		FHS	0.379253	0.827268	Pass
	JPM	HS	4.955608	0.083927	Pass
		Gaussian	3.177931	0.204137	Pass
		Student-t	4.806177	0.090438	Pass
		AR-GARCH	2.813765	0.244906	Pass
		FHS	4.955608	0.083927	Pass
$\alpha = 0.99$	AAPL	HS	4.358794	0.113110	Pass
		Gaussian	7.093688	0.028815	Fail
		Student-t	4.890482	0.086705	Pass
		AR-GARCH	14.836446	0.000600	Fail
		FHS	5.822197	0.054416	Pass
	META	HS	0.553074	0.758406	Pass
		Gaussian	7.093688	0.028815	Fail
		Student-t	0.553074	0.758406	Pass
		AR-GARCH	0.111641	0.945709	Pass
		FHS	1.413993	0.493123	Pass
	JPM	HS	4.358794	0.113110	Pass
		Gaussian	14.836446	0.000600	Fail
		Student-t	12.556101	0.001877	Fail
		AR-GARCH	14.836446	0.000600	Fail
		FHS	4.890482	0.086705	Pass

Table 14: **Christoffersen Conditional Coverage Test Results.** The table reports the likelihood ratio statistic (LR_{cc}), its p-value, and the decision at the 5% level

Interpretation

- At $\alpha = 0.95$:
  - AAPL: fails under all models $\Rightarrow$ systematic misspecification (coverage and/or independence).
  - META: passes across models $\Rightarrow$ no evidence against conditional coverage.

  - JPM: passes across models ⇒ no evidence against conditional coverage.

- At $\alpha = 0.99$:
  - AAPL: Gaussian and AR–GARCH fail; HS, Student-t, and FHS pass.
  - META: only Gaussian fails; all other models pass.
  - JPM: Gaussian, Student-t, and AR–GARCH fail; HS and FHS pass.

Therefore : the joint test highlights that light–tailed parametric specifications (*Gaussian*, and often *AR–GARCH* with Gaussian innovations) tend to underperform at the high confidence level (99%), especially for AAPL and JPM, whereas *Student-t*, *HS*, and *FHS* are generally more robust. At 95%, conditional coverage is adequate for META and JPM, but not for AAPL.

3.3 Expected Shortfall Backtest : Acerbi and Székely Z_1 test

Theory

Since ES is about tail severity, it is not possible to backtest it by couting the expectation as we did with the VaR. Two models with the same number of VaR violations may produce very different tail losses. Therefore we need a test that would allow us to see if, when a violations happens, if the losses inside the tail consistent with the ES predicted by the model.

We define once again $\{I_t\}$ as the sequence of violation and N as the total number of violations. Let $(\hat{q}_{t,\alpha}, \hat{e}_{t,\alpha})$ be the model's forecasted VaR and ES pair.

If the model is correct, then among the days where a violation happened, the model expects that:

$$\frac{L_t}{\hat{e}_{t,\alpha}} - 1 = 0 \quad \text{on average.}$$

- If realized tail losses match ES ⇒ ratio = 1 ⇒ expectation = 0.
- If losses are larger than ES ⇒ ratio > 1 ⇒ model underestimates tail risk.
- If losses are smaller than ES ⇒ ratio < 1 ⇒ model overestimates tail risk.

Therefore the hypothesis are given by

- H_0 : Both the VaR and ES forecasts are correctly specified. In particular, conditional on a VaR violation occurring ($I_t = 1$), the realized tail losses are consistent with the ES forecasts.

$$\mathbb{E}\left[\frac{L_t}{\hat{e}_{t,\alpha}} - 1 \,\middle|\, I_t = 1\right] = 0.$$

- H_1 : The ES forecasts are too low, meaning the model underestimates tail risk.

The test statistic is given by :

$$Z_1 = \frac{1}{N}\sum_{t=1}^{T} \frac{L_t I_t}{\hat{e}_{t,\alpha}} - 1.$$

- $Z_1 = 0 \;\rightarrow\;$ perfect ES calibration
- $Z_1 > 0 \;\rightarrow\;$ realized tail losses worse than predicted
- $Z_1 < 0 \;\rightarrow\;$ ES too conservative

Unlike the VaR tests, the sampling distribution of Z_1 is not known in closed form, since it

depends on the full predictive distribution of the model. We therefore approximate it by simulation:

- For each forecast date t, we simulate M artificial losses from the model's one-step-ahead predictive distribution and, for each simulated path m, compute the corresponding statistic $Z_1^{(m)}$ (discarding paths with no violations).
- The empirical distribution of $\{Z_1^{(m)}\}_{m=1}^{M}$ is used as an approximation of the null distribution of Z_1, and the one-sided p-value is given by

$$p = \frac{1}{M} \sum_{m=1}^{M} \mathbf{1}\{Z_1^{(m)} \geq Z_1\}.$$

The results can be interpreted as :

- If p is not too small, the observed Z_1 is compatible with its null distribution and we do not reject H_0: ES forecasts are consistent with realized tail losses.
- If p is small (i.e. Z_1 is unusually large), we reject H_0: realized tail losses are on average larger than predicted and the ES is underestimated.

Results

Level	Asset	Model	Z_1	p-value	Decision
$\alpha = 0.95$	AAPL	HS	0.030912	0.319000	Pass
		Gaussian	0.193161	0.000000	Fail
		Student-t	-0.014922	0.504000	Pass
		AR-GARCH	0.204793	0.000000	Fail
		FHS	0.072099	0.227000	Pass
	META	HS	0.104028	0.155000	Pass
		Gaussian	0.270184	0.000000	Fail
		Student-t	0.062503	0.231000	Pass
		AR-GARCH	0.207901	0.000000	Fail
		FHS	0.083205	0.208000	Pass
	JPM	HS	0.171050	0.045000	Fail
		Gaussian	0.593548	0.000000	Fail
		Student-t	0.351561	0.019000	Fail
		AR-GARCH	0.486159	0.000000	Fail
		FHS	0.183453	0.078000	Pass
$\alpha = 0.99$	AAPL	HS	0.174490	0.002088	Fail
		Gaussian	0.351411	0.002049	Fail
		Student-t	0.042867	0.296107	Pass
		AR-GARCH	0.202613	0.014418	Fail
		FHS	0.127768	0.237998	Pass
	META	HS	0.208775	0.131356	Pass
		Gaussian	0.174087	0.024590	Fail
		Student-t	0.087137	0.243550	Pass
		AR-GARCH	0.453170	0.001030	Fail
		FHS	0.268174	0.131880	Pass
	JPM	HS	0.177038	0.030494	Fail
		Gaussian	0.444655	0.001025	Fail
		Student-t	0.082797	0.246138	Pass
		AR-GARCH	0.377650	0.002060	Fail
		FHS	0.171182	0.188272	Pass

Table 15: **Expected Shortfall Backtest (Acerbi–Székely Z_1).** The table reports the observed Z_1 statistic, its one-sided p-value, and the decision at the 5% level.

Interpretation

The test evaluates whether ES forecasts are correct, conditional on VaR being correct. This is a one–sided test; we reject for small p–values (e.g., 5%).

Overall patterns are clear and consistent across assets and confidence levels:

- Gaussian and AR–GARCH often fail the Z_1 test (frequent small p–values), indicating systematic ES underestimation, especially at the 99% level and for JPM.
- Student–t and FHS generally pass, suggesting heavier tails or filtered historical residuals provide more reliable ES in the extremes.
- HS is mixed: it passes for AAPL and META at 95%, but tends to fail for JPM and at 99%.

Asset–specific remarks:

- AAPL: at 95%, HS, Student–t, and FHS pass while Gaussian and AR–GARCH fail; at 99%, only Student–t and FHS pass, confirming that tail modeling matters more at higher confidence.

- META: across both 95% and 99%, Gaussian and AR–GARCH fail, whereas HS, Student–t, and FHS pass, pointing to the benefit of heavier–tailed or empirical approaches.
- JPM: the strictest case. At 95%, only FHS passes; at 99%, Student–t and FHS pass, while HS, Gaussian, and AR–GARCH fail. This indicates particularly heavy tails for JPM where light–tailed specifications underestimate ES.

In short, models with heavier tails or filtered historical innovations provide ES forecasts that are more consistent with realized tail losses, while light–tailed Gaussian specifications and AR–GARCH with Gaussian innovations tend to understate tail risk, notably at the 99% level and for JPM.

3.4 Overall of backtest

To synthesize results across tests, we build a simple scorecard that assigns +1 for each backtest passed at the 5% level: (i) Kupiec POF (unconditional coverage), (ii) Christoffersen independence/conditional coverage (timing and joint coverage), and (iii) Acerbi–Székely Z_1 for ES (tail severity).

For a given asset, model, and confidence level, the score ranges from 0 to 3. A score of **3** means the model is *jointly* well calibrated: it hits the nominal exception rate, exhibits no clustering of violations, and does not underestimate tail losses—hence it is the most reliable choice for that asset and α in our sample. Lower scores indicate specific weaknesses (bias in coverage, clustered exceptions, or ES underestimation), helping diagnose why a model underperforms.

Level	Asset	Model	Score
$\alpha = 0.95$	AAPL	HS	1
		Gaussian	0
		Student-t	1
		AR-GARCH	0
		FHS	1
	META	HS	3
		Gaussian	2
		Student-t	3
		AR-GARCH	2
		FHS	3
	JPM	HS	2
		Gaussian	2
		Student-t	2
		AR-GARCH	2
		FHS	3
$\alpha = 0.99$	AAPL	HS	2
		Gaussian	0
		Student-t	3
		AR-GARCH	0
		FHS	3
	META	HS	3
		Gaussian	0
		Student-t	3
		AR-GARCH	2
		FHS	3
	JPM	HS	2
		Gaussian	0
		Student-t	1
		AR-GARCH	0
		FHS	3

Table 16: Backtest scorecard (0–3) by level, asset, and model. Score adds +1 for each test passed (Kupiec POF, Christoffersen CC/Ind, ES Z_1). Color code: red = 0, orange = 1, yellow = 2, green = 3.

Method ranking (overall):

- FHS is the most reliable: many 3/3 scores; only weak spot is AAPL at 95% (score 1).
- Student–t performs strongly, but degrades for JPM at 99% (score 1).
- HS is middling and asset–dependent (from 1 on AAPL–95% up to 3 on META).
- AR–GARCH (Gaussian) and Gaussian i.i.d. are the weakest: several 0s, especially at 99%.

By asset and level:

- AAPL : at 95%: no model clears all tests (scores 0–1); at 99%: FHS and Student–t reach 3, HS=2, Gaussian/AR–GARCH=0.
- META : easiest asset: at 95% HS, Student–t, FHS all score 3 (Gaussian/AR–GARCH=2); at 99% HS, Student–t, FHS again 3, AR–GARCH=2, Gaussian=0.
- JPM : hardest at 99%: at 95% FHS=3, others=2; at 99% only FHS=3, HS=2, Stu-

dent–t=1, Gaussian and AR–GARCH=0.

Takeaways:

- Passing all three tests (score 3) indicates a model that attains correct exception frequency, shows no clustering, and does not underestimate tail losses; this is achieved most consistently by FHS, and often by Student–t.
- Light–tailed Gaussian specifications (with or without GARCH) frequently fail at high confidence, particularly for JPM.
- Asset heterogeneity is clear: META is most forgiving, JPM is most demanding at 99%, and AAPL improves markedly from 95% to 99% only with heavy–tailed/filtered methods.

Numerical implementation considerations

Rolling routine: `build_rolling_forecasts`
Using the `forecast_model` functions, we build a rolling routine that:

1. loops over $t = W, \dots, T-1$,
2. extracts the estimation window $\{t-W+1, \dots, t\}$,
3. computes $(\mathrm{VaR}_{\alpha,t+1}, \mathrm{ES}_{\alpha,t+1}, \texttt{params}_{t+1})$ for each model and each α,
4. retrieves the realized out-of-sample loss L_{t+1},
5. constructs the exception indicator (VaR breach)

$$I_{t+1} := \mathbf{1}\{L_{t+1} > \mathrm{VaR}_{\alpha,t+1}\}.$$

The routine returns a single `DataFrame` with (at least) the columns:

$$(\texttt{date}, \texttt{asset}, \texttt{method}, \alpha, \mathrm{VaR}, \mathrm{ES}, L_{\mathrm{real}}, R_{\mathrm{real}}, \texttt{breach}, \texttt{params}).$$

A flag `store_params` is used to reduce memory usage, since VaR backtests only use the breach series and do not require storing `params`.

VaR backtesting: Kupiec and Christoffersen
Once the rolling table is built, VaR backtesting becomes model-agnostic:

- **Kupiec POF** (unconditional coverage) only requires $\{I_t\}$ via $N = \sum_t I_t$ and T, and compares the empirical exception rate $\hat{p} = N/T$ to the target $p_0 = 1-\alpha$.
- **Christoffersen CC** (conditional coverage) additionally tests independence by modeling $\{I_t\}$ as a two-state first-order Markov chain and combining coverage and independence tests.

In practice, both tests are computed directly from the `breach` column of the rolling `DataFrame`.

ES backtesting: Acerbi–Székely Z_1 The Z_1 ES test requires $(L_t, \mathrm{VaR}_t, \mathrm{ES}_t)$ and compares an observed statistic to its distribution under the null model, estimated by simulation. This is where the pair `forecast_model` / `simulate_one_step_model` is essential:

- the rolling routine provides, for each date, VaR_t, ES_t, and $\texttt{params}_t$,
- the Z_1 routine calls `simulate_one_step_model(params_t, M)` to generate $\{L_t^{(m)}\}$,
- a one-sided p-value is reported (primarily detecting ES underestimation).

This design prevents code duplication and guarantees that ES backtesting is consistent with the predictive distribution used to produce VaR/ES.

4 Copula Fitting

In this section, we model the dependence between AAPL, META and JPM over a single estimation window of length $W = 252$. Following Sklar's theorem, the dependence structure is extracted from the marginal distributions by working with pseudo-observations, which are then used to fit Gaussian and Student-t copulas. We finally simulate synthetic return series and compare them to the empirical data.

4.1 Pseudo-observations and empirical dependence

For each asset $i \in \{\text{AAPL}, \text{META}, \text{JPM}\}$ and each date $t = 1, \ldots, W$, we compute the pseudo-observations

$$U_{t,i} = \frac{\text{rank}(R_{t,i})}{W+1},$$

where $R_{t,i}$ denotes the raw log-return. These correspond to empirical quantiles and therefore take values in $(0, 1)$. This transformation removes the marginal distributions while preserving the cross-sectional dependence.

Figure 17 shows, for each asset pair, the scatterplots of raw returns $(R_{t,i}, R_{t,j})$ and of pseudo-observations $(U_{t,i}, U_{t,j})$. The scatterplots of raw returns $(R_{t,i}, R_{t,j})$ show mild positive dependence for all asset pairs, with AAPL–META exhibiting the strongest relationship and AAPL–JPM the weakest. The clouds are mostly elliptical and centered around zero, with no visually obvious asymmetry or concentration in the joint tail regions.

When transforming the data into pseudo-observations $(U_{t,i}, U_{t,j})$, the marginal distributions become uniform but the dependence patterns remain similar: the point clouds are relatively diffuse and do not show clear evidence of tail dependence at a visual level. The dependence structure appears mostly symmetric and moderate.

Overall, the plots do not allow us to clearly distinguish between tail-dependent and tail-independent models purely by visual inspection. This motivates fitting both a Gaussian copula and a Student-t copula and comparing their likelihoods to determine which model captures the dependence more accurately.

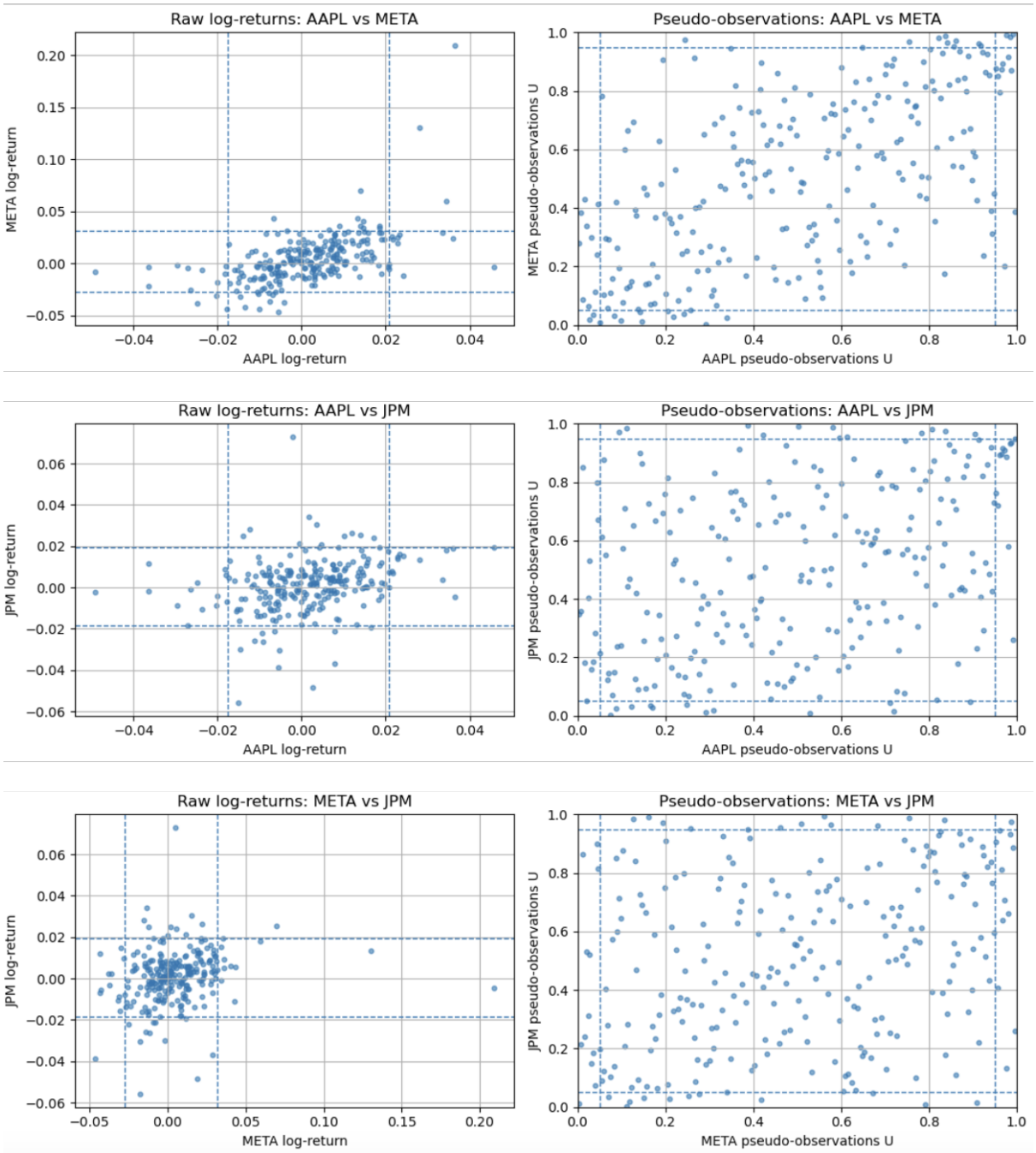


Figure 17: Scatterplots of raw returns and pseudo-observations for each asset pair.

4.2 Gaussian and Student-t copula estimation

Gaussian copula. Given pseudo-observations $U_{t,i}$, we transform each margin using normal scores $Z_{t,i} = \Phi^{-1}(U_{t,i})$ and estimate the Gaussian copula correlation matrix as the empirical correlation of $(Z_{t,1}, Z_{t,2}, Z_{t,3})$. The resulting maximum-likelihood estimator is

$$\Sigma_{\text{Gauss}} = \begin{pmatrix} 1.000 & 0.600 & 0.343 \\ 0.600 & 1.000 & 0.287 \\ 0.343 & 0.287 & 1.000 \end{pmatrix}.$$

Student-t copula. We estimate jointly the correlation matrix and the degrees of freedom ν via maximum likelihood using the t-copula density. The fitted parameters are

$$\Sigma_{\mathrm{t}} = \begin{pmatrix} 1.000 & 0.595 & 0.349 \\ 0.595 & 1.000 & 0.302 \\ 0.349 & 0.302 & 1.000 \end{pmatrix}, \qquad \nu \approx 13.06.$$

Model comparison. The maximised log-likelihoods are

$$\ell_{\mathrm{Gauss}} = 69.29, \qquad \ell_{\mathrm{t}} = 70.99.$$

The Student-t copula achieves a higher log-likelihood, indicating a better fit to the data. This is consistent with the presence of some degree of tail dependence in equity returns, even if it is not very pronounced visually.

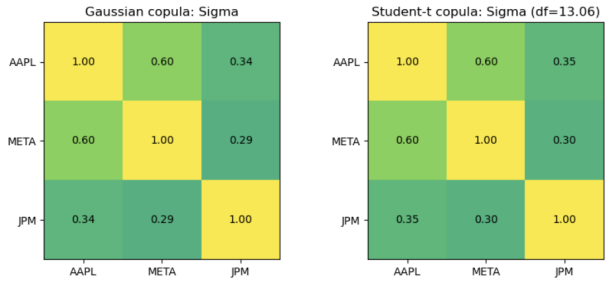


Figure 18: Estimated correlation matrices for the Gaussian copula and Student-t copula

4.3 Simulation from copulas and comparison of marginal distributions

Given the fitted Gaussian and Student-t copulas, we first simulate T dependent uniform samples

$$U_t^{\mathrm{Gauss}} \sim C_{\mathrm{Gauss}}, \qquad U_t^{\mathrm{t}} \sim C_{\mathrm{t}}, \qquad t = 1, \ldots, T,$$

where each U_t is a 3-dimensional vector with components corresponding to AAPL, META and JPM. This is implemented with the `random(T)` method of the `GaussianCopula` and `StudentCopula` objects from the `copulae` package:

$$U^{\mathrm{Gauss}} = \texttt{gauss_cop.random}(T), \qquad U^{\mathrm{t}} = \texttt{t_cop.random}(T).$$

To recover return series in the original scale, we apply the empirical inverse CDF of each asset to the simulated uniforms. For a given asset i, with historical returns $\{R_{1,i}, \ldots, R_{T,i}\}$, we define the empirical inverse CDF $\hat{F}_i^{-1}$ by interpolating between the order statistics of the sample. In code, this is implemented by the function

$$R_{t,i}^{\mathrm{sim,Gauss}} = \hat{F}_i^{-1}\left(U_{t,i}^{\mathrm{Gauss}}\right), \qquad R_{t,i}^{\mathrm{sim,t}} = \hat{F}_i^{-1}\left(U_{t,i}^{\mathrm{t}}\right),$$

where `empirical_inverse_cdf(u, data)` sorts the historical returns `data`, maps each $u \in (0, 1)$ to a fractional rank position, and linearly interpolates between neighbouring order statistics.

This procedure guarantees that, for each asset, the simulated returns preserve the empirical marginal distribution, while the dependence structure is governed by the chosen copula (Gaussian or Student-t).

4.4 Visualization

Histograms on a logarithmic scale. For each asset (AAPL, META, JPM), we compare the marginal distributions of

$$R_i, \quad R_i^{\text{sim,Gauss}}, \quad R_i^{\text{sim,t}}$$

using overlaid histograms. **The y-axis is in logarithmic scale to highlight the tails**.

Figure 19–21 show that, for all three assets, the histograms of the simulated series (Gaussian and Student-t copulas) almost overlap the empirical histogram. This confirms that the empirical inverse CDF transformation successfully reproduces the marginal distributions.

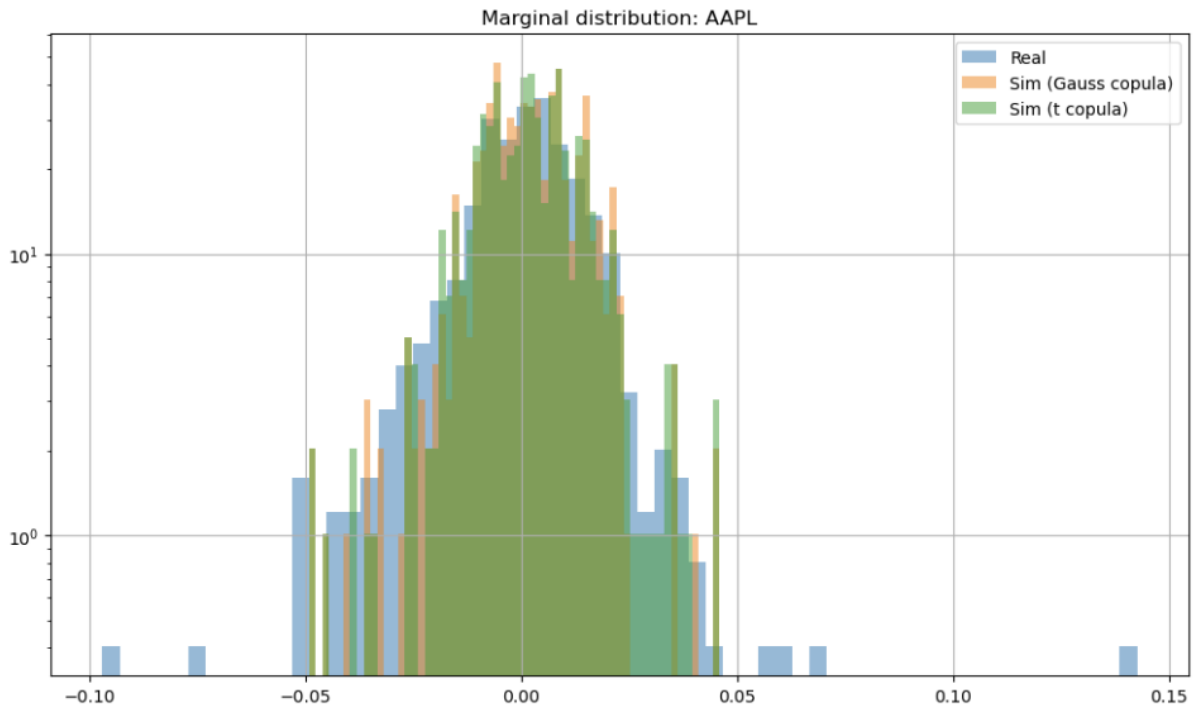


Figure 19: Distribution of AAPL returns: real vs. Gaussian-copula and t-copula simulations.

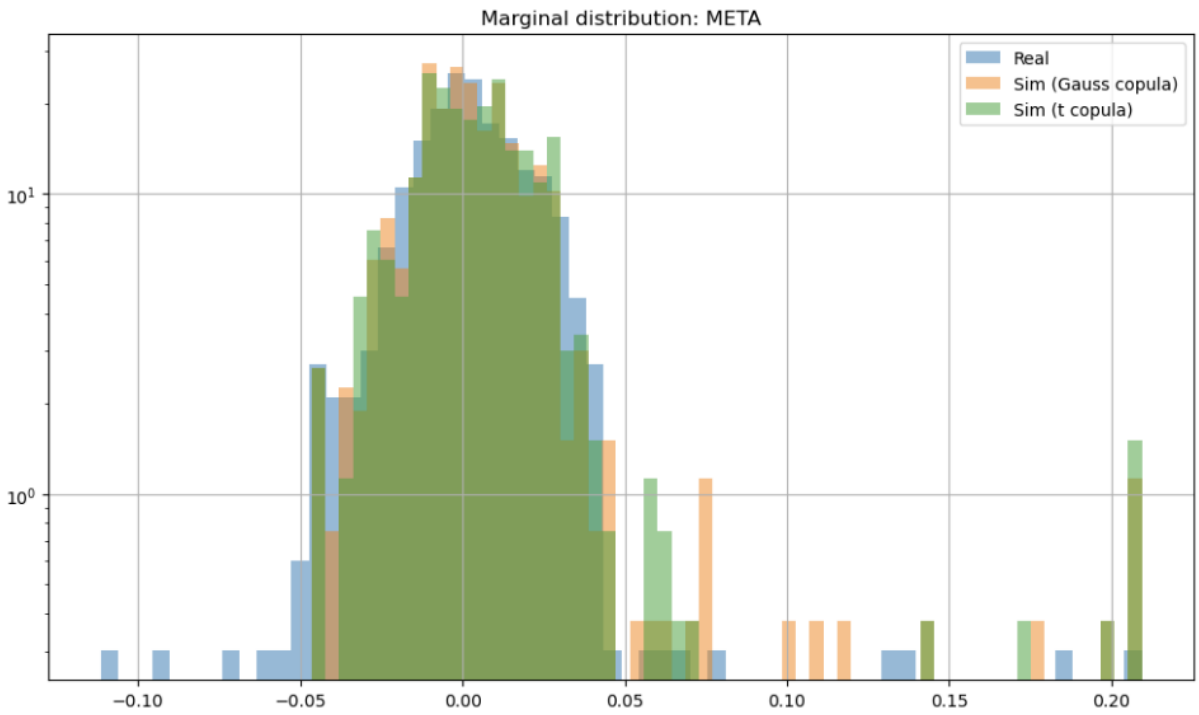


Figure 20: Distribution of META returns: real vs. Gaussian-copula and t-copula simulations.

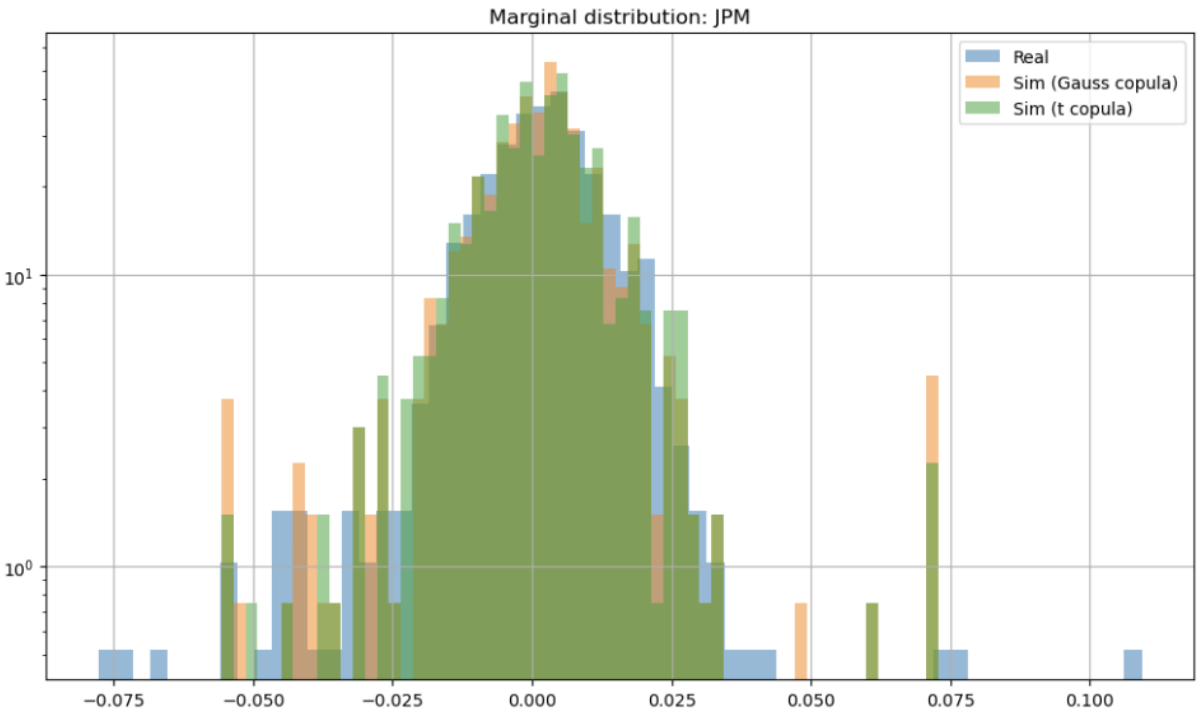


Figure 21: Distribution of JPM returns: real vs. Gaussian-copula and t-copula simulations.

Joint dependence: real vs simulated data When plotting the real data and the two copula simulations in separate panels (Figure 22), we can clearly see the shape and dispersion produced by each model: the Gaussian copula generates tighter and more elliptical clouds, while the Student-t copula produces slightly heavier tails and a wider spread, closer to what we observe in the empirical data. We also produce superposed scatterplots (Figure 23), where real

and simulated points are shown on the same graph; this allows us to directly assess how well the simulated points overlap with the actual observations. From these overlaid figures, we see that both copulas reproduce the central mass reasonably well, but the Student-t copula aligns better with the real joint extremes, indicating a more realistic dependence structure.

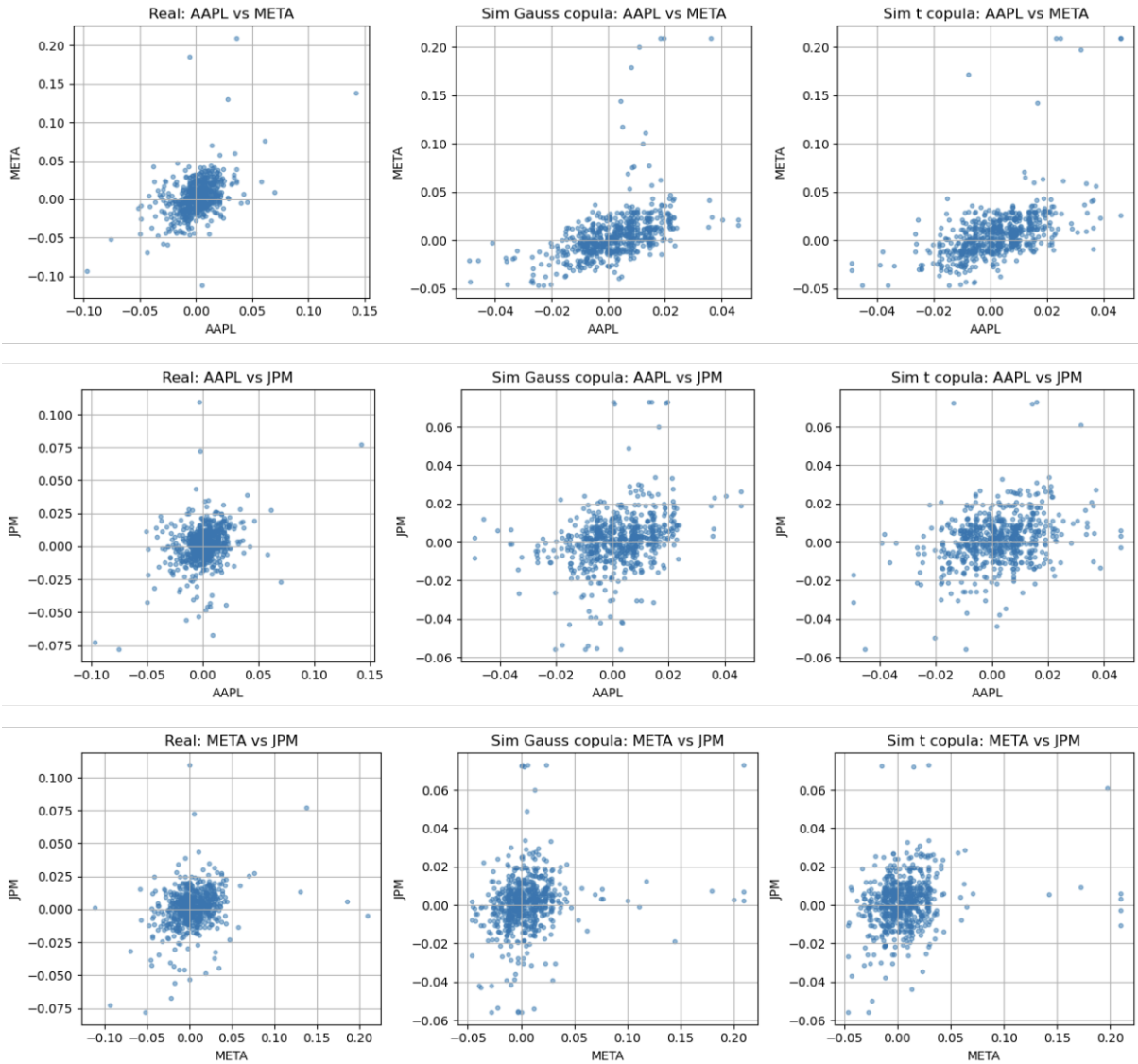
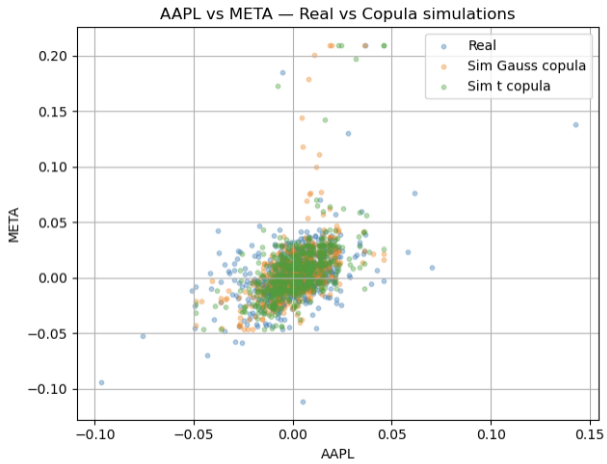
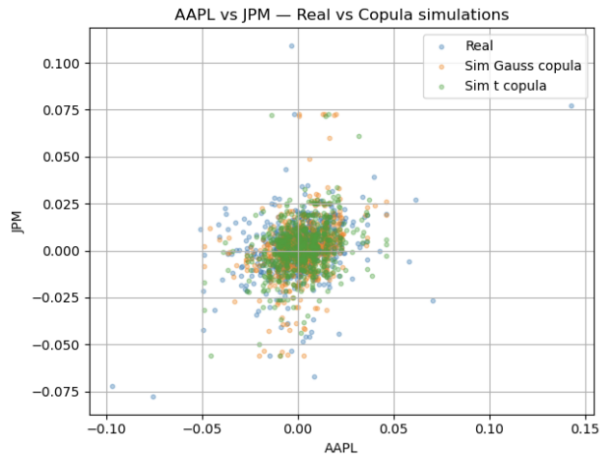


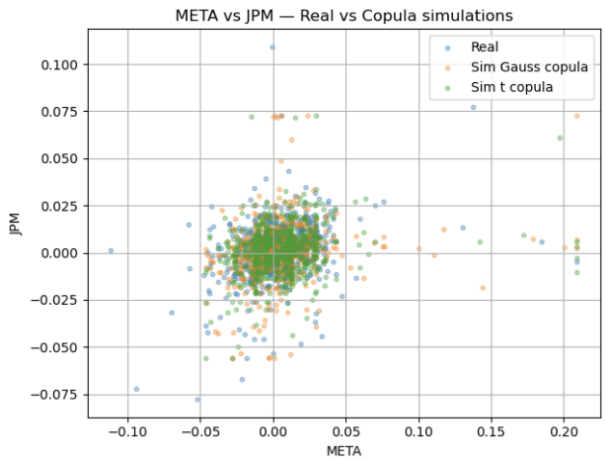
Figure 22: Joint dependence for each asset pair: real data, Gaussian-copula simulations and Student-t-copula simulations



(a) AAPL vs META



(b) AAPL vs JPM



(c) META vs JPM

Figure 23: Real vs Copula simulations (superposed)

4.5 Summary

Pseudo-observations reveal a moderate and broadly symmetric dependence between the three equities, with AAPL–META exhibiting the strongest correlation. The Student-t copula provides a better fit than the Gaussian copula, as reflected by a higher log-likelihood and by a more realistic reproduction of joint extremes in the simulated scatterplots.

By construction, the simulated returns preserve the empirical marginal distributions, which is confirmed empirically by the overlaid histograms on a logarithmic scale: the entire shape of the distribution, including the tails, is accurately matched. The main differences between the two models therefore arise from the multivariate dependence, where the Student-t copula delivers the most realistic joint behaviour.

5 Backtesting equally weighted portfolio VaR and ES

We now move from single-asset risk to a portfolio setting. The goal is to estimate and backtest 1-day-ahead VaR/ES for an equal-weighted portfolio of AAPL, META, and JPM using the same rolling-window procedure as before, and to compare univariate benchmarks with a dependence-aware (copula) approach.

The benefits of having a portfolio are :

- Diversification : Equal weights diversify idiosyncratic risk, typically reducing variance and (sometimes) tail losses compared to having a single asset.
- Risk properties : ES is coherent and benefits from diversification (sub-additive)

Equal-weighted construction. Let $R_{i,t}$ be the (log) return of asset $i \in \{\text{AAPL}, \text{META}, \text{JPM}\}$ on day t. With fixed weights $w_i = \frac{1}{3}$, the portfolio return is

$$R_{p,t} = \sum_{i=1}^{3} w_i \, R_{i,t} = \tfrac{1}{3}\big(R_{\text{AAPL},t} + R_{\text{META},t} + R_{\text{JPM},t}\big).$$

In backtesting, at each date t we compute $R_{p,t}$ from the last W days, fit the chosen model(s) on $\{R_{p,t-W+1}, \ldots, R_{p,t}\}$, and produce the 1-step-ahead VaR/ES forecast for $t+1$. For the copula route, we fit marginals to $\{R_{i,t}\}$ and a copula on the last W days, simulate $\{R_{i,t+1}^{(m)}\}_{m=1}^{N}$, form $R_{p,t+1}^{(m)} = \frac{1}{3}\sum_i R_{i,t+1}^{(m)}$, then compute portfolio VaR/ES from $\{R_{p,t+1}^{(m)}\}$.

5.1 Modelling the portfolio VaR and ES with rolling window

The modelling is done as in question 3 and we get the following. The graphs shows the equal-weighted portfolio's realized one-day returns (black) and 1-step-ahead VaR at level α from HS, Gaussian, Student-t, AR-GARCH, and FHS. An exception occurs when the black line falls below a model's VaR curve. The bottom panel uses $\alpha = 0.99$ (larger-magnitude, more conservative thresholds).

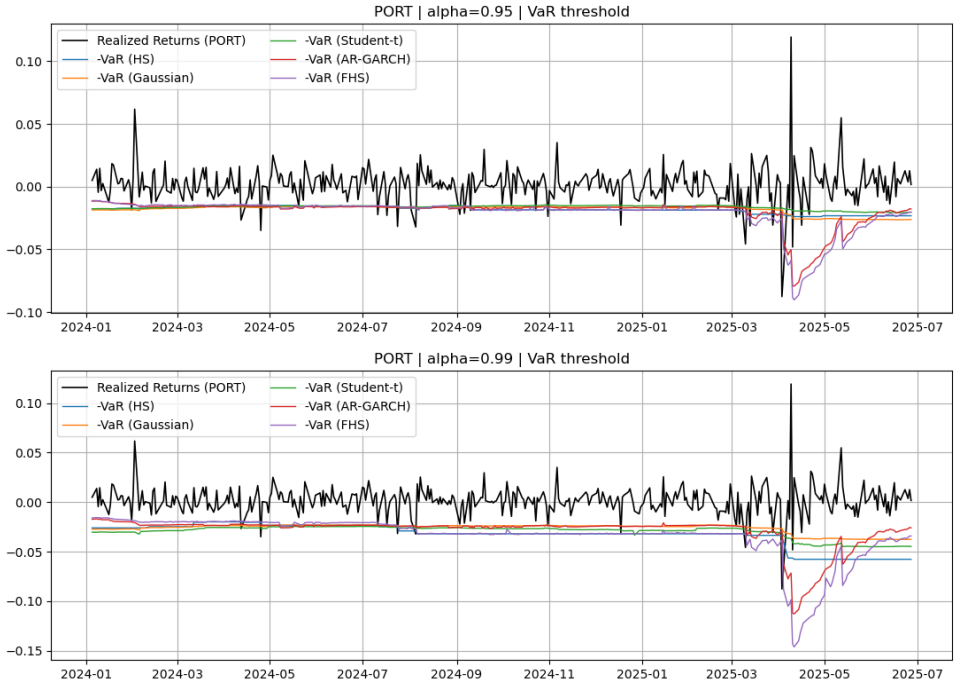


Figure 24: Portfolio VaR at 95% (top) and 99% (bottom). The spring-2025 drawdown drives the largest losses; Student-t and FHS react with the deepest (most negative) VaR, while Gaussian and HS remain flatter and sit closer to returns, leading to more crossings—especially at 99%. AR-GARCH adjusts more than Gaussian/HS but less than Student-t/FHS.

5.2 Backtest on the Portfolio

5.2.1 Kupiec Proportion Of Failure Test

Level	Model	LR_POF	p-value	Decision
$\alpha = 0.95$	HS	1.580101	0.208746	Pass
	Gaussian	1.580101	0.208746	Pass
	Student-t	3.622796	0.056993	Pass
	AR-GARCH	1.580101	0.208746	Pass
	FHS	1.073138	0.300238	Pass
$\alpha = 0.99$	HS	2.355902	0.124810	Pass
	Gaussian	14.310172	0.000155	Fail
	Student-t	9.516832	0.002036	Fail
	AR-GARCH	9.516832	0.002036	Fail
	FHS	5.477110	0.019267	Fail

Table 17: Portfolio Kupiec Proportion-of-Failures (POF) test at 5%: likelihood ratio (LR_POF), p-value, and decision.

Interpretation (portfolio POF). The Kupiec test checks whether the *unconditional* exception rate matches the nominal α.

- At $\alpha = 0.95$ **all models pass**: the portfolio VaR forecasts are well calibrated in frequency, none shows a statistically different rate of breaches from 5%.
- At $\alpha = 0.99$ only **HS passes**; **Gaussian**, **Student–t**, **AR–GARCH**, and **FHS** *fail* (very small p-values), indicating miscalibration of the 1% tail. The POF statistic is two-

sided, so failures mean "too many or too few" exceptions; in light of the portfolio plots and ES results, this most likely reflects *too many* exceptions (underestimated tail risk) for the parametric/filtered models at 99%.

Overall, unconditional coverage is adequate at 95%, but at 99% only the non-parametric HS matches the observed breach rate for the portfolio; the others require re-calibration (e.g., heavier tails or larger scaling) in the extreme tail.

5.2.2 Christoffersen Independence and Conditional Coverage Test

Level	Model	$\mathbf{LR}_{\text{ind}}$	p-value	Decision
$\alpha = 0.95$	HS	0.131373	0.717012	Pass
	Gaussian	1.237005	0.266049	Pass
	Student-t	0.546188	0.459880	Pass
	AR-GARCH	1.237005	0.266049	Pass
	FHS	1.538718	0.214809	Pass
$\alpha = 0.99$	HS	2.534580	0.111377	Pass
	Gaussian	0.526274	0.468178	Pass
	Student-t	0.973707	0.323757	Pass
	AR-GARCH	0.973707	0.323757	Pass
	FHS	0.450114	0.502281	Pass

Table 18: Portfolio Christoffersen Independence test at 5%: LR_{ind}, p-value, and decision. All p-values exceed 0.05, so independence is not rejected for any model at either confidence level.

Level	Model	$\mathbf{LR}_{\text{cc}}$	p-value	Decision
$\alpha = 0.95$	HS	1.711474	0.424970	Pass
	Gaussian	2.817106	0.244497	Pass
	Student-t	4.168984	0.124370	Pass
	AR-GARCH	2.817106	0.244497	Pass
	FHS	2.611856	0.270921	Pass
$\alpha = 0.99$	HS	4.890482	0.086705	Pass
	Gaussian	14.836446	0.000600	Fail
	Student-t	10.490539	0.005272	Fail
	AR-GARCH	10.490539	0.005272	Fail
	FHS	5.927224	0.051632	Pass

Table 19: Portfolio Christoffersen conditional coverage test at the 5% level: likelihood ratio LR_{cc}, p-value, and decision.

Interpretation (portfolio Christoffersen tests). Independence : The table shows all p-values > 0.05 at both $\alpha = 0.95$ and $\alpha = 0.99$ for every model; thus we do not reject independence. There is no evidence that portfolio VaR exceptions are clustered in time.

Conditional coverage : From the Table, at $\alpha = 0.95$ all models pass the joint test (correct rate *and* no clustering). At $\alpha = 0.99$, **Gaussian**, **Student–t**, and **AR–GARCH** *fail* (small p-values), while **HS** and **FHS** *pass* (FHS is borderline at $p = 0.052$). Since independence is not rejected, these 99% failures are primarily due to *biased coverage* (too many exceptions) rather than clustered timing. In short, the extreme tail of the portfolio is harder to calibrate for the parametric models, whereas nonparametric HS/FHS remain adequate at the joint (coverage+timing) level.

5.2.3 Acerbi and Székely Z_1 test

Level	Model	N (tail hits)	p-value (one-sided)	Decision
$\alpha = 0.95$	HS	24	0.003000	Fail
	Gaussian	24	0.000000	Fail
	Student-t	27	0.042000	Fail
	AR-GARCH	24	0.000000	Fail
	FHS	23	0.003000	Fail
$\alpha = 0.99$	HS	7	0.000000	Fail
	Gaussian	13	0.004098	Fail
	Student-t	11	0.262834	Pass
	AR-GARCH	11	0.007209	Fail
	FHS	9	0.020661	Fail

Table 20: Portfolio ES backtest (Acerbi–Székely Z_1): number of tail hits N, one-sided p-value, and decision at the 5% level (reject for small p-values, indicating ES underestimation).

Interpretation (portfolio ES backtest, Z_1). The Z_1 test asks whether realized tail losses, *conditional on a VaR breach*, are consistent with the ES forecast (one–sided: reject for underestimation).

- $\alpha = 0.95$: All models *fail* (very small p-values). Portfolio tail losses during breaches are, on average, larger than predicted ES across HS, Gaussian, Student–t, AR–GARCH and FHS. Hence, at 95% the portfolio tail is systematically underestimated.
- $\alpha = 0.99$: Only **Student–t** *passes* ($p \approx 0.26$). HS, Gaussian, AR–GARCH and FHS all *fail*, indicating ES underestimation at the 1% tail. The Student–t's heavier tails appear necessary to match observed breach severities for the portfolio.

In short, while unconditional coverage can look acceptable, ES severity is harder: the portfolio's extreme losses are heavier than most models anticipate, and only a heavy–tailed parametric specification (Student–t) is adequate at 99%. Potential reasons include residual tail dependence across names and limited sample information about the deepest tail.

5.3 Copula Implementation for Portfolio VaR/ES

We follow the exact copula workflow from Question 4, but now map simulated component returns into the portfolio loss.

5.3.1 Kupiec Proportion Of Failure Test

Level	Model	LR (POF)	p-value (POF)	Decision
$\alpha = 0.95$	Copula-Gaussian	1.580101	0.208746	Pass
	Copula-Student	1.580101	0.208746	Pass
$\alpha = 0.99$	Copula-Gaussian	3.788416	0.051608	Pass
	Copula-Student	0.415670	0.519106	Pass

Table 21: POF backtest: likelihood ratio (LR), p-value, and decision at the 5% level (reject for small p-values).

Interpretation

- $\alpha = 0.95$: Both copulas (Gaussian and Student) *pass* with identical, comfortable p-values (≈ 0.21). Exception frequency is consistent with 5%.
- $\alpha = 0.99$: Both copulas also *pass*. The Gaussian copula is *borderline* ($p = 0.0516$), whereas the Student copula passes comfortably ($p = 0.519$), suggesting slightly better 1% calibration under heavier tails.

Overall, copula-based portfolio VaR achieves correct *unconditional* coverage; at the extreme tail, the Student copula appears more robust than the Gaussian copula.

5.3.2 Christoffersen Independence and Conditional Coverage Test

Level	Model	LR (IND)	p-value (IND)	Decision
$\alpha = 0.95$	Copula-Gaussian	0.131373	0.717012	Pass
	Copula-Student	0.131373	0.717012	Pass
$\alpha = 0.99$	Copula-Gaussian	2.033781	0.153838	Pass
	Copula-Student	3.898783	0.048321	Fail

Table 22: Christoffersen independence test: likelihood ratio (LR), p-value, and decision at the 5% level (reject for small p-values).

Level	Model	LR (CC)	p-value (CC)	Decision
$\alpha = 0.95$	Copula-Gaussian	1.711474	0.424970	Pass
	Copula-Student	1.711474	0.424970	Pass
$\alpha = 0.99$	Copula-Gaussian	5.822197	0.054416	Pass
	Copula-Student	4.314453	0.115645	Pass

Table 23: Christoffersen conditional coverage (CC) test: likelihood ratio (LR), p-value, and decision at the 5% level (reject for small p-values).

Interpretation

Independence. At $\alpha = 0.95$, both copulas (Gaussian and Student) *pass* the independence test ($p \approx 0.72$), indicating no evidence of clustered exceptions. At $\alpha = 0.99$, the **Gaussian copula passes** ($p = 0.154$) while the **Student copula fails** ($p = 0.048$), suggesting mild clustering of violations for the Student copula in the extreme tail.

Conditional coverage. At $\alpha = 0.95$, both copulas *pass* the joint coverage+independence test. At $\alpha = 0.99$, **both** copulas still *pass* (Gaussian is borderline with $p = 0.054$). Note that CC uses a χ^2_2 reference distribution; hence a modest independent test rejection for the Student copula can still yield a non-rejection in the joint test when unconditional coverage is adequate.

Overall, copula-based portfolio VaR shows acceptable joint performance; any timing issues appear only for the Student copula at 99% and are mild.

5.3.3 Acerbi and Székely Z_1 test

Level	Model	p-value (one-sided)	Decision
$\alpha = 0.95$	Copula-Gaussian	0.014000	Fail
	Copula-Student	0.053000	Pass
$\alpha = 0.99$	Copula-Gaussian	0.196939	Pass
	Copula-Student	0.047910	Fail

Table 24: Portfolio ES backtest (Acerbi–Székely Z_1): one-sided p-value and decision at the 5% level (reject for small p-values, indicating ES underestimation).

Interpretation

- $\boldsymbol{\alpha = 0.95}$: **Copula–Gaussian fails** ($p = 0.014$), implying realized tail losses exceed its ES; **Copula–Student passes** ($p = 0.053$, borderline), suggesting adequate tail–severity on average.
- $\boldsymbol{\alpha = 0.99}$: **Copula–Gaussian passes** ($p = 0.197$), while **Copula–Student fails** ($p = 0.0479$), indicating underestimation in the far tail for the Student copula.

Takeaway: neither copula dominates across levels. At 95% the Gaussian copula is too light–tailed for ES, whereas at 99% the Student copula appears slightly too low for the portfolio's deepest tail. This echoes the mixed timing evidence at 99% (mild clustering for Student), and points to calibration differences across tail regions.

5.4 Comparison of Models and Impact of Copulas

The backtesting results show that modelling dependence through copulas can improve portfolio risk estimation in some situations, particularly for extreme quantiles, although the benefits are not uniform across all metrics or confidence levels. The conclusions differ notably between VaR and ES.

Value-at-Risk (VaR). For the univariate portfolio models (HS, Gaussian, Student-t, AR-GARCH, FHS), the Kupiec POF test indicates that all models are acceptable at the 95% level. However, at the 99% level, several parametric models fail to achieve correct coverage, typically underestimating tail risk; HS is the only univariate model with consistently acceptable performance. The Christoffersen conditional coverage test also shows clustering of violations for Gaussian, Student-t, and AR-GARCH VaR at 99%.

In comparison, both Gaussian and Student-t copulas pass the POF test at 95% and 99%. The Student-t copula exhibits the strongest calibration at 99%, although the Gaussian copula remains borderline in the conditional coverage test. Overall, copula-based VaR tends to provide more reliable results in the extreme tail, but the improvement should be interpreted with caution: the gains are present mainly at the 99% level and relative to parametrically constrained univariate models.

Expected Shortfall (ES). The Acerbi–Székely Z_1 test leads to a more mixed assessment. For univariate models, all specifications fail at the 95% level, while at 99% only the Student-t model shows acceptable calibration. At 95%, the Gaussian copula fails while the Student-t copula passes (borderline), suggesting that dependence modelling can help for moderately extreme losses. At 99%, however, the pattern reverses: the Gaussian copula passes and the Student-t copula fails, indicating that copula-based ES is sensitive to the tail region considered.

Thus, dependence modelling leads to partial improvements for ES, but these improvements are neither systematic nor consistent across confidence levels.

Interpretation. The pseudo-observations indicate moderate but nontrivial dependence among the assets. Modelling this dependence, particularly with a Student-t copula, helps capture joint extremes more realistically, which can benefit VaR estimation at very high confidence levels. However, ES remains difficult to calibrate due to data scarcity in the far tail, and copula-based ES estimates do not uniformly outperform simpler alternatives.

Summary. Copulas offer a richer framework for modelling joint behaviour, and they can enhance tail risk estimation in certain settings, especially for VaR at the 99% level. However, the improvement is not universal: for ES, the performance of copula-based models varies depending on the confidence level and the copula chosen. Overall, dependence modelling adds useful flexibility, but its benefits should be viewed as context-dependent rather than uniformly superior.

Numerical implementation considerations

Considering copulas

All *univariate* methods (HS, Gaussian, Student-t, AR–GARCH, FHS) can be applied without modification by running the same pipeline on the portfolio series.

The *copula* approach requires a dedicated rolling routine (but follows the same overall philosophy):

- at each t, convert component returns to pseudo-observations $U_{t,i}$,
- fit a copula (Gaussian or Student-t) on the last W days,
- simulate N multivariate scenarios $(R^{(m)}_{t+1,1}, \ldots, R^{(m)}_{t+1,d})$, map them back to returns using empirical inverse CDFs, and form portfolio scenarios $R^{(p,m)}_{t+1}$, hence losses $L^{(p,m)}_{t+1}$,
- estimate $\mathrm{VaR}_{\alpha,t+1}$ and $\mathrm{ES}_{\alpha,t+1}$ from the simulated loss sample,
- store the fitted copula parameters and marginal empirical information as `params` to enable re-simulation in the Z_1 test.

Thus, even though copulas require a model-specific rolling procedure, the backtesting layer remains unchanged: we still feed the breach series to Kupiec/Christoffersen and use `params` plus `simulate_one_step` for Z_1.

The copula method just gets its own `forecast_copula_portfolio` and `build_rolling_forecasts_portfolio_copula` functions.

A Commented Code

A.1 Main implementation

```python
# %% [markdown]
# # Project 1: Market Risk     VaR, ES, and Copulas

# %% [markdown]
# **Names of all group members:**
# - Mahe Velay (mahe.velay@epfl.ch)
# - Adela de Robert (adelaide.robert@epfl.ch)
# - Timoth e Potti  (timothee.pottie@epfl.ch)
#
#
# ---

# %% [markdown]
# ## 0) Setup
#
# Creates necessary folders and sets hyperparameters for the project.

# %%
# If a package import fails, install it in your environment, e.g.:
# %pip install yfinance arch copulae statsmodels seaborn

import os
import numpy as np
import copulae
import pandas as pd
import scipy.stats as stats
from pathlib import Path
import matplotlib.pyplot as plt
from scipy.stats import norm
from scipy.stats import jarque_bera
from scipy.stats import t
from statsmodels.distributions.empirical_distribution import ECDF
from statsmodels.tsa.stattools import pacf
from arch import arch_model
from statsmodels.stats.diagnostic import acorr_ljungbox




# your own script with helper functions, if any
# we prefer to introduce helpers chronologyally as they appear in the code for understanding purpose
# import utils as U
# our imports for simplicity
from scipy.stats import chi2

# Locate the Project 1 directory to this notebook's working directory
PROJECT_DIR = Path.cwd()
DATA_DIR = PROJECT_DIR / 'data'
OUT_DIR = PROJECT_DIR / 'output'
for d in [DATA_DIR, OUT_DIR]:
```

```python
    d.mkdir(parents=True, exist_ok=True)

# Parameters
TICKERS = ['AAPL', 'META', 'JPM']
START = '2023-01-01'
END = '2025-06-30'
WINDOW = 252                        # rolling/first-window length in periods (days)
ALPHAS = [0.95, 0.99]
np.random.seed(0)

print('Project dir:', PROJECT_DIR)
print('Output ->', OUT_DIR)

# %% [markdown]
# Download and save Adjusted Close for the tickers over the given range into `data/` (CSV per ticker).

# %%
import yfinance as yf

print('Downloading data to', DATA_DIR)
for t in TICKERS:
    print(f'  -> {t}')
    df = yf.download(t, start=START, end=END, progress=False, auto_adjust=False)
    if df.empty:
        print(f'     Warning: no data for {t}')
        continue
    out = df.reset_index()
    out = out[['Date', 'Adj Close']]
    out.to_csv(DATA_DIR / f'{t}.csv', index=False)
print('Done.')

# %% [markdown]
# Now, load the data back from CSVs

# %%
files = [f for f in os.listdir(DATA_DIR) if f.lower().endswith('.csv')]
frames = []
for f in files:
    p = os.path.join(DATA_DIR, f)
    df = pd.read_csv(p, parse_dates=['Date'])
    df = df[['Date', 'Adj Close']]
    # Coerce to numeric and drop malformed rows
    df['Adj Close'] = pd.to_numeric(df['Adj Close'], errors='coerce')
    df = df.dropna(subset=['Date', 'Adj Close'])
    df = df.rename(columns={'Adj Close': f.split('.')[0]})
    df = df.set_index('Date').sort_index()
    frames.append(df)
prices = pd.concat(frames, axis=1).dropna(how='all')

print(prices.head(2))

# %% [markdown]
# ## 1) Empirical stylized facts

# %% [markdown]
```

```
#
# 1.  Construct daily log-returns for AAPL, MSFT, JPM; plot series and comment on trends/volatility.
# 2.  Estimate correlation functions of returns and absolute returns across assets and lags 0  25  ; comment.
# 3.  QQ plots vs Normal; perform  J a r q u e Bera  test and discuss normality.
#

# %% [markdown]
# ## a)

# %%
# Check for missing values
prices.isna().sum()

# %%
# Compute log-returns
log_ret = np.log(prices / prices.shift(1)).dropna()
print(log_ret.head(2))

# %%
ax = log_ret.plot(subplots=True, figsize=(12, 8), legend=False, grid=True)
for a, col in zip(ax, log_ret.columns):
    a.set_title(f'Log-returns of {col}')
ax[-1].set_xlabel('Date')
plt.tight_layout()
plt.show()


# %% [markdown]
# - Log-returns fluctuate around zero     no visible trend (mean ~ 0)
# - Volatility is not constant     clear volatility clustering
# - Occasional large spikes     evidence of fat tails / extreme events
# - META shows highest overall volatility; JPM the lowest
# - Time-varying volatility suggests need for GARCH-type models

# %% [markdown]
# ## b)

# %%
# cross correlation function
def cross_correlation_lags(x, y, max_lag):
    lags = range(0, max_lag + 1)
    corr_values = [x.corr(y.shift(l)) for l in lags]
    return pd.Series(corr_values, index=lags, name="cross_corr")

#Autocorrelation function
def ACF(x, max_lags):
    lags = range(0, max_lags + 1)
    acf_values = [x.corr(x.shift(l)) for l in lags]
    return pd.Series(acf_values, index=lags, name="autocorr")

# %% [markdown]
# ### Cross-correlation

# %%
```

```python
# all possible pairs of tickers
pairs = [(t1, t2) for t1 in TICKERS for t2 in TICKERS if t1 != t2]

# Plotting cross-correlation for log_returns
fig, axes = plt.subplots(2, 3, figsize=(15, 6), sharex=True, sharey=True)
axes = axes.flatten()

for ax, (t1, t2) in zip(axes, pairs):
    ccl = cross_correlation_lags(log_ret[t1], log_ret[t2], max_lag=25)
    ax.stem(ccl.index, ccl.values, basefmt=' ')
    ax.set_title(f'{t1} vs {t2}')
    ax.set_xlabel('Lag')
    ax.set_ylabel('Correlation')
    ax.grid(True)

plt.suptitle('Cross-correlation of log returns between stocks in function of lag', y=1.02)
plt.tight_layout()
plt.show()

# plotting cross-correlation for absolute log_returns
fig, axes = plt.subplots(2, 3, figsize=(15, 6), sharex=True, sharey=True)
axes = axes.flatten()

for ax, (t1, t2) in zip(axes, pairs):
    ccl = cross_correlation_lags(log_ret[t1].abs(), log_ret[t2].abs(), max_lag=25)
    ax.stem(ccl.index, ccl.values, basefmt=' ')
    ax.set_title(f'{t1} vs {t2}')
    ax.set_xlabel('Lag')
    ax.set_ylabel('Correlation')
    ax.grid(True)

plt.suptitle('Cross-correlation of absolute log returns between stocks in function of lag', y=1.02)
plt.tight_layout()
plt.show()




# %% [markdown]
# ### Autocorrelation

# %%
# Plotting autocorrelation for log_returns
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharex=True, sharey=True)
for ax, ticker in zip(axes, TICKERS):
    acl = ACF(log_ret[ticker], max_lags=25)
    ax.stem(acl.index, acl.values, basefmt=' ')
    ax.set_title(f'Autocorrelation of {ticker} log returns')
    ax.set_xlabel('Lag')
    ax.set_ylabel('Autocorrelation')
    ax.grid(True)
```

```
plt.suptitle('Autocorrelation of log returns in function of lag', y=1.02)
plt.tight_layout()
plt.show()

# Plotting autocorrelation for absolute log_returns
fig, axes = plt.subplots(1, 3, figsize=(15, 4), sharex=True, sharey=True)
for ax, ticker in zip(axes, TICKERS ):
    acl = ACF(log_ret[ticker].abs(), max_lags=25)
    ax.stem(acl.index, acl.values, basefmt=' ')
    ax.set_title(f'Autocorrelation of |{ticker}| log returns')
    ax.set_xlabel('Lag')
    ax.set_ylabel('Autocorrelation')
    ax.grid(True)
plt.suptitle('Autocorrelation of absolute log returns in function of lag', y=1.02)
plt.tight_layout()
plt.show()

# %% [markdown]
# - Volatility clustering: strong autocorrelation in |returns| volatility is persistent.
# - Linear dependence: raw returns show near-zero autocorrelation no predictability.
# - Cross-asset linkages: strong contemporaneous correlations (esp. M E T A AAPL ), but lagged cross-correlations drop to zero no l e a d lag effects.

# %% [markdown]
# ### QQ-Plot

# %%
for col in log_ret.columns:
    stats.probplot(log_ret[col], dist="norm", plot=plt)
    plt.title(f"QQ Plot of {col} returns")
    plt.grid(True)
    plt.show()

# %% [markdown]
# The QQ-plots show clear deviations from the normal line in both tails, indicating that all three assets exhibit heavy-tailed return distributions. While the central quantiles roughly follow the Gaussian fit, the extreme quantiles diverge substantially, providing strong visual evidence against normality.

# %% [markdown]
# ### J a r q u e Bera test

# %%
for col in log_ret.columns:
    stat, p = jarque_bera(log_ret[col])
    print(f"{col}: JB={stat:.2f}, p-value={p:.4g}")

# %% [markdown]
# The J a r q u e Bera test confirms this: all assets display extremely large JB statistics with p-values equal to zero, leading to a decisive rejection of normality. Together, these results show that
```

```
    returns are fat-tailed and that Gaussian-based risk measures would
    underestimate tail risk.

# %% [markdown]
# ## 2) First-window modeling: VaR, ES, and distributions

# %% [markdown]
# Use the first estimation window W (days) on each asset separately
    with losses L =   R  .
# Compare:
#
# - historical,
# - Gaussian,
# - Student-t,
# - AR(p)+GARCH(1,1) with Normal/Student-t,
# - Filtered Historical Simulation (FHS).

# %% [markdown]
# ### Fitting of the different models

# %%
def tail_mean_es(x, var_value):
    """
    ES estimator = mean of x knowing x >= VaR. If tail empty, returns
        VaR.
    """
    x = np.asarray(x, dtype=float)
    tail = x[x >= var_value]
    return float(tail.mean()) if tail.size > 0 else float(var_value)


# ============================================================
# Historical Simulation  (on LOSSES)
# ============================================================

def forecast_hs(L_window, alpha, quantile_method="weibull"):
    """
    Inputs:
      L_window: pd.Series of losses (length W)
      alpha: confidence level (0.95 or 0.99)
    Outputs:
      (VaR, ES, params) where params is used later for simulation in Z1
          .
    """
    x = np.asarray(pd.Series(L_window).dropna().values, dtype=float)
    var = np.quantile(x, alpha, method=quantile_method)
    es = tail_mean_es(x, var)
    params = {"sample_losses": x}  # for resampling later
    return float(var), float(es), params

def simulate_hs(params, size, rng):
    """
    Simulate 1-step losses from HS by bootstrap resampling.
    """
    x = params["sample_losses"]
    return rng.choice(x, size=int(size), replace=True)


```

```python
# ============================================================
# Gaussian  (fit on LOSSES)
# ============================================================

def forecast_gaussian(L_window, alpha):
    x = np.asarray(pd.Series(L_window).dropna().values, dtype=float)
    mu, sigma = stats.norm.fit(x)
    sigma = float(sigma) if float(sigma) > 0 else float(np.std(x, ddof=1) if len(x) > 1 else 1e-12)

    z = norm.ppf(alpha)
    var = mu + sigma * z
    es = mu + sigma * norm.pdf(z) / (1.0 - alpha)

    params = {"mu": float(mu), "sigma": float(sigma)}
    return float(var), float(es), params

def simulate_gaussian(params, size, rng):
    mu, sigma = params["mu"], params["sigma"]
    return mu + sigma * rng.standard_normal(size=int(size))


# ============================================================
# Student-t  (fit on LOSSES)
# ============================================================

def forecast_student_t(L_window, alpha):
    x = np.asarray(pd.Series(L_window).dropna().values, dtype=float)
    nu, mu, sigma = stats.t.fit(x)
    nu = float(nu); mu = float(mu); sigma = float(sigma) if float(sigma) > 0 else 1e-12

    q = stats.t.ppf(alpha, df=nu)
    pdf_q = stats.t.pdf(q, df=nu)

    var = mu + sigma * q

    # ES exists only when nu > 1
    if nu <= 1:
        es = np.inf
    else:
        es = mu + sigma * pdf_q * (nu + q**2) / ((1.0 - alpha) * (nu - 1.0))

    params = {"nu": nu, "mu": mu, "sigma": sigma}
    return float(var), float(es), params


def simulate_student_t(params, size, rng):
    nu, mu, sigma = params["nu"], params["mu"], params["sigma"]
    return mu + sigma * rng.standard_t(df=nu, size=int(size))


# ============================================================
# PACF-based AR order selection
#   Computes PACF, identifies significant lags, and selects the
#   AR order as the first sequence of consecutive significant lags.
# ============================================================
```

```python

def PACF_with_order(x, max_lags, alpha, plot=True):
    """
    Computes the PACF of a time series, checks which lags are significantly different
    from zero, and automatically selects an AR order based on the first sequence of
    consecutive significant partial autocorrelations.
    """

    pacf_vals, confint = pacf(x, nlags=max_lags, alpha=1-alpha, method="ywmle")
    lags = np.arange(len(pacf_vals))

    # significant if CI does NOT contain 0
    sig_mask = (confint[:, 0] > 0) | (confint[:, 1] < 0)
    sig_mask[0] = False  # ignore lag 0

    # count how many consecutive lags starting from 1 are significant
    p = 0
    for lag in range(1, len(sig_mask)):
        if sig_mask[lag]:
            p += 1
        else:
            break

    # Plot integrated in the PACF function
    if plot:
        plt.figure(figsize=(8, 4))
        markerline, stemlines, baseline = plt.stem(lags, pacf_vals, basefmt=" ")
        plt.setp(stemlines, linewidth=1.0)

        # horizontal axis at 0
        plt.hlines([0], 0, max_lags, colors="k")

        upper_bound = confint[1, 1]
        lower_bound = confint[1, 0]
        plt.axhline(upper_bound, linestyle="--", linewidth=0.8)
        plt.axhline(lower_bound, linestyle="--", linewidth=0.8)

        if p > 0:
            plt.axvline(p, color="red", linestyle="--", alpha=0.7,
                        label=f"selected p = {p}")
            plt.legend()

        plt.xlabel("Lag")
        plt.ylabel("PACF")
        plt.title(f"PACF (selected p = {p}) - Alpha = {alpha}")
        plt.xlim(-0.5, max_lags + 0.5)
        plt.tight_layout()
        plt.show()

    return p, pacf_vals, confint


def simulate_student_t(params, size, rng):
    nu, mu, sigma = params["nu"], params["mu"], params["sigma"]
```

```python
    return mu + sigma * rng.standard_t(df=nu, size=int(size))


# ============================================================
# AR(p)-GARCH(1,1) Gaussian innovations (fit on RETURNS)
#     We forecast distribution of next-day LOSS:
#       R_{t+1}|F_t ~ N(mu_next, sigma_next^2)
#       L_{t+1} = -R_{t+1} ~ N(-mu_next, sigma_next^2)
# ============================================================

def fit_ar_garch_gaussian(r_window, p, scale=100.0, disp="off"):
    """
    Fit AR(p)-GARCH(1,1) with normal innovations on RETURNS (scaled by 'scale').
    Returns 'res' (arch_model fit result).
    """
    r = np.asarray(pd.Series(r_window).dropna().values, dtype=float)
    r_s = r * float(scale)
    am = arch_model(r_s, mean="ARX", lags=int(p), vol="GARCH", p=1, q=1, dist="normal")
    res = am.fit(disp=disp)
    return res


def garch_one_step_mu_sigma(res, scale=100.0):
    """
    Extract 1-step ahead forecast of RETURN mean and std (de-scaled).
    """
    f = res.forecast(horizon=1, reindex=False)
    mu_next_s = float(f.mean.values[-1, 0])
    var_next_s = float(f.variance.values[-1, 0])

    mu_next = mu_next_s / float(scale)
    sigma_next = np.sqrt(max(var_next_s, 0.0)) / float(scale)
    sigma_next = float(sigma_next) if sigma_next > 0 else 1e-12
    return float(mu_next), float(sigma_next)

def forecast_ar_garch_gaussian_from_mu_sigma(mu_next, sigma_next, alpha):
    """
    Compute VaR/ES for LOSS using the predicted N(-mu_next, sigma_next^2).
    """
    z = norm.ppf(alpha)
    var = (-mu_next) + sigma_next * z
    es = (-mu_next) + sigma_next * norm.pdf(z) / (1.0 - alpha)
    params = {"mu_next": float(mu_next), "sigma_next": float(sigma_next)}
    return float(var), float(es), params

def simulate_ar_garch_gaussian(params, size, rng):
    """
    Simulate 1-step losses under predicted Gaussian conditional distribution.
    """
    mu_next, sigma_next = params["mu_next"], params["sigma_next"]
    r_sim = mu_next + sigma_next * rng.standard_normal(size=int(size))
    return -r_sim # "-" to have the result on LOSSES
```

```python


# ================================================================
# Filtered Historical Simulation (FHS) (using SAME GARCH fit)
# ================================================================

def prepare_std_resid_pool(res):
    """
    Return standardized residuals z_t (mean0,var1) as numpy array for bootstrap.
    """
    z = np.asarray(pd.Series(res.std_resid).dropna().values, dtype=float)
    if z.size == 0:
        raise ValueError("No standardized residuals from this GARCH fit.")
    z = (z - z.mean()) / z.std(ddof=1) #ddof=1 for the non-biased std
    return z


def forecast_fhs_from_res(res, alpha, M=20000, random_state=42, scale=100.0, quantile_method="weibull"):
    """
    Build FHS 1-step loss distribution:
      L = -(mu_next + sigma_next * z*),  z* resampled from standardized residuals pool.
    Returns (VaR, ES, params), where params stores (mu_next, sigma_next, z_pool).
    """
    rng = np.random.default_rng(int(random_state))
    mu_next, sigma_next = garch_one_step_mu_sigma(res, scale=scale)
    z_pool = prepare_std_resid_pool(res)

    z_star = rng.choice(z_pool, size=int(M), replace=True)
    # hygiene (optional but stable)
    if len(z_star) > 1:
        z_star = (z_star - z_star.mean()) / z_star.std(ddof=1)

    r_sim = mu_next + sigma_next * z_star
    L_sim = -r_sim

    var = np.quantile(L_sim, alpha, method=quantile_method)
    es = tail_mean_es(L_sim, var)

    params = {"mu_next": float(mu_next), "sigma_next": float(sigma_next), "z_pool": z_pool}
    return float(var), float(es), params


def simulate_fhs(params, size, rng):
    mu_next, sigma_next = params["mu_next"], params["sigma_next"]
    z_pool = params["z_pool"]
    z_star = rng.choice(z_pool, size=int(size), replace=True)
    if len(z_star) > 1:
        z_star = (z_star - z_star.mean()) / z_star.std(ddof=1)
    r_sim = mu_next + sigma_next * z_star
    return -r_sim

```

```python
# %%
W = 252
max_lags = 25
alphas = [0.95, 0.99]
results = []

# %% [markdown]
# ### Historical Simulation

# %%
for ticker in TICKERS:
    print(f"\nAsset: {ticker}")
    for alpha in alphas:
        VaR_hs, ES_hs, param_hs = forecast_hs(L_window=-log_ret[ticker].iloc[:W], alpha=alpha)
        print(f"  alpha = {alpha:.2f} :  VaR = {VaR_hs:.6f}, ES = {ES_hs:.6f}")

        results.append({'method': 'HS','ticker': ticker,'alpha': alpha,'VaR': VaR_hs,'ES': ES_hs})

# %% [markdown]
# ### Gaussian

# %%
for ticker in TICKERS:
    print(f"\nAsset: {ticker}")
    for alpha in alphas:
        VaR_gauss, ES_gauss, param_gauss = forecast_gaussian(L_window=-log_ret[ticker].iloc[:W], alpha=alpha)
        print(f"  alpha = {alpha:.2f} :  VaR = {VaR_gauss:.6f}, ES = {ES_gauss:.6f} (Gaussian fit : mu={param_gauss['mu']:.6f}, sigma={param_gauss['sigma']:.6f})")

        results.append({'method': 'Gaussian','ticker': ticker,'alpha': alpha,'VaR': VaR_gauss,'ES': ES_gauss})

# %% [markdown]
# ### Student-T

# %%
for ticker in TICKERS:
    print(f"\nAsset: {ticker}")
    for alpha in alphas:
        VaR_stud, ES_stud, param_stud = forecast_student_t(L_window=-log_ret[ticker].iloc[:W], alpha=alpha)
        print(f"  alpha = {alpha:.2f} :  VaR = {VaR_stud:.6f}, ES = {ES_stud:.6f} (student fit : nu ={param_stud['nu']:.6f}, mu={param_stud['mu']:.6f}, sigma={param_stud['sigma']:.6f})")

        results.append({'method': 'Student-t','ticker': ticker,'alpha': alpha,'VaR': VaR_stud,'ES': ES_stud})

# %% [markdown]
# ### Conditional Parametric - Selection of AR order

# %%
results_p = []
```

```python

for ticker in TICKERS:
    for alpha in alphas:
        print(f"\n=== {ticker} | alpha = {int(alpha*100)}% ===")
        p, pacf_vals, confint = PACF_with_order(
            log_ret[ticker],
            max_lags=max_lags,
            alpha=alpha,
            plot=False
        )
        print("PACF suggested AR order p =", p)

        # store result for later use
        results_p.append(
            {"asset": ticker,
             "conf_level": alpha,
             "p": p}
        )

p_table = pd.DataFrame(results_p)
p_matrix = p_table.pivot(index="asset", columns="conf_level", values="p")

# %% [markdown]
# ### AR(p) - GARCH(1,1) fitting

# %%
for ticker in TICKERS:
    for alpha in alphas:

        # AR order from PACF selection
        p = int(p_matrix.loc[ticker, alpha]) # alwas 0 for us but this form for generality

        # take window of returns
        r_window = log_ret[ticker].iloc[:W]

        # fit AR(p) GARCH (1,1) (Gaussian)
        res = fit_ar_garch_gaussian(r_window=r_window, p=p, scale=100.0, disp="off")

        # 1-step ahead forecasts (de-scaled)
        mu_next, sigma_next = garch_one_step_mu_sigma(res, scale=100.0)

        # diagnostics: LjungBox on standardized residuals
        std_resid = np.asarray(res.std_resid, dtype=float)
        # lags = 10
        lb_mean_10 = acorr_ljungbox(std_resid, lags=[10], return_df=True)
        lb_arch_10 = acorr_ljungbox(std_resid**2, lags=[10], return_df=True)
        lb_stat_10    = lb_mean_10["lb_stat"].iloc[0]
        lb_pval_10    = lb_mean_10["lb_pvalue"].iloc[0]
        lb_pval_10_sq = lb_arch_10["lb_pvalue"].iloc[0]
        # lags = 25
        lb_mean_25 = acorr_ljungbox(std_resid, lags=[25], return_df=True)
```

```
        lb_arch_25 = acorr_ljungbox(std_resid**2, lags=[25], return_df=True)
        lb_stat_25    = lb_mean_25["lb_stat"].iloc[0]
        lb_pval_25    = lb_mean_25["lb_pvalue"].iloc[0]
        lb_pval_25_sq = lb_arch_25["lb_pvalue"].iloc[0]

        # VaR/ES (Gaussian)
        VaR_norm, ES_norm, params_garch_gauss = forecast_ar_garch_gaussian_from_mu_sigma(mu_next, sigma_next, alpha=alpha)

        print(f"\n=== {ticker} | alpha = {int(alpha*100)}% ===")
        print(f"  LB Q(10) p (mean)          : {lb_pval_10:.3f}")
        print(f"  LB Q(10) p (squared)       : {lb_pval_10_sq:.3f}")
        print(f"  LB Q(25) p (mean)          : {lb_pval_25:.3f}")
        print(f"  LB Q(25) p (squared)       : {lb_pval_25_sq:.3f}")
        print(f"  mu_hat(t+1), sigma_hat(t+1): {mu_next:.3e}, {sigma_next:.3e}")
        print(f"  VaR (Gaussian)             : {VaR_norm:.3e}")
        print(f"  ES  (Gaussian)             : {ES_norm:.3e}")

        # store results
        results.append({
            'method': 'AR(p)-GARCH(1,1) (Gaussian)',
            'ticker': ticker,
            'alpha': alpha,
            'VaR': float(VaR_norm),
            'ES':  float(ES_norm)
        })


# %% [markdown]
# ### Filtered Historical Simulation (FHS)

# %%
results_fhs = []
random_state = 42
M = 1000
scale = 100.0

for ticker in TICKERS:
    r_win = log_ret[ticker].iloc[:W].dropna()
    for alpha in alphas:
        p = int(p_matrix.loc[ticker, alpha])

        # Fit AR(p)-GARCH(1,1) on scaled returns
        am = arch_model(r_win * scale, mean='ARX', lags=p, vol='GARCH', p=1, q=1, dist='normal')
        res = am.fit(disp='off')

        # FHS on that fitted model (pass the *confidence level* alpha)
        var_fhs, es_fhs, params = forecast_fhs_from_res(res, alpha=alpha, M=M, random_state=random_state, scale=scale, quantile_method="weibull")

        results_fhs.append({
            "asset": ticker,
            "conf_level": alpha,
```

```
            "p": p,
            "VaR_FHS": var_fhs,
            "ES_FHS": es_fhs,
            "mu_next": params["mu_next"],
            "sigma_next": params["sigma_next"]
        })

        print(f"\n=== {ticker} | alpha = {int(alpha*100)}% ===")
        print(f"VaR_FHS={var_fhs:.6f}  ES_FHS={es_fhs:.6f}")

        results.append({
            'method': 'AR(p)-GARCH(1,1) (Gaussian) - FHS',
            'ticker': ticker,
            'alpha': alpha,
            'VaR': float(var_fhs),
            'ES':  float(es_fhs)
        })


# %% [markdown]
# ### Plot of distribution

# %%
from scipy import stats
from scipy.stats import gaussian_kde

def simulate_and_plot_fitted_loss_distributions(L_series, r_window, p, scale=100.0,
                                                title_suffix="", M=2_000, rng=None):
    """
    Plot histogram of losses L (historical window) and overlay KDE densities from:
      - HS (bootstrap)
      - Gaussian (parametric)
      - Student-t (parametric)
      -  A R GARCH  predictive (Gaussian innovations)
      - FHS predictive
    All simulations use your simulate_* functions (consistent with Z1 setup).
    """
    if rng is None:
        rng = np.random.default_rng(42)

    # historical losses in the window
    L = np.asarray(pd.Series(L_series).dropna().values, dtype=float)
    if L.size < 5:
        raise ValueError("Need more data in L_series to plot densities.")

    # grid for plotting
    lo, hi = np.quantile(L, [0.01, 0.99])
    pad = 0.25 * (hi - lo + 1e-12)
    grid = np.linspace(lo - pad, hi + pad, 800)

    # --- Fit/params for HS/Gauss/Student on LOSSES (alpha irrelevant here) ---
    # we can pass any alpha; it doesn't affect params for Gauss/t in your code (only VaR/ES)
```

```
    dummy_alpha = 0.95

    _, _, params_hs = forecast_hs(L, dummy_alpha)
    _, _, params_g  = forecast_gaussian(L, dummy_alpha)
    _, _, params_t  = forecast_student_t(L, dummy_alpha)

    # --- Fit AR-GARCH on RETURNS window, then create params for AR-GARCH sim ---
    res = fit_ar_garch_gaussian(r_window=r_window, p=int(p), scale=scale, disp="off")
    mu_next, sigma_next = garch_one_step_mu_sigma(res, scale=scale)
    _, _, params_ag = forecast_ar_garch_gaussian_from_mu_sigma(mu_next, sigma_next, dummy_alpha)

    # --- Fit FHS params from same GARCH fit ---
    # Use forecast_fhs_from_res to get a params dict with z_pool, mu_next, sigma_next
    _, _, params_fhs = forecast_fhs_from_res(
        res=res,
        alpha=dummy_alpha,
        M=2000,                 # only for VaR/ES in that function (not used here)
        random_state=123,
        scale=scale
    )

    # --- Simulate samples using your simulate_* ---
    L_sim_hs  = simulate_hs(params_hs, M, rng)
    L_sim_g   = simulate_gaussian(params_g, M, rng)
    L_sim_t   = simulate_student_t(params_t, M, rng)
    L_sim_ag  = simulate_ar_garch_gaussian(params_ag, M, rng)
    L_sim_fhs = simulate_fhs(params_fhs, M, rng)

    # --- KDEs (density estimates) ---
    kde_hs  = gaussian_kde(L_sim_hs)
    kde_g   = gaussian_kde(L_sim_g)
    kde_t   = gaussian_kde(L_sim_t)
    kde_ag  = gaussian_kde(L_sim_ag)
    kde_fhs = gaussian_kde(L_sim_fhs)

    # --- Plot ---
    plt.figure(figsize=(9, 5))
    plt.hist(L, bins=35, density=True, alpha=0.45, label="Historical losses (hist)")

    plt.plot(grid, kde_hs(grid),  linewidth=1.6, label="HS (bootstrap KDE)")
    plt.plot(grid, kde_g(grid),   linewidth=1.6, label="Gaussian (sim KDE)")
    plt.plot(grid, kde_t(grid),   linewidth=1.6, label="Student-t (sim KDE)")
    plt.plot(grid, kde_ag(grid),  linewidth=1.8, label="AR-GARCH pred (sim KDE)")
    plt.plot(grid, kde_fhs(grid), linewidth=1.8, label="FHS pred (sim KDE)")

    plt.xlabel("Loss L")
    plt.ylabel("Density")
```

```python
    plt.title(f"Loss distributions - {title_suffix}")
    plt.legend()
    plt.tight_layout()
    plt.show()



# %%
for ticker in TICKERS:
    L_series = (-log_ret[ticker].iloc[:W]).dropna()

    # again, p from AR(p) is always p=0 for us but for generality
    p = int(p_matrix.loc[ticker, 0.95])

    simulate_and_plot_fitted_loss_distributions(
        L_series=L_series,
        r_window=log_ret[ticker].iloc[:W],
        p=p,
        title_suffix=ticker,
        M=1000,
        scale=100.0
    )


# %% [markdown]
# ### Comparison of results

# %%
# transforming results into a pd.DataFrame for manipulations
if isinstance(results, list):
    results = pd.DataFrame(results)

results = results[['method', 'ticker', 'alpha', 'VaR', 'ES']]

# sorting for nicer plots
results = results.sort_values(['alpha'])

tickers = results['ticker'].unique()
alphas  = sorted(results['alpha'].unique())

for t in tickers:
    sub = results[results['ticker'] == t].copy()
    methods = sub['method'].unique()
    x = np.arange(len(methods))

    fig, axes = plt.subplots(1, 2, figsize=(12, 4), sharey=False)
    fig.suptitle(f'{t}     VaR and ES by method and alpha')

    metrics = ['VaR', 'ES']

    for ax, metric in zip(axes, metrics):
        # width so that all alpha bars fit nicely in [ 0  .4, 0.4]
        width = 0.8 / len(alphas)

        for i, a in enumerate(alphas):
            data_a = sub[sub['alpha'] == a].set_index('method').reindex
                (methods)
            heights = data_a[metric].values
```

```python

            # shifting each alpha group to the left/right
            ax.bar(x + (i - (len(alphas)-1)/2) * width, heights,
                   width=width, label=f'alpha={a}')

        ax.set_title(metric)
        ax.set_xlabel('Method')
        ax.set_ylabel(metric)
        ax.set_xticks(x)
        ax.set_xticklabels(methods, rotation=45, ha='right')
        ax.legend()

    plt.tight_layout()
    plt.show()


# %% [markdown]
# ## 3) Rolling-window backtesting of VaR and ES

# %% [markdown]
# Use a rolling window of size W to produce 1-step-ahead VaR/ES at 95% and 99% for each method in Exercise 2. Then, implement the following statistical tests:
#
# - VaR backtests: Kupiec POF and Christoffersen independence tests.
# - ES backtest:  A c e r b i Szkely  Z1 test.

# %% [markdown]
# ## 3.0) Elementary functions for part 3)

# %%
def build_rolling_forecasts(log_ret, TICKERS, conf_level, W, p_matrix,
                            scale=100.0, M_fhs=20000, seed_fhs=42,
                            quantile_method="weibull",
                            store_params=True):
    """
    Build rolling 1-step ahead VaR/ES forecasts (on LOSSES) for multiple methods.

    Output DataFrame columns:
      - date: date of the realized point (t+1)
      - asset
      - alpha
      - method  in {"HS","Gaussian","Student-t","AR-GARCH","FHS"}
      - VaR, ES   (on losses)
      - L_real    realized loss at that date
      - R_real    realized return at that date (=-L_real)
      - breach    1 if L_real > VaR else 0
      - params    dict (model informations for Z1 backtest) or None if store_params=False (no need for kupiec/Christoffersen)
    """
    rows = []

    for ticker in TICKERS:
        r = log_ret[ticker].dropna()
        n = len(r)
        if n <= W:
```

```
            raise ValueError(f"Not enough data for {ticker}: need > W={W}, got {n}.")

        # end = index position of the last point in the estimation window (t)
        # forecast is for the next point (t+1) => realized at index end+1
        for end in range(W - 1, n - 1):
            r_window = r.iloc[end - W + 1 : end + 1]
            L_window = -r_window

            date = r.index[end + 1]
            R_real = float(r.iloc[end + 1])
            L_real = float(-R_real)

            for alpha in conf_level:

                # --------------------
                # HS
                # --------------------
                var_hs, es_hs, params_hs = forecast_hs(L_window, alpha, quantile_method=quantile_method)
                rows.append({
                    "date": date, "asset": ticker, "alpha": float(alpha),
                    "method": "HS",
                    "VaR": float(var_hs), "ES": float(es_hs),
                    "L_real": L_real, "R_real": R_real,
                    "breach": int(L_real > var_hs),
                    "params": params_hs if store_params else None
                })

                # --------------------
                # Gaussian
                # --------------------
                var_g, es_g, params_g = forecast_gaussian(L_window, alpha)
                rows.append({
                    "date": date, "asset": ticker, "alpha": float(alpha),
                    "method": "Gaussian",
                    "VaR": float(var_g), "ES": float(es_g),
                    "L_real": L_real, "R_real": R_real,
                    "breach": int(L_real > var_g),
                    "params": params_g if store_params else None
                })

                # --------------------
                # Student-t
                # --------------------
                var_t, es_t, params_t = forecast_student_t(L_window, alpha)
                rows.append({
                    "date": date, "asset": ticker, "alpha": float(alpha),
                    "method": "Student-t",
                    "VaR": float(var_t), "ES": float(es_t),
                    "L_real": L_real, "R_real": R_real,
```

```
                    "breach": int(L_real > var_t),
                    "params": params_t if store_params else None
                })

                # --------------------
                # AR(p)-GARCH(1,1) Gaussian, and (2.e) FHS
                # --------------------
                p = int(p_matrix.loc[ticker, alpha])

                # Fit once, reuse for AR-GARCH and FHS
                res = fit_ar_garch_gaussian(r_window, p=p, scale=scale, disp="off")
                mu_next, sigma_next = garch_one_step_mu_sigma(res, scale=scale)

                # AR-GARCH Gaussian (on LOSSES)
                var_ag, es_ag, params_ag = forecast_ar_garch_gaussian_from_mu_sigma(mu_next, sigma_next, alpha)
                rows.append({
                    "date": date, "asset": ticker, "alpha": float(alpha),
                    "method": "AR-GARCH",
                    "VaR": float(var_ag), "ES": float(es_ag),
                    "L_real": L_real, "R_real": R_real,
                    "breach": int(L_real > var_ag),
                    "params": params_ag if store_params else None
                })

                # FHS (we use a rolling seed so it doesn't reuse the same bootstrap pattern for every day)
                var_f, es_f, params_f = forecast_fhs_from_res(
                    res, alpha,
                    M=M_fhs,
                    random_state=int(seed_fhs + end),   # day-dependent seed
                    scale=scale,
                    quantile_method=quantile_method
                )
                rows.append({
                    "date": date, "asset": ticker, "alpha": float(alpha),
                    "method": "FHS",
                    "VaR": float(var_f), "ES": float(es_f),
                    "L_real": L_real, "R_real": R_real,
                    "breach": int(L_real > var_f),
                    "params": params_f if store_params else None
                })

    df_forecasts = pd.DataFrame(rows)
    df_forecasts.sort_values(["alpha", "date"], inplace=True)
    df_forecasts.reset_index(drop=True, inplace=True)
    return df_forecasts


# %%
# ---------------------------
# Helpers
```

```python
# ----------------------------

def breaches_from_var(L_real, VaR):
    """
    Return array of 0/1 breaches: I_t = 1{L_real_t > VaR_t}.
    Inputs can be list/np.array/pd.Series.
    """
    L = np.asarray(L_real, dtype=float)
    V = np.asarray(VaR, dtype=float)
    if L.shape != V.shape:
        raise ValueError("L_real and VaR must have the same shape.")
    return (L > V).astype(int)


def _loglik_bernoulli(n1, n0, p, eps=1e-12):
    """
    Log-likelihood for Bernoulli with p, with safe logs.
    """
    p = float(np.clip(p, eps, 1.0 - eps))
    return n1 * np.log(p) + n0 * np.log(1.0 - p)


# ============================================================
# Kupiec POF test (Unconditional coverage)
# ============================================================

def kupiec_pof_test(breaches, alpha, eps=1e-12):
    """
    Kupiec Proportion-Of-Failures test (unconditional coverage).

    H0: P(breach) = 1 - alpha
    Input:
      breaches: array-like of 0/1
      alpha: confidence level for VaR (0.95 or 0.99)
    Output: dict with LR stat, p-value, counts.
    """
    I = np.asarray(breaches, dtype=int)
    T = int(I.size)
    N = int(I.sum())
    p0 = float(1.0 - alpha)

    # MLE under H1
    phat = N / T if T > 0 else np.nan

    # log-likelihood under H0 and H1
    ll0 = _loglik_bernoulli(N, T - N, p0, eps=eps)
    ll1 = _loglik_bernoulli(N, T - N, phat if T > 0 else 0.5, eps=eps)

    LR = float(-2.0 * (ll0 - ll1))
    pval = float(1.0 - chi2.cdf(LR, df=1))

    return {
        "T": T,
        "N": N,
        "alpha": float(alpha),
        "p0": p0,
        "phat": float(phat),
        "LR_pof": LR,
```

```
        "pvalue_pof": pval,
    }


# ============================================================
# Christoffersen (Independence & Conditional Coverage)
# ============================================================

def christoffersen_independence_test(breaches, eps=1e-12):
    """
    Christoffersen independence test using 2-state Markov chain on
        breaches.

    H0: breaches are independent (i.i.d Bernoulli with constant p)
    H1: first-order Markov dependence (p01 != p11)

    Returns dict with LR_ind, p-value, and transition counts.
    """
    I = np.asarray(breaches, dtype=int)
    if I.size < 2:
        raise ValueError("Need at least 2 observations for
            Christoffersen independence test.")

    # Transition counts n_ij
    I0 = I[:-1]
    I1 = I[1:]
    n00 = int(((I0 == 0) & (I1 == 0)).sum())
    n01 = int(((I0 == 0) & (I1 == 1)).sum())
    n10 = int(((I0 == 1) & (I1 == 0)).sum())
    n11 = int(((I0 == 1) & (I1 == 1)).sum())

    # Under independence (restricted): single p_hat
    T = int(I.size)
    N = int(I.sum())
    phat = N / T if T > 0 else 0.5

    ll0 = _loglik_bernoulli(n01 + n11, n00 + n10, phat, eps=eps)

    # Under Markov (unrestricted): p01 and p11
    denom01 = n00 + n01
    denom11 = n10 + n11
    p01 = n01 / denom01 if denom01 > 0 else 0.0
    p11 = n11 / denom11 if denom11 > 0 else 0.0

    ll1 = 0.0
    if denom01 > 0:
        ll1 += _loglik_bernoulli(n01, n00, p01, eps=eps)
    if denom11 > 0:
        ll1 += _loglik_bernoulli(n11, n10, p11, eps=eps)

    LR = float(-2.0 * (ll0 - ll1))
    pval = float(1.0 - chi2.cdf(LR, df=1))

    return {
        "n00": n00, "n01": n01, "n10": n10, "n11": n11,
        "phat": float(phat),
        "p01": float(p01), "p11": float(p11),
        "LR_ind": LR,
```

```
        "pvalue_ind": pval,
    }


def christoffersen_conditional_coverage_test(breaches, alpha, eps=1e-12):
    """
    Christoffersen conditional coverage test:
      LR_cc = LR_pof + LR_ind  ~ chi2(df=2) under H0.

    Returns dict with:
      - Kupiec POF results,
      - Independence results,
      - Joint LR_cc and p-value.
    """
    out_pof = kupiec_pof_test(breaches, alpha, eps=eps)
    out_ind = christoffersen_independence_test(breaches, eps=eps)

    LR_cc = float(out_pof["LR_pof"] + out_ind["LR_ind"])
    pval_cc = float(1.0 - chi2.cdf(LR_cc, df=2))

    return {
        **out_pof,
        **out_ind,
        "LR_cc": LR_cc,
        "pvalue_cc": pval_cc,
    }


# ============================================================
# Acerbi & Szekely Z1 test for ES
# ============================================================

def z1_statistic(L_real, VaR, ES):
    """
    Compute observed Z1 and N (number of VaR breaches):
      I_t = 1{L_t > VaR_t}
      Z1 = (1/N) sum_t (L_t/ES_t - 1) I_t

    Returns (Z1, N). If N==0 -> (np.nan, 0)
    """
    L = np.asarray(L_real, dtype=float)
    V = np.asarray(VaR, dtype=float)
    E = np.asarray(ES, dtype=float)

    if not (L.shape == V.shape == E.shape):
        raise ValueError("L_real, VaR, ES must have the same shape.")

    I = (L > V)
    N = int(I.sum())
    if N == 0:
        return np.nan, 0

    z = (L[I] / E[I]) - 1.0
    return float(np.mean(z)), N


```

```python
def acerbi_szekely_z1_test(L_real, VaR, ES, params_list, simulate_one_step, M=1000, seed=123):
    """
    Simulation-based Z1 test (primarily one-sided for underestimation of ES).

    Inputs:
      L_real: array-like realized losses (out-of-sample)
      VaR:    array-like predicted VaR on losses (same length)
      ES:     array-like predicted ES on losses (same length)
      params_list: list-like of params dicts, one per date t (same length)
      simulate_one_step: function(params, size, rng) -> np.ndarray of simulated losses
                         (use your simulate_hs / simulate_gaussian / simulate_student_t / simulate_ar_garch_gaussian / simulate_fhs)
      M: number of simulations (1000 in the project)
      seed: RNG seed

    Output: dict with Z1_obs, pvalue (one-sided), and diagnostics.
    """
    L = np.asarray(L_real, dtype=float)
    V = np.asarray(VaR, dtype=float)
    E = np.asarray(ES, dtype=float)

    if not (L.shape == V.shape == E.shape):
        raise ValueError("L_real, VaR, ES must have the same shape.")
    if len(params_list) != len(L):
        raise ValueError("params_list must have the same length as L_real.")

    # Observed Z1
    Z1_obs, N_obs = z1_statistic(L, V, E)
    if N_obs == 0 or np.isnan(Z1_obs):
        return {
            "Z1_obs": np.nan,
            "N_obs": N_obs,
            "pvalue_one_sided": np.nan,
            "M_effective": 0,
            "note": "No VaR breaches in observed data (N_obs=0): Z1 test inconclusive."
        }

    rng = np.random.default_rng(int(seed))
    Z1_sims = []
    skipped = 0

    for m in range(int(M)):
        # simulate one loss per date (conditionally on that date params)
        L_sim = np.empty_like(L)
        for t in range(len(L)):
            L_sim[t] = float(simulate_one_step(params_list[t], 1, rng)[0])

        Z1_m, N_m = z1_statistic(L_sim, V, E)

```

```
        # If the simulated path has zero breaches, Z1 undefined; skip this simulation
        if N_m == 0 or np.isnan(Z1_m):
            skipped += 1
            continue

        Z1_sims.append(Z1_m)

    Z1_sims = np.asarray(Z1_sims, dtype=float)
    M_eff = int(Z1_sims.size)

    if M_eff == 0:
        return {
            "Z1_obs": float(Z1_obs),
            "N_obs": N_obs,
            "pvalue_one_sided": np.nan,
            "M_effective": 0,
            "note": "All simulations had zero breaches (cannot form null distribution)."
        }

    # One-sided p-value: reject for large positive Z1 (ES underestimation)
    pval = float(np.mean(Z1_sims >= Z1_obs))

    return {
        "Z1_obs": float(Z1_obs),
        "N_obs": int(N_obs),
        "pvalue_one_sided": pval,
        "M_requested": int(M),
        "M_effective": M_eff,
        "M_skipped_no_breach": int(skipped),
        "Z1_sim_mean": float(np.mean(Z1_sims)),
        "Z1_sim_std": float(np.std(Z1_sims, ddof=1)) if M_eff > 1 else 0.0,
    }


# %% [markdown]
# ## 3.a) Kupiec

# %%
from scipy.stats import chi2

# %%
# Parameters
W = 252
conf_level_bt = [0.95, 0.99]

# --- Build rolling forecasts (can take time because of GARCH/FHS) ---
df_forecasts = build_rolling_forecasts(
    log_ret=log_ret,
    TICKERS=TICKERS,
    conf_level=conf_level_bt,
    W=W,
    p_matrix=p_matrix,
    scale=100.0,
    M_fhs=20000,
```

```python
    seed_fhs=42,
    quantile_method="weibull",
    store_params=False      # Kupiec does NOT need params, so let's save some space
)

print(df_forecasts.head())
print("Rows:", len(df_forecasts))
print("Alphas:", sorted(df_forecasts["alpha"].unique()))
print("Methods:", sorted(df_forecasts["method"].unique()))

# Kupiec POF test for each (asset, method, alpha)
methods = ["HS", "Gaussian", "Student-t", "AR-GARCH", "FHS"]

kupiec_results = []

for ticker in TICKERS:
    for alpha_bt in conf_level_bt:
        for method in methods:
            sub = df_forecasts[
                (df_forecasts["asset"] == ticker) &
                (df_forecasts["method"] == method) &
                (df_forecasts["alpha"] == alpha_bt)
            ].sort_values("date")

            breaches = sub["breach"].to_numpy(dtype=int)

            out = kupiec_pof_test(breaches, alpha=alpha_bt)

            kupiec_results.append({
                "alpha": alpha_bt,
                "asset": ticker,
                "method": method,
                "T": out["T"],
                "N": out["N"],
                "expected_N": out["T"] * out["p0"],
                "phat": out["phat"],
                "p0": out["p0"],
                "LR_pof": out["LR_pof"],
                "pvalue_pof": out["pvalue_pof"],
            })

kupiec_df = (
    pd.DataFrame(kupiec_results)
      .sort_values(["alpha"])
      .reset_index(drop=True)
)

kupiec_df


# %%
# adding the explicit result of the Kupiec test to the dataframe
prim_confid_value = 0.05
kupiec_df["pass_5pct"] = kupiec_df["pvalue_pof"] > prim_confid_value
display(kupiec_df.loc[:, ["alpha","asset","method","pass_5pct"]])
display(kupiec_df.pivot_table(index=["alpha","asset","method"], values="pass_5pct", aggfunc="first", sort=False))
```

```
# (macro-interpretation : if (pass_5pct == True) => the model was "good" (w.r.t. this test, the corresponding alpha, and from a statistical pov (prim_confid_value)))

# %%
methods = ["HS", "Gaussian", "Student-t", "AR-GARCH", "FHS"]
conf_level_bt = [0.95, 0.99]

for ticker in TICKERS:
    for alpha_bt in conf_level_bt:
        plt.figure(figsize=(11, 4))
        sub0 = df_forecasts[(df_forecasts["asset"] == ticker) & (df_forecasts["alpha"] == alpha_bt)].sort_values("date")

        if len(sub0) == 0:
            continue

        plt.plot(sub0["date"], -sub0["R_real"], color="black", label="Realized Losses", linewidth=1.2)

        for method in methods:
            sub = sub0[sub0["method"] == method]
            if len(sub) == 0:
                continue
            plt.plot(sub["date"], sub["VaR"], label=f"VaR ({method})", linewidth=1.0)

        plt.title(f"{ticker} | alpha={alpha_bt} | VaR threshold")
        plt.legend(ncol=2)
        plt.tight_layout()
        plt.grid(True)
        plt.show()

# %% [markdown]
# ## 3.a) Christoffersen

# %%
conf_level_bt = [0.95, 0.99]
methods = ["HS", "Gaussian", "Student-t", "AR-GARCH", "FHS"]

christ_results = []

for ticker in TICKERS:
    for alpha_bt in conf_level_bt:
        for method in methods:
            sub = df_forecasts[
                (df_forecasts["asset"] == ticker) &
                (df_forecasts["method"] == method) &
                (df_forecasts["alpha"] == alpha_bt)
            ].sort_values("date")

            breaches = sub["breach"].to_numpy(dtype=int)

            out = christoffersen_conditional_coverage_test(breaches, alpha=alpha_bt)

            christ_results.append({
                "alpha": alpha_bt,
```

```
                "asset": ticker,
                "method": method,
                "T": out["T"],
                "N": out["N"],

                # Unconditional coverage (Kupiec)
                "LR_pof": out["LR_pof"],
                "pvalue_pof": out["pvalue_pof"],

                # Independence
                "n00": out["n00"], "n01": out["n01"], "n10": out["n10"], "n11": out["n11"],
                "p01": out["p01"], "p11": out["p11"],
                "LR_ind": out["LR_ind"],
                "pvalue_ind": out["pvalue_ind"],

                # Joint conditional coverage
                "LR_cc": out["LR_cc"],
                "pvalue_cc": out["pvalue_cc"],
            })

christ_df = (
    pd.DataFrame(christ_results)
      .sort_values(["alpha"])
      .reset_index(drop=True)
)

christ_df


# %%
# adding the explicit result of the Christoffersen tests to the dataframe
# pass/fail at 5% level again
christ_df["pass_cc_5pct"]  = christ_df["pvalue_cc"]  > prim_confid_value
christ_df["pass_pof_5pct"] = christ_df["pvalue_pof"] > prim_confid_value
christ_df["pass_ind_5pct"] = christ_df["pvalue_ind"] > prim_confid_value


# %%
christ_df

# %%
display(christ_df.loc[:, ["alpha","asset","method","pass_cc_5pct"]])
display(christ_df.pivot_table(index=["alpha","asset","method"], values="pass_cc_5pct", aggfunc="first", sort=False))

# %% [markdown]
# ## 3.a) Z1

# %%
# ES backtest (Acerbi Szekely Z1), alpha in {0.95, 0.99}
W = 252
conf_level_bt = [0.95, 0.99]

```

```python
# IMPORTANT: Z1 needs params => store_params=True
df_forecasts_z1 = build_rolling_forecasts(
    log_ret=log_ret,
    TICKERS=TICKERS,
    conf_level=conf_level_bt,
    W=W,
    p_matrix=p_matrix,
    scale=100.0,
    M_fhs=20000,
    seed_fhs=42,
    quantile_method="weibull",
    store_params=True
)

methods = ["HS", "Gaussian", "Student-t", "AR-GARCH", "FHS"]
M = 1000
seed = 123

z1_results = []

for ticker in TICKERS:
    for alpha_bt in conf_level_bt:
        for method in methods:
            sub = df_forecasts_z1[
                (df_forecasts_z1["asset"] == ticker) &
                (df_forecasts_z1["method"] == method) &
                (df_forecasts_z1["alpha"] == alpha_bt)
            ].sort_values("date")

            L_real = sub["L_real"].to_numpy(dtype=float)
            VaR    = sub["VaR"].to_numpy(dtype=float)
            ES     = sub["ES"].to_numpy(dtype=float)
            params_list = sub["params"].tolist()

            if method == "HS":
                simulate_one_step = simulate_hs
            elif method == "Gaussian":
                simulate_one_step = simulate_gaussian
            elif method == "Student-t":
                simulate_one_step = simulate_student_t
            elif method == "AR-GARCH":
                simulate_one_step = simulate_ar_garch_gaussian
            elif method == "FHS":
                simulate_one_step = simulate_fhs
            else:
                raise ValueError("Unknown method: " + str(method))

            out = acerbi_szekely_z1_test(
                L_real=L_real,
                VaR=VaR,
                ES=ES,
                params_list=params_list,
                simulate_one_step=simulate_one_step,
                M=M,
                seed=seed
            )

            z1_results.append({
```

```
                    "alpha": alpha_bt,
                    "asset": ticker,
                    "method": method,
                    "Z1_obs": out.get("Z1_obs", np.nan),
                    "N_obs": out.get("N_obs", 0),
                    "pvalue_one_sided": out.get("pvalue_one_sided", np.nan),
                    "M_effective": out.get("M_effective", 0),
                    "M_skipped_no_breach": out.get("M_skipped_no_breach", 0),
                })

z1_df = (
    pd.DataFrame(z1_results)
      .sort_values(["alpha"])
      .reset_index(drop=True)
)

z1_df

# %%
# adding the explicit result of the Z1 test to the dataframe
# pass/fail at 5% level
z1_df["pass_5pct"] = z1_df["pvalue_one_sided"] > prim_confid_value

# %%
display(z1_df.loc[:, ["alpha","asset","method","pass_5pct"]])
display(z1_df.pivot_table(index=["alpha","asset","method"], values="pass_5pct", aggfunc="first", sort=False))


# %% [markdown]
# ## Compact&Visual results of backtests

# %%
# merging tables (keeping only what we need)
k = kupiec_df[["alpha", "asset", "method", "pvalue_pof", "pass_5pct"]].rename(
    columns={"pvalue_pof": "p_kupiec", "pass_5pct": "kupiec_pass"}
)

c = christ_df[["alpha", "asset", "method", "pvalue_cc", "pass_cc_5pct"]].rename(
    columns={"pvalue_cc": "p_chris_cc", "pass_cc_5pct": "chris_cc_pass"}
)

z = z1_df[["alpha", "asset", "method", "pvalue_one_sided", "pass_5pct", "N_obs"]].rename(
    columns={"pvalue_one_sided": "p_z1", "pass_5pct": "z1_pass", "N_obs": "N_obs_z1"}
)

summary = (
    k.merge(c, on=["alpha", "asset", "method"], how="outer")
     .merge(z, on=["alpha", "asset", "method"], how="outer")
)

```

```python
# color cells to display visually
def fmt_cell(p, passed, label="p"):
    if pd.isna(passed):
        return "    (inconcl.)"
    if pd.isna(p):
        return "    (p=NaN)"
    return ("     " if bool(passed) else "     ") + f"({label}={p:.3f})"

summary["Kupiec"] = [fmt_cell(p, b, "p") for p, b in zip(summary["p_kupiec"], summary["kupiec_pass"])]
summary["Chris-CC"] = [fmt_cell(p, b, "p") for p, b in zip(summary["p_chris_cc"], summary["chris_cc_pass"])]
summary["Z1 (one-sided)"] = [fmt_cell(p, b, "p") for p, b in zip(summary["p_z1"], summary["z1_pass"])]

# simple score (count only available tests)
summary["score_total"] = (
    summary["kupiec_pass"].fillna(False).astype(int) +
    summary["chris_cc_pass"].fillna(False).astype(int) +
    summary["z1_pass"].fillna(0).astype(int)
)

summary["tests_counted"] = (
    summary["kupiec_pass"].notna().astype(int) +
    summary["chris_cc_pass"].notna().astype(int) +
    summary["z1_pass"].notna().astype(int)
)

# final compact and visual display table
scorecard = summary[[
    "alpha", "asset", "method",
    "Kupiec", "Chris-CC", "Z1 (one-sided)",
    "score_total", "tests_counted", "N_obs_z1"
]]

scorecard



# %%
display(scorecard.loc[:, ["alpha","asset","method","score_total"]])
display(scorecard.pivot_table(index=["alpha","asset","method"], values="score_total", aggfunc="first", sort=False))

# %% [markdown]
# ## 4) Copula fitting (first window)

# %% [markdown]
# 1. Visualize dependence in returns and copula space using pseudo-observations.
# 2. Fit Gaussian and t copulas; report parameters.
# 3. Simulate from fitted copulas and map to empirical marginals; compare with original returns.

# %%
from scipy.stats import norm, t as student_t
from copulae import GaussianCopula, StudentCopula
```

```python

# %%
# ======================================================
# Pseudo-observations U on a window
# ======================================================

def compute_pseudo_obs(window_ret: pd.DataFrame) -> pd.DataFrame:
    W = len(window_ret)
    U = window_ret.rank(method="average") / (W + 1)
    return U

# ======================================================
# Empirical inverse CDF
# ======================================================

def empirical_inverse_cdf_from_sorted(u: np.ndarray, data_sorted: np.ndarray) -> np.ndarray:
    """
    u: array in (0,1)
    data_sorted: sorted 1D array of returns (length n)
    """
    u = np.asarray(u, dtype=float)
    data_sorted = np.asarray(data_sorted, dtype=float)
    n = len(data_sorted)

    pos = u * (n + 1)
    pos = np.clip(pos, 1, n) - 1

    lower = np.floor(pos).astype(int)
    upper = np.ceil(pos).astype(int)
    w = pos - lower

    return (1 - w) * data_sorted[lower] + w * data_sorted[upper]


def fit_gauss_t_copulas_copulae(U: pd.DataFrame, df_init: float = 10.0):
    d = U.shape[1]
    gauss_cop = GaussianCopula(dim=d)
    gauss_cop.fit(U.values)

    t_cop = StudentCopula(dim=d, df=df_init)
    t_cop.fit(U.values)

    R_gauss = np.asarray(gauss_cop.sigma, dtype=float)
    R_t = np.asarray(t_cop.sigma, dtype=float)
    nu_t = float(t_cop.params.df)

    return gauss_cop, t_cop, R_gauss, R_t, nu_t

# ======================================================
# like before
# ======================================================

def forecast_copula_portfolio(window_ret: pd.DataFrame,
                              alpha: float,
                              copula_kind: str,
                              N: int = 1000,
```

```
                                seed: int = 123,
                                quantile_method: str = "weibull"):
    assets = list(window_ret.columns)
    d = len(assets)

    U = compute_pseudo_obs(window_ret)

    gauss_cop, t_cop, R_gauss, R_t, nu_t = fit_gauss_t_copulas_copulae(
        U)

    copula_kind = copula_kind.lower()
    if copula_kind in ["gaussian", "gauss"]:
        sigma = R_gauss
        df = None
        U_sim = np.asarray(gauss_cop.random(int(N)), dtype=float)
    elif copula_kind in ["student", "t", "student-t", "t-copula"]:
        sigma = R_t
        df = float(nu_t)
        U_sim = np.asarray(t_cop.random(int(N)), dtype=float)
    else:
        raise ValueError("copula_kind must be 'gaussian' or 'student'."
            )

    eps = 1e-12
    U_sim = np.clip(U_sim, eps, 1.0 - eps)

    # Storring sorted marginals (empirical inverse CDF)
    data_sorted = {}
    for a in assets:
        data_sorted[a] = np.sort(window_ret[a].values.astype(float))

    # Transforming to returns (empirical inverse)
    R_sim = np.zeros((int(N), d), dtype=float)
    for j, a in enumerate(assets):
        R_sim[:, j] = empirical_inverse_cdf_from_sorted(U_sim[:, j],
            data_sorted[a])

    R_port_sim = R_sim.mean(axis=1)
    L_port_sim = -R_port_sim

    var = float(np.quantile(L_port_sim, alpha, method=quantile_method))
    es = tail_mean_es(L_port_sim, var)

    params = {
        "assets": assets,
        "copula_kind": "gaussian" if df is None else "student",
        "sigma": np.asarray(sigma, dtype=float),
        "df": df,
        "data_sorted": data_sorted
    }
    return float(var), float(es), params

def simulate_copula_portfolio(params: dict, size: int, rng: np.random.
    Generator):
    assets = params["assets"]
    sigma = np.asarray(params["sigma"], dtype=float)
    df = params.get("df", None)
    data_sorted = params["data_sorted"]
```

```python

    d = len(assets)
    n = int(size)

    # Cholesky
    try:
        C = np.linalg.cholesky(sigma)
    except np.linalg.LinAlgError:
        C = np.linalg.cholesky(sigma + 1e-10 * np.eye(d))

    Z = rng.standard_normal((n, d)) @ C.T

    eps = 1e-12
    if df is None:
        U = norm.cdf(Z)
    else:
        df = float(df)
        S = rng.chisquare(df, size=n) / df
        T = Z / np.sqrt(S)[:, None]
        U = student_t.cdf(T, df=df)

    U = np.clip(U, eps, 1.0 - eps)

    R = np.zeros((n, d), dtype=float)
    for j, a in enumerate(assets):
        R[:, j] = empirical_inverse_cdf_from_sorted(U[:, j], data_sorted[a])

    R_port = R.mean(axis=1)
    return -R_port


# %%
def build_rolling_forecasts_portfolio_copula(log_ret: pd.DataFrame,
                                             assets: list,
                                             W: int,
                                             alphas: list,
                                             copula_kind: str,
                                             N: int = 1000,
                                             seed: int = 123,
                                             quantile_method: str = "weibull",
                                             store_params: bool = True):
    """
    Rolling 1-step ahead VaR/ES forecasts for an equal-weighted portfolio
    using Gaussian or Student-t copula fit on component returns.

    Returns a DataFrame with one row per (date, alpha), aligned with your Q3 format.
    """
    ret_mat = log_ret[assets].dropna(how="any")  # aligned component returns
    n = len(ret_mat)
    if n <= W:
        raise ValueError(f"Not enough aligned data: need > W={W}, got {n}.")
```

```python

    method_name = "Copula-Gaussian" if copula_kind.lower() in ["gaussian", "gauss"] else "Copula-Student"
    rows = []

    # end is "t" index position, we forecast for end+1
    for end in range(W - 1, n - 1):
        window_ret = ret_mat.iloc[end - W + 1: end + 1]  # W x 3, dates up to t
        next_date = ret_mat.index[end + 1]               # date of t+1

        # realized portfolio return at t+1 (equal weights)
        R_real = float(ret_mat.iloc[end + 1].mean())
        L_real = float(-R_real)

        for alpha in alphas:
            var, es, params = forecast_copula_portfolio(
                window_ret=window_ret,
                alpha=float(alpha),
                copula_kind=copula_kind,
                N=int(N),
                seed=int(seed + end),                # like for unique asset data analysis : deterministic across windows
                quantile_method=quantile_method
            )

            rows.append({
                "date": next_date,
                "asset": "PORT",
                "method": method_name,
                "alpha": float(alpha),
                "VaR": float(var),
                "ES": float(es),
                "R_real": R_real,
                "L_real": L_real,
                "breach": int(L_real > var),
                "params": params if store_params else None
            })

    df = pd.DataFrame(rows).sort_values(["alpha", "date"]).reset_index(drop=True)
    return df


# %% [markdown]
# ## 4.a)

# %%
import itertools

# ============================================
# Settings and data selection
# ============================================

W = 252
assets = ["AAPL", "META", "JPM"]

R_window = log_ret[assets].dropna(how="any").iloc[:W]    # (W, 3)
```

```python

# ==========================================
# Pseudo-observations U
# ==========================================

U = R_window.rank(method="average") / (W + 1)

print("Summary of pseudo-observations U:")
display(U.describe())

# quantile thresholds for visual tail diagnostics
q_low, q_high = 0.05, 0.95

# ==========================================
# Scatter plots (R vs U) for each pair
# ==========================================

for i, j in itertools.combinations(assets, 2):
    fig, axes = plt.subplots(1, 2, figsize=(11, 4))

    # Left : raw returns space
    axes[0].scatter(R_window[i], R_window[j], s=12, alpha=0.6)
    axes[0].set_xlabel(f"{i} log-return")
    axes[0].set_ylabel(f"{j} log-return")
    axes[0].set_title(f"Raw log-returns: {i} vs {j}")
    axes[0].grid(True)

    # showing 5%/95% lines in return space (to separate tails)
    xi_low, xi_high = R_window[i].quantile(q_low), R_window[i].quantile(q_high)
    xj_low, xj_high = R_window[j].quantile(q_low), R_window[j].quantile(q_high)
    axes[0].axvline(xi_low, linestyle="--", linewidth=1)
    axes[0].axvline(xi_high, linestyle="--", linewidth=1)
    axes[0].axhline(xj_low, linestyle="--", linewidth=1)
    axes[0].axhline(xj_high, linestyle="--", linewidth=1)

    # Right: pseudo-observations space
    axes[1].scatter(U[i], U[j], s=12, alpha=0.6)
    axes[1].set_xlabel(f"{i} pseudo-observations U")
    axes[1].set_ylabel(f"{j} pseudo-observations U")
    axes[1].set_title(f"Pseudo-observations: {i} vs {j}")
    axes[1].grid(True)

    # showing tail regions again (in U-space)
    axes[1].axvline(q_low, linestyle="--", linewidth=1)
    axes[1].axvline(q_high, linestyle="--", linewidth=1)
    axes[1].axhline(q_low, linestyle="--", linewidth=1)
    axes[1].axhline(q_high, linestyle="--", linewidth=1)
    axes[1].set_xlim(0, 1)
    axes[1].set_ylim(0, 1)

    plt.tight_layout()
    plt.show()


# %% [markdown]
# ## 4.b)
```

```python

# %%
# ---------------------------------------------
# Fitting copulas on the single estimation window
# ---------------------------------------------
W = 252
assets = ["AAPL", "META", "JPM"]

R_window = log_ret[assets].dropna(how="any").iloc[:W]
U = compute_pseudo_obs(R_window)

gauss_cop, t_cop, R_gauss, R_t, nu_t = fit_gauss_t_copulas_copulae(U)

print("Gaussian copula sigma (correlation matrix):")
print(pd.DataFrame(R_gauss, index=assets, columns=assets))
print("\nStudent-t copula sigma (correlation matrix):")
print(pd.DataFrame(R_t, index=assets, columns=assets))
print(f"\nStudent-t copula df (nu): {nu_t:.4f}")

print("\nGaussian log-likelihood:", gauss_cop.log_lik(U.values))
print("Student-t log-likelihood:", t_cop.log_lik(U.values))

# ---------------------------------------------
# Plotting the parameter matrices (heatmaps)
# ---------------------------------------------
fig, axes = plt.subplots(1, 2, figsize=(10, 4))

im0 = axes[0].imshow(R_gauss, vmin=-1, vmax=1)
axes[0].set_title("Gaussian copula: Sigma")
axes[0].set_xticks(range(len(assets)))
axes[0].set_yticks(range(len(assets)))
axes[0].set_xticklabels(assets)
axes[0].set_yticklabels(assets)

# annotatting values
for i in range(len(assets)):
    for j in range(len(assets)):
        axes[0].text(j, i, f"{R_gauss[i, j]:.2f}", ha="center", va="center")

im1 = axes[1].imshow(R_t, vmin=-1, vmax=1)
axes[1].set_title(f"Student-t copula: Sigma (df={nu_t:.2f})")
axes[1].set_xticks(range(len(assets)))
axes[1].set_yticks(range(len(assets)))
axes[1].set_xticklabels(assets)
axes[1].set_yticklabels(assets)

# annotatting values
for i in range(len(assets)):
    for j in range(len(assets)):
        axes[1].text(j, i, f"{R_t[i, j]:.2f}", ha="center", va="center")


# %% [markdown]
# ## 4.c)

# %%
```

```python
def simulate_assets_from_fitted_copula_single_window(log_ret: pd.DataFrame,
                                                     assets: list,
                                                     W: int,
                                                     copula_kind: str,
                                                     T: int,
                                                     seed: int = 123):
    """
    Fit the copula on the first window of length W (as in Q4),
    then simulate T synthetic returns for the d assets.

    Returns:
      R_sim_df: DataFrame (T x d) of simulated returns
      R_port_sim: pd.Series (T,) equal-weight portfolio returns (optional but handy)
      fitted_params: dict with sigma/df used (for reporting)
    """
    # estimation window
    R_window = log_ret[assets].dropna(how="any").iloc[:W]
    U = compute_pseudo_obs(R_window)

    # fitting copulas
    gauss_cop, t_cop, R_gauss, R_t, nu_t = fit_gauss_t_copulas_copulae(U)

    copula_kind = copula_kind.lower()
    rng = np.random.default_rng(int(seed))

    if copula_kind in ["gaussian", "gauss"]:
        U_sim = np.asarray(gauss_cop.random(int(T)), dtype=float)
        sigma = np.asarray(R_gauss, dtype=float)
        df = None
    elif copula_kind in ["student", "t", "student-t", "t-copula"]:
        U_sim = np.asarray(t_cop.random(int(T)), dtype=float)
        sigma = np.asarray(R_t, dtype=float)
        df = float(nu_t)
    else:
        raise ValueError("copula_kind must be 'gaussian' or 'student'.")

    # (avoid exact 0/1) ***
    eps = 1e-12
    U_sim = np.clip(U_sim, eps, 1.0 - eps)

    # inverse ECDF from window (sorted marginals)
    data_sorted = {a: np.sort(R_window[a].values.astype(float)) for a in assets}

    # transforming Us to returns ---
    R_sim = np.zeros((int(T), len(assets)), dtype=float)
    for j, a in enumerate(assets):
        R_sim[:, j] = empirical_inverse_cdf_from_sorted(U_sim[:, j], data_sorted[a])

    R_sim_df = pd.DataFrame(R_sim, columns=assets)
    R_port_sim = R_sim_df.mean(axis=1)   # equal-weight portfolio return (<=> 1/3 each)

```

```
    fitted_params = {"sigma": sigma, "df": df, "nu_t": df}

    return R_sim_df, R_port_sim, fitted_params


# %%
# ============================================
# Q4(c) run
# ============================================
assets = ["AAPL", "META", "JPM"]
W = 252
log_ret_aligned = log_ret[assets].dropna(how="any")

T_total = len(log_ret_aligned) # (total number of observations)
print("T_total =", T_total)

R_sim_g, Rp_sim_g, params_g = simulate_assets_from_fitted_copula_single_window(
    log_ret=log_ret,
    assets=assets,
    W=W,
    copula_kind="gaussian",
    T=T_total,
    seed=123
)

R_sim_t, Rp_sim_t, params_t = simulate_assets_from_fitted_copula_single_window(
    log_ret=log_ret,
    assets=assets,
    W=W,
    copula_kind="student",
    T=T_total,
    seed=123
)

print("Gaussian copula fitted sigma:\n", pd.DataFrame(params_g["sigma"], index=assets, columns=assets))
print("\nStudent-t copula fitted sigma:\n", pd.DataFrame(params_t["sigma"], index=assets, columns=assets))
print("\nStudent-t df:", params_t["df"])

# ============================================
# Visual comparisons: (histograms and scatter)
# ============================================

# comparison of marginal distributions (histograms)
for a in assets:
    plt.figure(figsize=(10, 6))
    plt.hist(log_ret_aligned[a].values, bins=60, alpha=0.5, density=True, log=True, label="Real")
    plt.hist(R_sim_g[a].values, bins=60, alpha=0.5, density=True, log=True, label="Sim (Gauss copula)")
    plt.hist(R_sim_t[a].values, bins=60, alpha=0.5, density=True, log=True, label="Sim (t copula)")
    plt.title(f"Marginal distribution: {a}")
    plt.legend()
    plt.tight_layout()
```

```
    plt.grid(True)
    plt.show()

# comparison of dependence on simulated data vs real data (scatter)
for i, j in itertools.combinations(assets, 2):
    fig, axes = plt.subplots(1, 3, figsize=(13, 4))

    axes[0].scatter(log_ret_aligned[i], log_ret_aligned[j], s=10, alpha
        =0.5)
    axes[0].set_title(f"Real: {i} vs {j}")
    axes[0].set_xlabel(i); axes[0].set_ylabel(j)
    axes[0].grid(True)

    axes[1].scatter(R_sim_g[i], R_sim_g[j], s=10, alpha=0.5)
    axes[1].set_title(f"Sim Gauss copula: {i} vs {j}")
    axes[1].set_xlabel(i); axes[1].set_ylabel(j)
    axes[1].grid(True)

    axes[2].scatter(R_sim_t[i], R_sim_t[j], s=10, alpha=0.5)
    axes[2].set_title(f"Sim t copula: {i} vs {j}")
    axes[2].set_xlabel(i); axes[2].set_ylabel(j)
    axes[2].grid(True)

    plt.tight_layout()
    plt.show()

for i, j in itertools.combinations(assets, 2):
    plt.figure(figsize=(6.5, 5))

    # Real
    plt.scatter(
        log_ret_aligned[i], log_ret_aligned[j],
        s=10, alpha=0.35, label="Real"
    )

    # Sim Gaussian copula
    plt.scatter(
        R_sim_g[i], R_sim_g[j],
        s=10, alpha=0.35, label="Sim Gauss copula"
    )

    # Sim Student-t copula
    plt.scatter(
        R_sim_t[i], R_sim_t[j],
        s=10, alpha=0.35, label="Sim t copula"
    )

    plt.title(f"{i} vs {j}     Real vs Copula simulations")
    plt.xlabel(i)
    plt.ylabel(j)
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()

# comparison btw portfolio total distribution, not just marginals
Rp_real = log_ret_aligned.mean(axis=1).values

```

```
plt.figure(figsize=(10, 6))
plt.hist(Rp_real, bins=60, alpha=0.5, density=True, log=True, label="Real portfolio")
plt.hist(Rp_sim_g.values, bins=60, alpha=0.5, density=True, log=True, label="Sim portfolio (Gauss copula)")
plt.hist(Rp_sim_t.values, bins=60, alpha=0.5, density=True, log=True, label="Sim portfolio (t copula)")
plt.title("Portfolio return distribution (equal-weighted)")
plt.legend()
plt.tight_layout()
plt.grid(True)
plt.show()


# %% [markdown]
# ## 5) Portfolio VaR/ES with copulas (rolling)

# %% [markdown]
# Equal-weighted portfolio of AAPL, MSFT, JPM. Compare univariate models (as in Exercise 3) vs copula-based VaR/ES with rolling windows.
#
# At each time, fit copulas on last W days, simulate N scenarios, estimate VaR/ES from simulated portfolio returns, then backtest.

# %% [markdown]
# ## 5.a)

# %%
# at this point, we have all the necessary routines, needed for a complete implemented

# ----------------------------
# 5(a) Portfolio returns/losses
# ----------------------------
assets = ["AAPL", "META", "JPM"]
ticker_port = "PORT" # "PORT" meaning portfolio

# Equal-weight portfolio return series (aligned dates)
rp = log_ret[assets].dropna(how="any").mean(axis=1)
log_ret_port = pd.DataFrame({ticker_port: rp})

# ----------------------------
# p_matrix for PORT (again, p=0 everywhere)
# we add a PORT row with 0 for the same columns as p_matrix
# ----------------------------
p_matrix_port = p_matrix.copy()

if ticker_port not in p_matrix_port.index:
    p_matrix_port.loc[ticker_port, :] = 0

# ----------------------------
# Rolling forecasts on PORT using the same routine as in 3)
# ----------------------------
W = 252
conf_level = [0.95, 0.99]

df_port_forecasts = build_rolling_forecasts(
```

```
    log_ret=log_ret_port,
    TICKERS=[ticker_port],
    conf_level=conf_level,
    W=W,
    p_matrix=p_matrix_port,
    scale=100.0,
    M_fhs=20000,
    seed_fhs=42,
    quantile_method="weibull",
    store_params=True
)

print(df_port_forecasts.head())
print("df_port_forecasts shape:", df_port_forecasts.shape)
print("Methods:", df_port_forecasts["method"].unique())

# %%
# plotting VaR forecasts along with the realized PORT returns (from
    df_port_forecasts)
methods = ["HS", "Gaussian", "Student-t", "AR-GARCH", "FHS"]
conf_level_bt = [0.95, 0.99]

ticker_port = "PORT"  # as above

# here we plot -VaR forecasts along with returns
# since VaR_t are compiuted on losses (=-returns)
for alpha_bt in conf_level_bt:
    plt.figure(figsize=(11, 4))

    sub0 = df_port_forecasts[
        (df_port_forecasts["asset"] == ticker_port) &
        (df_port_forecasts["alpha"] == alpha_bt)
    ].sort_values("date")

    if len(sub0) == 0:
        continue

    # R_real is duplicated across methods; keep one point per date for
        the black line
    sub_real = sub0.drop_duplicates(subset=["date"])

    plt.plot(sub_real["date"], sub_real["R_real"], color="black",
             label="Realized Returns (PORT)", linewidth=1.2)

    for method in methods:
        sub = sub0[sub0["method"] == method].sort_values("date")
        if len(sub) == 0:
            continue
        plt.plot(sub["date"], -sub["VaR"], label=f"-VaR ({method})",
            linewidth=1.0)

    plt.title(f"{ticker_port} | alpha={alpha_bt} | VaR threshold")
    plt.legend(ncol=2)
    plt.tight_layout()
    plt.grid(True)
    plt.show()
```

```python
# %%
# ----------------------------------------------------
# Backtesting (Kupiec + Christoffersen + Z1) on PORT
# ----------------------------------------------------
kupiec_rows = []
chris_rows = []
z1_rows = []

for alpha in conf_level:
    for method in df_port_forecasts["method"].unique():
        sub = df_port_forecasts[
            (df_port_forecasts["alpha"] == alpha) &
            (df_port_forecasts["method"] == method)
        ].sort_values("date")

        breaches = sub["breach"].to_numpy(dtype=int)

        # Kupiec
        out_pof = kupiec_pof_test(breaches, alpha)
        kupiec_rows.append({
            "alpha": alpha, "asset": ticker_port, "method": method,
            **out_pof
        })

        # Christoffersen (conditional coverage)
        out_cc = christoffersen_conditional_coverage_test(breaches, alpha=alpha)
        chris_rows.append({
            "alpha": alpha,
            "asset": ticker_port,
            "method": method,
            "T": out_cc["T"],
            "N": out_cc["N"],

            # Unconditional coverage (Kupiec)
            "LR_pof": out_cc["LR_pof"],
            "pvalue_pof": out_cc["pvalue_pof"],

            # Independence
            "n00": out_cc["n00"], "n01": out_cc["n01"], "n10": out_cc["n10"], "n11": out_cc["n11"],
            "p01": out_cc["p01"], "p11": out_cc["p11"],
            "LR_ind": out_cc["LR_ind"],
            "pvalue_ind": out_cc["pvalue_ind"],

            # Joint conditional coverage
            "LR_cc": out_cc["LR_cc"],
            "pvalue_cc": out_cc["pvalue_cc"],
        })

        # Z1 (Acerbi-Szekely) needs the "simulation_*" function of our models
        L_real = sub["L_real"].to_numpy(dtype=float)
        VaR = sub["VaR"].to_numpy(dtype=float)
        ES  = sub["ES"].to_numpy(dtype=float)
        params_list = sub["params"].tolist()

        if method == "HS":
```

```
            sim_fun = simulate_hs
        elif method == "Gaussian":
            sim_fun = simulate_gaussian
        elif method == "Student-t":
            sim_fun = simulate_student_t
        elif method == "AR-GARCH":
            sim_fun = simulate_ar_garch_gaussian
        elif method == "FHS":
            sim_fun = simulate_fhs
        else:
            raise ValueError("Unknown method: " + str(method))

        out_z1 = acerbi_szekely_z1_test(
            L_real=L_real,
            VaR=VaR,
            ES=ES,
            params_list=params_list,
            simulate_one_step=sim_fun,
            M=1000,
            seed=123
        )
        z1_rows.append({
            "alpha": alpha, "asset": ticker_port, "method": method,
            **out_z1
        })

kupiec_df_port = pd.DataFrame(kupiec_rows).sort_values(["alpha"]).reset_index(drop=True)
chris_df_port  = pd.DataFrame(chris_rows).sort_values(["alpha"]).reset_index(drop=True)
z1_df_port     = pd.DataFrame(z1_rows).sort_values(["alpha"]).reset_index(drop=True)

print("\nKupiec (PORT):")
display(kupiec_df_port)

print("\nChristoffersen (PORT):")
display(chris_df_port)

print("\nZ1 (PORT):")
display(z1_df_port)

# %%
# pass/fail at 5% level
kupiec_df_port["pass_5pct"] = kupiec_df_port["pvalue_pof"] > prim_confid_value

chris_df_port["pass_cc_5pct"]  = chris_df_port["pvalue_cc"]  > prim_confid_value
chris_df_port["pass_pof_5pct"] = chris_df_port["pvalue_pof"] > prim_confid_value
chris_df_port["pass_ind_5pct"] = chris_df_port["pvalue_ind"] > prim_confid_value

z1_df_port["pass_5pct"] = z1_df_port["pvalue_one_sided"] > prim_confid_value
```

```python

# %%
display(kupiec_df_port.loc[:, ["alpha","asset","method","pass_5pct"]])
display(kupiec_df_port.pivot_table(index=["alpha","asset","method"], values="pass_5pct", aggfunc="first", sort=False))


# %%
display(chris_df_port.loc[:, ["alpha","asset","method","pass_cc_5pct"]])
display(chris_df_port.pivot_table(index=["alpha","asset","method"], values="pass_cc_5pct", aggfunc="first", sort=False))

# %%
display(z1_df_port.loc[:, ["alpha","asset","method","pass_5pct"]])
display(z1_df_port.pivot_table(index=["alpha","asset","method"], values="pass_5pct", aggfunc="first", sort=False))

# %% [markdown]
# ## Compact&Visual results of backtests

# %%
# we provide here the same visual resumed result as in part 3 fair univariate analysis

# merging tables (keep only what we need)
k_port = kupiec_df_port[["alpha", "asset", "method", "pvalue_pof", "pass_5pct"]].rename(
    columns={"pvalue_pof": "p_kupiec", "pass_5pct": "kupiec_pass"}
)

c_port = chris_df_port[["alpha", "asset", "method", "pvalue_cc", "pass_cc_5pct"]].rename(
    columns={"pvalue_cc": "p_chris_cc", "pass_cc_5pct": "chris_cc_pass"}
)

z_port = z1_df_port[["alpha", "asset", "method", "pvalue_one_sided", "pass_5pct", "N_obs"]].rename(
    columns={"pvalue_one_sided": "p_z1", "pass_5pct": "z1_pass", "N_obs": "N_obs_z1"}
)

summary_port = (
    k_port.merge(c_port, on=["alpha", "asset", "method"], how="outer")
     .merge(z_port, on=["alpha", "asset", "method"], how="outer")
)

# color cells to display visually
def fmt_cell(p, passed, label="p"):
    if pd.isna(passed):
        return "    (inconcl.)"
    if pd.isna(p):
        return "    (p=NaN)"
    return ("     " if bool(passed) else "     ") + f"({label}={p:.3f})"

```

```
summary_port["Kupiec"] = [fmt_cell(p, b, "p") for p, b in zip(
    summary_port["p_kupiec"], summary_port["kupiec_pass"])]
summary_port["Chris-CC"] = [fmt_cell(p, b, "p") for p, b in zip(
    summary_port["p_chris_cc"], summary_port["chris_cc_pass"])]
summary_port["Z1 (one-sided)"] = [fmt_cell(p, b, "p") for p, b in zip(
    summary_port["p_z1"], summary_port["z1_pass"])]

# simple score (count only available tests)
summary_port["score_total"] = (
    summary_port["kupiec_pass"].fillna(False).astype(int) +
    summary_port["chris_cc_pass"].fillna(False).astype(int) +
    summary_port["z1_pass"].fillna(0).astype(int)
)

summary_port["tests_counted"] = (
    summary_port["kupiec_pass"].notna().astype(int) +
    summary_port["chris_cc_pass"].notna().astype(int) +
    summary_port["z1_pass"].notna().astype(int)
)

# final compact display and visual table
scorecard_port = summary_port[[
    "alpha", "asset", "method",
    "Kupiec", "Chris-CC", "Z1 (one-sided)",
    "score_total", "tests_counted", "N_obs_z1"
]]

scorecard_port



# %% [markdown]
# ## 5.b),c)

# %%
# ==========================================
# Rolling copula forecasts on portfolio
# ==========================================

assets = ["AAPL", "META", "JPM"]
W = 252
conf_level = [0.95, 0.99]
N_cop = 1000
seed_cop = 123

# Rolling forecasts for Gaussian copula
df_cop_gauss = build_rolling_forecasts_portfolio_copula(
    log_ret=log_ret,
    assets=assets,
    W=W,
    alphas=conf_level,
    copula_kind="gaussian",
    N=N_cop,
    seed=seed_cop,
    quantile_method="weibull",
    store_params=True
)

```

```python
# Rolling forecasts for Student-t copula
df_cop_student = build_rolling_forecasts_portfolio_copula(
    log_ret=log_ret,
    assets=assets,
    W=W,
    alphas=conf_level,
    copula_kind="student",
    N=N_cop,
    seed=seed_cop,
    quantile_method="weibull",
    store_params=True
)

# Merge the two copula outputs
df_cop_forecasts = pd.concat([df_cop_gauss, df_cop_student],
    ignore_index=True)
df_cop_forecasts.sort_values(["alpha"], inplace=True)
df_cop_forecasts.reset_index(drop=True, inplace=True)

print(df_cop_forecasts.head())
print("df_cop_forecasts shape:", df_cop_forecasts.shape)
print("Copula methods:", df_cop_forecasts["method"].unique())

# %%
# ==========================================
# Backtesting on copula forecasts (PORT)
# Kupiec, Christoffersen and Z1
# ==========================================

kupiec_rows_cop = []
chris_rows_cop = []
z1_rows_cop = []

for alpha in conf_level:
    for method in df_cop_forecasts["method"].unique():
        sub = df_cop_forecasts[
            (df_cop_forecasts["alpha"] == alpha) &
            (df_cop_forecasts["method"] == method)
        ].sort_values("date")

        breaches = sub["breach"].to_numpy(dtype=int)

        # Kupiec
        out_pof = kupiec_pof_test(breaches, alpha)
        kupiec_rows_cop.append({
            "asset": "PORT", "method": method, "alpha": alpha,
            **out_pof
        })

        # Christoffersen
        out_cc = christoffersen_conditional_coverage_test(breaches,
            alpha=alpha)
        chris_rows_cop.append({
            "asset": "PORT",
            "method": method,
            "alpha": alpha,
            "T": out_cc["T"],
            "N": out_cc["N"],
```

```python

            "LR_pof": out_cc["LR_pof"],
            "pvalue_pof": out_cc["pvalue_pof"],

            "n00": out_cc["n00"], "n01": out_cc["n01"], "n10": out_cc["n10"], "n11": out_cc["n11"],
            "p01": out_cc["p01"], "p11": out_cc["p11"],
            "LR_ind": out_cc["LR_ind"],
            "pvalue_ind": out_cc["pvalue_ind"],
            "LR_cc": out_cc["LR_cc"],
            "pvalue_cc": out_cc["pvalue_cc"],
        })

        # Z1 ( A c e r b i Szekely ) for ES
        L_real = sub["L_real"].to_numpy(dtype=float)
        VaR = sub["VaR"].to_numpy(dtype=float)
        ES  = sub["ES"].to_numpy(dtype=float)
        params_list = sub["params"].tolist()

        # for both copula methods, same simulate function:
        sim_fun = simulate_copula_portfolio

        out_z1 = acerbi_szekely_z1_test(
            L_real=L_real,
            VaR=VaR,
            ES=ES,
            params_list=params_list,
            simulate_one_step=sim_fun,
            M=1000,
            seed=123
        )

        z1_rows_cop.append({
            "asset": "PORT", "method": method, "alpha": alpha,
            **out_z1
        })

kupiec_df_cop = pd.DataFrame(kupiec_rows_cop).sort_values(["alpha"]).reset_index(drop=True)
chris_df_cop  = pd.DataFrame(chris_rows_cop).sort_values(["alpha"]).reset_index(drop=True)
z1_df_cop     = pd.DataFrame(z1_rows_cop).sort_values(["alpha"]).reset_index(drop=True)

print("\nKupiec (COPULA PORTFOLIO):")
display(kupiec_df_cop)

print("\nChristoffersen (COPULA PORTFOLIO):")
display(chris_df_cop)

print("\nZ1 (COPULA PORTFOLIO):")
display(z1_df_cop)

# %%
# pass/fail at 5% level
kupiec_df_cop["pass_5pct"] = kupiec_df_cop["pvalue_pof"] > prim_confid_value

```

```
chris_df_cop["pass_cc_5pct"]  = chris_df_cop["pvalue_cc"]  > prim_confid_value
chris_df_cop["pass_pof_5pct"] = chris_df_cop["pvalue_pof"] > prim_confid_value
chris_df_cop["pass_ind_5pct"] = chris_df_cop["pvalue_ind"] > prim_confid_value

z1_df_cop["pass_5pct"] = z1_df_cop["pvalue_one_sided"] > prim_confid_value

# %%
display(kupiec_df_cop.loc[:, ["alpha","asset","method","pass_5pct"]])
display(kupiec_df_cop.pivot_table(index=["alpha","asset","method"], values="pass_5pct", aggfunc="first", sort=False))


# %%
display(chris_df_cop.loc[:, ["alpha","asset","method","pass_cc_5pct"]])
display(chris_df_cop.pivot_table(index=["alpha","asset","method"], values="pass_cc_5pct", aggfunc="first", sort=False))

# %%
display(z1_df_cop.loc[:, ["alpha","asset","method","pass_5pct"]])
display(z1_df_cop.pivot_table(index=["alpha","asset","method"], values="pass_5pct", aggfunc="first", sort=False))

# %%
# we provide here the same visual resumed result as in part 3 fair univariate analysis

# merging tables (keep only what we need)
k_cop = kupiec_df_cop[["alpha", "asset", "method", "pvalue_pof", "pass_5pct"]].rename(
    columns={"pvalue_pof": "p_kupiec", "pass_5pct": "kupiec_pass"}
)

c_cop = chris_df_cop[["alpha", "asset", "method", "pvalue_cc", "pass_cc_5pct"]].rename(
    columns={"pvalue_cc": "p_chris_cc", "pass_cc_5pct": "chris_cc_pass"}
)

z_cop = z1_df_cop[["alpha", "asset", "method", "pvalue_one_sided", "pass_5pct", "N_obs"]].rename(
    columns={"pvalue_one_sided": "p_z1", "pass_5pct": "z1_pass", "N_obs": "N_obs_z1"}
)

summary_cop = (
    k_cop.merge(c_cop, on=["alpha", "asset", "method"], how="outer")
     .merge(z_cop, on=["alpha", "asset", "method"], how="outer")
)

# color cells to display visually
def fmt_cell(p, passed, label="p"):
    if pd.isna(passed):
        return "    (inconcl.)"
    if pd.isna(p):
```

```
        return "    (p=NaN)"
    return ("     " if bool(passed) else "     ") + f"({label}={p:.3f})"

summary_cop["Kupiec"] = [fmt_cell(p, b, "p") for p, b in zip(summary_cop["p_kupiec"], summary_cop["kupiec_pass"])]
summary_cop["Chris-CC"] = [fmt_cell(p, b, "p") for p, b in zip(summary_cop["p_chris_cc"], summary_cop["chris_cc_pass"])]
summary_cop["Z1 (one-sided)"] = [fmt_cell(p, b, "p") for p, b in zip(summary_cop["p_z1"], summary_cop["z1_pass"])]

# simple score (count only available tests)
summary_cop["score_total"] = (
    summary_cop["kupiec_pass"].fillna(False).astype(int) +
    summary_cop["chris_cc_pass"].fillna(False).astype(int) +
    summary_cop["z1_pass"].fillna(0).astype(int)
)

summary_cop["tests_counted"] = (
    summary_cop["kupiec_pass"].notna().astype(int) +
    summary_cop["chris_cc_pass"].notna().astype(int) +
    summary_cop["z1_pass"].notna().astype(int)
)

# final compact & visual display table
scorecard_cop = summary_cop[[
    "alpha", "asset", "method",
    "Kupiec", "Chris-CC", "Z1 (one-sided)",
    "score_total", "tests_counted", "N_obs_z1"
]]

scorecard_cop
```